

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

MSSV 21207001

Bùi Thành Đạt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN

Open-Source RISC-V SoC Implementation
with DNN Acceleration: A Chipyard Framework Study
with FPGA Prototyping

NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

Tp. Hồ Chí Minh, tháng 07/2025

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

MSSV 21207001

Bùi Thành Đạt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN

Open-Source RISC-V SoC Implementation
with DNN Acceleration: A Chipyard Framework Study with FPGA Prototyping

NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

GIÁO VIÊN HƯỚNG DẪN

TS. Huỳnh Hữu Thuận

Tp. Hồ Chí Minh, tháng 07/2025

XÁC NHẬN HOÀN TẤT CHỈNH SỬA BÁO CÁO TỐT NGHIỆP

TÊN ĐỀ TÀI: .

Tên tiếng anh:

Họ và tên sinh viên:

Mã số sinh viên:

Hội đồng đánh giá họp ngày..... tháng năm.....

1. Chủ tịch:

2. Thư ký:

3. Ủy viên:

4. Ủy viên:

5. Ủy viên:

Người hướng dẫn: TS. Huỳnh Hữu Thuận

Xác nhận của người hướng dẫn:

LỜI CẢM ƠN

Thành phố Hồ Chí Minh, ngày 2 tháng 7 năm 2025
Sinh viên thực hiện

Bùi Thành Đạt

TRANG CAM KẾT TÍNH TRUNG THỰC CỦA BÁO CÁO

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

TÓM TẮT

ABSTRACT

The increasing computational demand of Deep Neural Networks (DNNs) has exposed the limitations of traditional general-purpose processors, creating a need for specialized hardware accelerators. While FPGAs have served as a prototyping platform, transitioning to more efficient and scalable Application-Specific Integrated Circuit (ASIC) solutions requires a comprehensive, open-source design methodology. This thesis presents the design, full-stack implementation, and evaluation of a Linux-capable RISC-V System-on-Chip (SoC) featuring a dedicated DNN accelerator. The primary objective is to demonstrate a complete open-source workflow, from high-level hardware configuration in Chisel to FPGA prototyping. The system was designed using the Chipyard framework, integrating a 64-bit RISC-V Rocket Core with the Gemmini systolic array accelerator via the RoCC interface. The implemented SoC was successfully deployed on a Xilinx VC707 FPGA board, booting a full Debian Linux operating system. Performance evaluation revealed that the Gemmini accelerator achieved an approximate **958x speedup** on a representative convolution task compared to a software-only implementation on the Rocket Core. This work successfully validates the Chipyard framework as a viable path for advanced SoC design and establishes a reproducible, full-stack foundation for future research in specialized hardware acceleration.

TABLE OF CONTENTS

LỜI CẢM ƠN	iii
TÓM TẮT	vii
ABSTRACT	ix
MỤC LỤC	xi
LIST OF FIGURES	xv
LIST OF TABLES	xix
1 Introduction	1
1.1 Background and Motivation	1
1.2 The Chipyard Framework and Full-Stack Integration	2
1.3 Research Objectives and Contributions	3
1.4 Significance and Impact	4
1.5 Thesis Structure and Organization	5
2 RISC-V Microprocessor Instruction Set Architecture	7
2.1 Overview of RISC-V	7
2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set	9
2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set	13
2.2 RISC-V Processor Cores	16
2.2.1 Pipeline Architecture in Microprocessors	17
2.2.2 Out-of-Order Architecture in Microprocessors	18
2.2.3 Open-Source RISC-V Processor Cores	19
2.3 System-on-Chip Design Framework: Chipyard - Rocket Chip	21
3 A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System	25
3.1 The Chipyard Compilation and Elaboration Flow	25
3.2 System Design Methodology with Chisel	27
3.2.1 Chisel and Scala	27
3.2.2 Verilator Simulation Tool	29

3.3	The Rocket Processor Core	29
3.4	Rocket Chip SoC Structure	30
3.4.1	Bus and Interconnect	31
3.4.2	Memory and Cache	35
3.4.3	Core-Complex	36
3.4.4	Peripheral Components	37
4	Architectural Foundations of DNN Acceleration	41
4.1	The Data-Compute Divide: The Memory Wall	41
4.2	A Classic Solution: The Systolic Principle	42
4.3	Case Studies in Modern Systolic Design	44
4.3.1	Google’s TPUv1: A Datacenter-Scale Design	44
4.3.2	NVIDIA NVDLA: An Open-Source Accelerator IP	44
4.4	A Formal Framework: Dataflows	46
4.5	The Gemmini Approach: A Full-Stack Research Platform	49
4.6	Chapter Summary	50
5	The Gemmini DNN Accelerator: An Architectural Overview	51
5.1	Top-Level Architectural Blueprint	51
5.2	Analysis of Core Architectural Components	51
5.2.1	The Systolic Compute Engine	51
5.2.2	The On-Chip Memory Subsystem	54
5.3	SoC Integration and System-Level Features	55
5.3.1	The RoCC Coprocessor Interface	55
5.3.2	Virtual Memory Support	55
5.4	Summary	56
6	Gemmini’s Internal Architecture and Programming Model	57
6.1	Internal Command and Control Flow	57
6.1.1	Command Unrolling and Buffering for Decoupled Execution via LoopConv and LoopMatmul	58
6.1.2	Reservation Station and Execution Control	62
6.1.3	Scratchpad: the Local Memory Subsystem	65
6.1.4	Mesh: the Systolic Array	67
6.2	The RoCC Interface that controls Gemmini	67
6.2.1	Resource Management and Arbitration	67

6.3	The Multi-Level Programming Model	71
6.3.1	Low-Level: Gemmini ISA	71
6.3.2	Mid-Level: Hand-Tuned C Library	71
6.3.3	High-Level: Compiler Support (Conceptual)	71
7	System Implementation and Testing	73
7.1	Hardware Implementation: SoC Configuration	73
7.1.1	The Rocket64b1gem16jtag Configuration	75
7.2	Gemmini Integration via RoCC	77
7.2.1	The RoCC Interface	77
7.3	Peripheral Integration	80
7.3.1	SD Card: AXI SD Card Controller and SD Mode	81
7.3.2	UART: AXI UART Controller	82
7.3.3	Ethernet: AXI Ethernet Controller	82
7.4	Development Environment	83
7.5	Software Stack and Build Process	83
7.5.1	Boot Flow: OpenSBI, U-Boot, and Linux	83
7.5.2	SoC Hardware Generation	84
7.6	Gemmini Testing	85
8	Implementation Results and Discussion	87
8.1	Hardware Implementation Analysis	87
8.1.1	System Block Design and Address Map	87
8.1.2	Vivado Implemented Design Results	88
8.2	Software Stack Validation	96
8.2.1	System Boot Sequence	96
8.2.2	Debian Linux Environment	97
8.3	Gemmini Accelerator Performance Evaluation	98
8.3.1	Functional Correctness	98
8.3.2	Performance Speedup	99
9	Conclusion and Future Work	101
	DANH MỤC CÔNG TRÌNH CỦA TÁC GIẢ	103
	TÀI LIỆU THAM KHẢO	105

LIST OF FIGURES

2.1	RISC-V is managed by RISC-V International. Source [17]	7
2.2	Instruction encoding formats in RV32I. Source [22]	10
2.3	Instructions in the RV32I base instruction set. [22]	12
2.4	Instructions in the RV64I base instruction set. [22]	14
2.5	Instruction encoding format for ADDIW in 64-bit architecture. [22] . .	14
2.6	Encoding format for 64-bit I-type shift instructions. [22]	15
2.7	Encoding format for 64-bit R-type shift instructions for word operations. [22]	15
2.8	Encoding formats for 64-bit LOAD and STORE instructions. [22] . . .	16
2.9	Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch.	17
2.10	Pipeline of Rocket core. Source [2]	19
2.11	Berkeley Out-of-Order Machine Processor Pipeline Architecture. [6] . .	20
2.12	Chipyard, developed by UC Berkeley. Source: [24]	22
2.13	Component structure of Chipyard. Source: [24]	22
3.1	The Chipyard Development and Compilation Flow	26
3.2	Chisel as a library within Scala.	27
3.3	The Chisel to HDL design elaboration flow	28
3.4	Structure of a Rocket Tile.	30
3.5	Proposed structure of a single-core RISC-V processor system with a Gemmini RoCC accelerator.	31
3.6	Connection model in TileLink and its interfaces. Source: [19]	32
3.7	Channels in a TileLink connection between two agents. Source: [19] . .	33
3.8	TileLink TL-UL connection structure. Source: [19]	35
3.9	Implementation structure of a heterogeneous multi-core processor system.	36
3.10	Peripherals in the Rocket Chip base system.	37
3.11	RoCC co-processor core and MMIO peripheral core in Rocket Chip. . .	38
4.1	The Relative Energy Cost of Computation vs. Memory Access	42
4.2	Conventional vs. Systolic Processing	43
4.3	The Google TPuv1 Block Diagram	45

4.4	The NVDLA High-Level Architecture	45
4.5	Data Reuse Opportunities in DNNs	46
4.6	Taxonomy of Common DNN Dataflows	47
4.7	Comparison of OS and WS Dataflows	48
4.8	Architectural Flexibility of the Gemmini Generator	50
5.1	Gemmini Logo. Source: [10].	51
5.2	High-level overview of the Gemmini accelerator template integrated with a host CPU and system memory hierarchy. Source: [10].	52
5.3	Microarchitecture of Gemmini’s Spatial Array	52
5.4	The im2col Transformation	54
5.5	Conceptual view of Gemmini’s decoupled access-execute pipelines, man- aged by separate hardware command queues for Load, Execute, and Store operations. Source: [10].	56
6.1	Overview of the internal modules in the generated Gemmini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers.	57
6.2	LoopConv module	59
6.3	LoopMatmul module	60
6.4	The Load and Store Controller module, which manages data movement between the scratchpad and DRAM. Both controllers load the com- mands from the ReservationStation into the DMACommandTracker, which is connected to the DMA read and write interfaces of the scratch- pad memory.	64
6.5	The ExecuteController module and its associated SRAM control in- terfaces.	65
6.6	The internal structure of the Scratchpad module, showing the various interfaces to the TLB, DMA, and internal memory banks.	66
6.7	The Mesh module.	68
6.8	Different Mesh configurations, while maintaining the same total num- ber of 16 PEs.	68
6.9	PE supports different dataflows (Output Stationary and Weight Sta- tionary).	69
6.10	MAC module, or Multiply-Accumulate unit, performs the core compu- tation in the systolic array.	70

7.1	The implemented Rocket SoC with Gemmini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemmini systolic array, and various peripherals are integrated via AXI interfaces.	74
7.2	Gemmini Configuration on Rocket64b1gem16jtag SoC.	77
7.3	Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core's datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels.	79
7.4	SD Mode: Multi-Block Read Operation. [3]	81
7.5	SD Mode: Multi-Block Write Operation. [3]	81
7.6	SPI Mode: Multi-Block Read Operation. [3]	82
7.7	SPI Mode: Multi-Block Write Operation. [3]	82
8.1	Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems.	88
8.2	Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART.	89
8.3	Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection.	90
8.4	Address Editor view, confirming the memory map for the integrated peripherals.	90
8.5	Device View of the implemented design on the FPGA fabric.	91
8.6	Detailed Utilization Summary for the entire SoC on the VC707 FPGA.	92
8.7	Hierarchical Utilization, showing the breakdown of resources by major system modules.	93
8.8	Hierarchical Utilization of the Gemmini accelerator module.	93
8.9	Power analysis settings.	93
8.10	Post-implementation on-chip power summary.	94
8.11	Hierarchical power report for the entire system.	94
8.12	Hierarchical power report for the Gemmini accelerator.	95

LIST OF TABLES

2.1	RISC-V Standard 32-bit ABI Register Conventions (RV32I). [22]	. . .	10
2.2	Common RISC-V Pseudo-instructions. [22]		11
2.3	Development status of extended RISC-V instruction sets. Source [22]	.	13
3.1	TileLink operational configurations. Source: [19]		32

Chapter 1: Introduction

This thesis presents the design, implementation, and evaluation of a RISC-V System-on-Chip (SoC) featuring a specialized Deep Neural Network (DNN) accelerator, conducted within the Computer Embedded Systems Laboratory (CESLAB) of the University of Science, VNUHCM (HCMUS). This work represents a significant step in our laboratory’s exploration of the emerging RISC-V ecosystem and open-source SoC design automation solutions for future chip development and fabrication.

1.1 Background and Motivation

The rapid advancement of artificial intelligence and machine learning has created an unprecedented demand for specialized computing architectures. Deep Neural Networks, which form the backbone of modern AI applications, require enormous computational resources that traditional general-purpose processors struggle to provide efficiently. This computational challenge, combined with the exponential growth in model complexity and the need for energy-efficient solutions, has driven the development of custom hardware accelerators specifically designed for DNN workloads.

Traditionally, the CESLAB has been involved in designing and developing hardware accelerators (or IP Cores) on FPGA using Intel Quartus, particularly leveraging the Platform Designer tool with Nios II soft processors and Avalon Bus interfaces. While this approach has served well for FPGA-based prototyping, it presents significant limitations for our laboratory’s strategic evolution toward chip fabrication and VLSI design. The Avalon Bus, though functional, lacks the sophistication and industry adoption of modern interconnect standards such as AXI4 and TileLink. More critically, the Intel Quartus ecosystem is primarily optimized for FPGA implementation and lacks the comprehensive VLSI design flow capabilities essential for ASIC development and chip fabrication.

As CESLAB envisions a future centered on chip prototyping and semiconductor fabrication, the limitations of our current toolchain have become increasingly apparent. The proprietary nature of Intel’s tools not only imposes licensing costs but also restricts our ability to customize the design flow, integrate with modern VLSI tools, and contribute to the broader open-source hardware community. Furthermore, the transition from FPGA prototypes to actual silicon requires a design methodology that seamlessly bridges both domains—a capability that traditional FPGA-centric tools

cannot provide.

The emergence of the RISC-V instruction set architecture has fundamentally changed the landscape of processor design and SoC development, offering a pathway to address these limitations. As an open-source ISA, RISC-V eliminates the barriers imposed by proprietary architectures, providing unprecedented flexibility for customization and modification. More importantly, this openness has catalyzed the development of comprehensive design ecosystems that support the entire SoC development flow, from high-level specification through FPGA prototyping to final VLSI implementation and chip fabrication.

1.2 The Chipyard Framework and Full-Stack Integration

Among the various RISC-V SoC design frameworks, Chipyard has emerged as the most comprehensive and mature solution for academic research and industrial VLSI development. Developed at UC Berkeley, Chipyard provides a complete, open-source SoC design environment that spans from high-level hardware generators written in Chisel to full VLSI implementation flows optimized for ASIC fabrication. This end-to-end capability is precisely what CESLAB requires to transition from FPGA-based prototyping to actual chip development.

The framework’s technical superiority over traditional FPGA design flows is evident in several key areas. Unlike the proprietary Avalon Bus used in Intel’s ecosystem, Chipyard leverages industry-standard interconnects including TileLink and AXI4. AXI4, in particular, has become the de facto standard for modern SoC design due to its superior performance characteristics, extensive tool support, and widespread adoption across the semiconductor industry. This adherence to industry standards ensures that designs developed using Chipyard can seamlessly integrate with commercial IP cores and industry-standard VLSI tools.

Most critically, Chipyard includes a fully integrated, open-source VLSI flow that encompasses synthesis, place-and-route, and timing closure using industry-standard tools such as Cadence Genus and Innovus. This VLSI-optimized design flow enables the direct translation of high-level hardware descriptions into fabrication-ready layouts, supporting multiple process technologies including advanced nodes from foundries like TSMC and GlobalFoundries. This capability is essential for CESLAB’s strategic goal of developing chip prototypes and eventual commercial silicon products.

A key innovation of Chipyard is its emphasis on *full-stack integration*. Unlike

traditional approaches that design and evaluate accelerators in isolation, Chipyard enables the creation of complete, Linux-capable systems that capture real-world effects such as operating system overhead, memory hierarchy interactions, and system-level resource contention. This holistic approach is crucial for understanding the true performance characteristics of specialized accelerators when deployed in practical computing environments—insights that are invaluable when transitioning from FPGA prototypes to optimized ASIC implementations.

The framework includes the Gemmini accelerator generator, an open-source DNN accelerator specifically designed for systolic array-based matrix computations. Gemmini embodies the principles of efficient DNN acceleration through its configurable systolic array architecture, which maximizes data reuse and minimizes the costly movement of data between computational units and external memory—a fundamental challenge known as the "memory wall" in computer architecture.

1.3 Research Objectives and Contributions

This thesis aims to demonstrate the capabilities of the Chipyard framework through the implementation of a practical RISC-V SoC that integrates a Rocket processor core with the Gemmini DNN accelerator, while establishing a comprehensive foundation for CESLAB’s transition toward chip prototyping and VLSI-based development. The specific objectives of this work include:

1. **Framework Evaluation and Migration:** Comprehensively evaluate the Chipyard framework as a replacement for Intel Quartus-based design flows, specifically assessing its VLSI capabilities, modern interconnect support (TileLink/AXI4), and suitability for ASIC development workflows.
2. **System Architecture Design:** Develop a comprehensive understanding of the Chipyard framework and utilize it to configure a custom SoC featuring a 64-bit Rocket core coupled with a 16×16 Gemmini systolic array accelerator (Rocket64b1gem16jtag configuration), demonstrating the framework’s flexibility and configurability.
3. **Full-Stack Implementation:** Implement the complete system stack, including hardware generation, software compilation, bootloader configuration, and Linux operating system integration, demonstrating the end-to-end capability of the open-source toolchain from high-level design to functional system.

4. **FPGA Prototyping and Validation:** Successfully deploy and validate the designed SoC on a Xilinx VC707 FPGA board, providing a realistic hardware platform for system evaluation and serving as a crucial stepping stone toward future ASIC implementation.
5. **Accelerator Integration and Testing:** Demonstrate the integration of the Gemmini accelerator as a Rocket Custom Coprocessor (RoCC), validate its functionality through comprehensive testing, and showcase how specialized computational units can be seamlessly incorporated into RISC-V systems using modern interconnect standards.
6. **Documentation and Knowledge Transfer:** Create comprehensive documentation of the design process, implementation challenges, and solutions to serve as a foundational reference for future CESLAB projects targeting chip prototyping and VLSI development.

1.4 Significance and Impact

This work represents the first comprehensive exploration of the RISC-V ecosystem within CESLAB and, more importantly, establishes a critical foundation for our laboratory’s strategic transition toward chip prototyping and semiconductor fabrication. By successfully implementing a complete, functioning system that combines a RISC-V processor with a state-of-the-art DNN accelerator using industry-standard design flows, this thesis demonstrates the maturity and practicality of open-source hardware design methodologies for serious VLSI development.

The strategic significance of this work extends far beyond the immediate technical achievements. This thesis serves as a comprehensive feasibility study and implementation roadmap for CESLAB’s evolution from FPGA-based prototyping to actual chip development. The successful demonstration of the Chipyard framework’s VLSI capabilities, coupled with its support for modern interconnect standards and integration with industry-standard EDA tools, validates our laboratory’s decision to adopt this platform as the foundation for future chip development projects.

Furthermore, this work establishes a new research paradigm within our laboratory that emphasizes open-source collaboration, reproducible research, and the development of freely available intellectual property. This approach not only reduces the barriers to advanced hardware research but also positions CESLAB as a contributor

to the global open-source hardware movement, potentially leading to collaborative opportunities with industry and academic partners pursuing similar chip development goals.

The comprehensive documentation generated by this thesis—covering everything from framework setup and configuration to FPGA deployment and system validation—serves as an invaluable knowledge base for future CESLAB researchers and students. This documentation addresses the practical challenges of implementing complex SoC designs using open-source tools, providing detailed solutions to common problems and establishing best practices for our laboratory’s future chip development projects.

Most critically, the full-stack nature of this implementation provides valuable insights into the complex interactions between hardware accelerators, operating systems, and application software—interactions that are often overlooked in studies that focus solely on isolated hardware components. These insights are crucial for developing practical, deployable AI systems that can deliver their theoretical performance benefits in real-world environments, and they will inform our laboratory’s approach to designing optimized ASIC implementations in future projects.

1.5 Thesis Structure and Organization

This thesis is organized into eight chapters that systematically present the theoretical foundations, implementation details, and evaluation results of our RISC-V SoC with Gemmini accelerator:

Chapter 2: RISC-V Microprocessor Instruction Set Architecture provides a comprehensive overview of the RISC-V ISA, focusing on the RV32I base instruction set and its advantages as an open-source architecture. This chapter establishes the theoretical foundation for understanding the processor core used in our SoC implementation.

Chapter 3: A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System introduces the system architecture and design methodology using the Chipyard framework. It details the compilation and elaboration flow, the role of Chisel/Scala in hardware design, and the overall system construction approach.

Chapter 4: Architectural Foundations of DNN Acceleration explores the fundamental challenges of Deep Neural Network acceleration, including the memory wall problem and the principles of systolic array architectures. This chapter provides

the theoretical context for understanding why specialized accelerators like Gemini are necessary for efficient DNN computation.

Chapter 5: The Gemini DNN Accelerator: An Architectural Overview presents a detailed analysis of the Gemini accelerator architecture, including its systolic compute engine, memory hierarchy, and integration model within the SoC framework.

Chapter 6: Gemini’s Internal Architecture and Programming Model delves into the internal implementation details of Gemini, examining its command and control flow, instruction processing pipeline, and programming interfaces that enable efficient DNN acceleration.

Chapter 7: System Implementation and Testing documents the complete implementation process, from SoC configuration and hardware generation to FPGA deployment and system validation. This chapter provides practical insights into the entire development workflow and demonstrates the successful integration of all system components.

Chapter 8: Conclusion and Future Work summarizes the achievements of this work, evaluates the success of the implementation against the stated objectives, and outlines potential directions for future research and development.

This structured approach ensures a comprehensive understanding of both the theoretical principles underlying modern SoC design and the practical challenges involved in implementing complex, multi-component systems using open-source tools and methodologies.

Chapter 2: RISC-V Microprocessor Instruction Set Architecture

2.1 Overview of RISC-V

Designing and developing a microprocessor, or Central Processing Unit (CPU), is a complex process requiring the collaboration of many experts in fields such as digital logic design, compiler development, and operating systems, along with significant financial resources. Major vendors like ARM, with prominent processor lines such as Cortex-M and Cortex-A, often impose high licensing fees and strict confidentiality requirements on their designs. This creates considerable barriers to hardware and software research and customization, particularly for academic institutions or small businesses. In response to these limitations, the RISC-V architecture [22, 21] was developed and introduced as an open-source solution, providing a flexible platform that eliminates licensing costs and allows for need-based customization. By fostering innovation and opening up a freer research ecosystem, RISC-V is reshaping the traditional approach to microprocessor development, aiming for more efficient and comprehensive solutions.



Figure 2.1: RISC-V is managed by RISC-V International. Source [17]

Essentially, RISC-V is an open-source microprocessor instruction set architecture (ISA), designed and developed based on the Reduced Instruction Set Computer (RISC) methodology, and it represents the fifth generation of this design approach. For a program executing on a microprocessor, the RISC design methodology stipulates that the microprocessor executes only simple assembly (machine code) instructions, which have a fixed length and are executed within a single, uniform clock cycle. A complex task is performed by multiple assembly instructions, making the instruction set simple, compact, and easy to develop hardware for rapidly. This is entirely different from the Complex Instruction Set Computer (CISC) architecture, which aims to complete a task with the fewest possible assembly instructions. To achieve this, CISC microprocessors must be designed to execute structurally complex assembly instructions, capable of performing multiple specialized functions sequentially and executing over multiple clock cycles.

Originating as a project developed by researchers and graduate students at the

University of California, Berkeley in 2010, the initial purpose of RISC-V was to create a practical ISA to support research and teaching in computer architecture, deployable on both hardware and software without licensing requirements. The original authors of RISC-V were individuals with considerable experience in computer design. Consequently, for stable development, RISC-V aimed for several goals, most of which have been achieved, including:

- A complete, fully open-source ISA, free for academic, educational, and industrial use, with practical applicability suitable not only for experimental simulation but also for hardware implementation.
- An ISA not oriented towards complex microarchitectures or fixed by any specific implementation technology (FPGA, ASIC, full-custom, etc.) but still allowing for efficient implementation in any of these directions.
- An ISA divided into base ISAs, usable for developing hardware accelerators or for educational purposes, and optional standard extensions to support general software development.
- An ISA supporting extensive instruction set extensions with specialized functionalities and numerous variants.
- Support for the IEEE 754-2008 floating-point standard.
- Support for parallel, multi-core, and heterogeneous many-core implementations.
- Both 32-bit and 64-bit address space variants developed for applications, operating system kernels, and hardware implementations.

Due to its open-source nature, RISC-V allows developers to freely use the design architecture without paying intellectual property and licensing fees. This creates development opportunities for RISC-V itself as it is adopted by the research community and educational institutions, gradually becoming popular in industry, and attracting significant attention from many large companies, corporations, and even governments of various countries. Furthermore, designers using the RISC-V ISA can utilize it without limitations in both hardware and software. This means developers are encouraged, but not required, to publicly disclose their designs and projects developed using RISC-V.

Currently, the development process and rights regarding the publication, release, and maintenance of intellectual property and official documentation related to RISC-V are managed by RISC-V International, based in Switzerland. This meets the standardization requirements for designers and users for commercial purposes, who demand a system using the RISC-V ISA that can operate and exist for many years.

To suit various implementers and ensure flexibility in designs with specialized characteristics, the RISC-V instruction set is divided and standardized into two main parts: the base instruction set and standard extensions. The RISC-V instruction set supports 32-bit, 64-bit, and 128-bit architectures with the respective base instruction sets RV32I, RV32E, RV64I, and RV128I. Among these, RV32I is the most important and common set for 32-bit microprocessors, featuring 40 integer manipulation instructions. The fact that RISC-V only standardizes and provides the instruction set allows hardware designers the freedom to choose how to implement their designs.

2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set

RV32I [22] is the base instruction set present in almost all 32-bit RISC-V microprocessor architectures. The assembly instructions in this set use 32 32-bit registers, named `x0` - `x31`, to store integers. Each register, in addition to being numbered from 0 to 31, is also referred to by its Application Binary Interface (ABI) name, which reflects its specific function. For example, register `x0` is a read-only register containing the constant value "0", and register `x1` is used as the return address register, storing the return address when a function is executed. Table 2.1 lists these registers and their functions in detail.

The assembly instructions in RV32I use six different 32-bit instruction formats, depending on the function of the instruction. These include R, I, S, U formats, as well as two additional formats for immediate values: B and J. These formats are clearly shown in Figure 2.2. Notably, the source registers (`rs1` and `rs2`) and the destination register (`rd`) are kept in the same position in most formats, simplifying instruction decoding. The only discernible difference lies in how `rs2`, `rd`, and immediate values are organized within the instruction.

Figure 2.3 details all instructions in the RV32I base instruction set with their respective formats, specific opcode values, and function codes (`funct3` and `funct7`). Functionally, the RV32I set is divided into five distinct groups:

- 21 integer computation instructions: Perform arithmetic (addition, subtraction, bit shifts), logic (AND, OR, XOR, NOT, etc.), and comparison operations

Table 2.1: RISC-V Standard 32-bit ABI Register Conventions (RV32I). [22]

Register	ABI Name	Description
x0	zero	Constant "0"
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5 → x7	t0 → t2	Temporary registers
x8	s0/fp	Memory/frame pointer
x9	s1	Saved register
x10, x11	a0, a1	Function parameter/return value
x12 → x17	a2 → a7	Function parameter
x18 → x27	s2 → s11	Saved register
x28 → x31	t3 → t6	Temporary registers

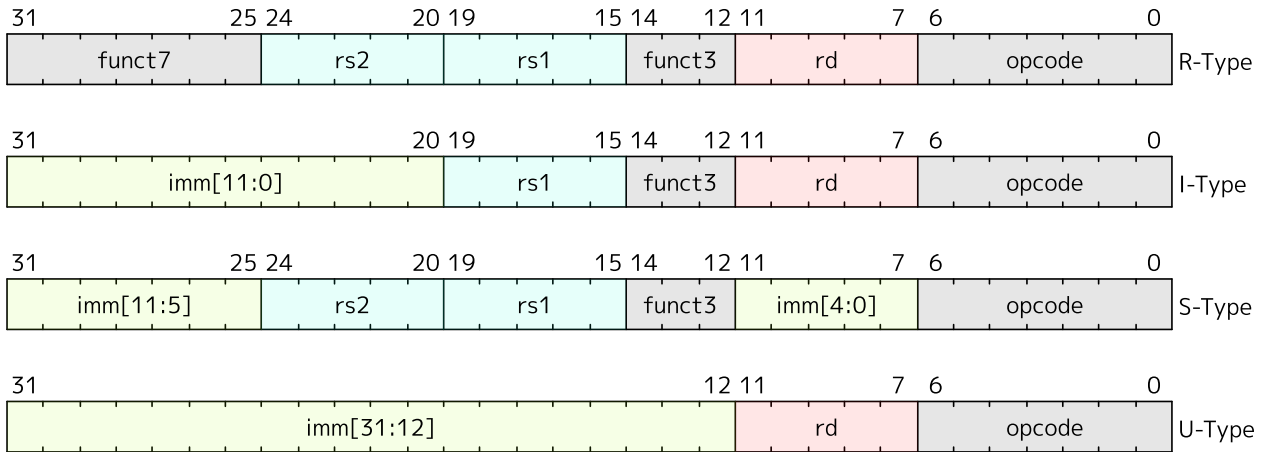


Figure 2.2: Instruction encoding formats in RV32I. Source [22]

on signed and unsigned numbers. Typically use R-type and I-type formats.

- 8 load-store instructions: LOAD and STORE byte, half-word, word (signed and unsigned), typically using I-type and S-type formats.
- 8 control-flow/branch instructions: Conditional branches, performing comparisons between two registers to decide whether to branch (BEQ/BNE, BLT, BGE), typically using I-type format.
- 2 system access instructions for the execution environment - ECALL and breakpoint - EBREAK, using I-type format.
- 1 memory access ordering instruction - FENCE, used by other processing cores

(in multi-core scenarios) or other peripheral devices in the system. The I-type format is used for this instruction with some specific bit encoding rules and opcodes.

With these 40 instructions, the microprocessor can perform most basic operations on integers. Additionally, several standard pseudo-instructions are defined to add tasks for which there are no dedicated instructions in the set. For example, the MOVE instruction - "mv rd, rs" - performs the function of moving/copying data between two registers. In a program, this instruction might be generated in the assembly code by a RISC-V compiler but is actually converted by assemblers into a standard instruction when translating to machine code, namely ADDI - Add Immediate "addi rd, rs, 0", which adds the source register to the constant "0" and assigns it to the destination register, achieving a similar function. Some other pseudo-instructions and their equivalent standard instructions are shown in Table 2.2.

Table 2.2: Common RISC-V Pseudo-instructions. [22]

Pseudo-instruction	Corresponding Base Instruction	Meaning
nop	addi x0, x0, 0	No operation
li rd, imm	addi rd, x0, imm	Load immediate
mv rd, rs	addi rd, rs, 0	Move register (copy)
not rd, rs	xori rd, rs, -1	One's complement (bitwise NOT)
neg rd, rs	sub rd, x0, rs	Two's complement (negate)
seqz rd, rs	sltiu rd, rs, 1	Set if equal to zero
snez rd, rs	sltu rd, x0, rs	Set if not equal to zero
sltz rd, rs	slt rd, rs, x0	Set if less than zero
sgtz rd, rs	slt rd, x0, rs	Set if greater than zero
beqz rs, offset	beq rs, x0, offset	Branch if equal to zero
bnez rs, offset	bne rs, x0, offset	Branch if not equal to zero
blez rs, offset	bge x0, rs, offset	Branch if less than or equal to zero
bgez rs, offset	bge rs, x0, offset	Branch if greater than or equal to zero
bltz rs, offset	blt rs, x0, offset	Branch if less than zero
bgtz rs, offset	blt x0, rs, offset	Branch if greater than zero
j offset	jal x0, offset	Jump to label (offset)
jal offset	jal x1, offset	Jump and link (call)
ret	jalr x0, 0(x1)	Return from subroutine

As mentioned, a goal of RISC-V is to support implementations requiring more

RV32I Base Instruction Set							
imm[31:12]					rd	0110111	LUI
imm[31:12]					rd	0010111	AUIPC
imm[20 10:1 11 19:12]					rd	1101111	JAL
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12 10:5]		rs2	rs1	000	imm[4:1 11]		BEQ
imm[12 10:5]		rs2	rs1	001	imm[4:1 11]		BNE
imm[12 10:5]		rs2	rs1	100	imm[4:1 11]		BLT
imm[12 10:5]		rs2	rs1	101	imm[4:1 11]		BGE
imm[12 10:5]		rs2	rs1	110	imm[4:1 11]		BLTU
imm[12 10:5]		rs2	rs1	111	imm[4:1 11]		BGEU
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]		rs2	rs1	000	imm[4:0]		SB
imm[11:5]		rs2	rs1	001	imm[4:0]		SH
imm[11:5]		rs2	rs1	010	imm[4:0]		SW
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
1000	0011	0011	00000	000	00000	0001111	FENCE.TSO
0000	0001	0000	00000	000	00000	0001111	PAUSE
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK

Figure 2.3: Instructions in the RV32I base instruction set. [22]

features beyond the base instruction set. This is reflected in RISC-V defining extensions [22] for its instruction set architecture. These extensions define additional instruction encodings for various functionalities. For example, the standard M extension adds instructions for integer multiplication and division, while the standard F extension defines instructions for single-precision floating-point computation and manipulation.

Table 2.3: Development status of extended RISC-V instruction sets. Source [22]

Instruction Set	Function Description	Version	Status
M	Integer multiplication and division instruction set	2.0	Approved
A	Atomic memory operation instruction set	2.1	Approved
F	Single-precision floating-point computation	2.2	Approved
D	Double-precision floating-point computation	2.2	Approved
Q	Quad-precision floating-point computation	2.2	Approved
C	Compressed instruction set	2.0	Approved
Zifence	Instruction-Fetch Fence instruction set	2.0	Approved
Zicsr	Control and Status Registers (CSRs) instruction set	2.0	Approved
Ztso	Total Store Ordering instruction set	1.0	Approved

Table 2.3 above clearly lists the standard extension sets developed and published by RISC-V International. It can be seen that many instruction sets have been ratified and can be used in designs. When implemented, architectures supporting the RV32I base instruction set (or RV64I depending on the architecture) combined with the standard extensions M, A, F, D, Zicsr, and Zifencei are often summarized and referred to as the RV32G (or RV64G) instruction set, named for its general-purpose utility.

2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set

Similar to RV32I, RV64I is the base instruction set for 64-bit microprocessor architectures, featuring 64-bit wide registers and supporting 64-bit addressing (XLEN=64), though instructions remain 32-bits wide. Architectures compliant with RV64I are compatible with RV32 instructions and include additional instructions to support operations between 32-bit and 64-bit data. For example, the LW instruction loads a 32-bit value and sign-extends it into a 64-bit register, while the new LD instruction loads a 64-bit word. The ADD instruction now performs 64-bit addition, while the ADDW instruction handles 32-bit addition, ignoring the upper 32 bits and returning a 32-bit signed value, sign-extended to 64 bits. These instructions enabling backward

compatibility typically have a "W" suffix to denote 32-bit wide operations. These instructions, along with new 64-bit load and store operations, are summarized in Figure 2.4.

RV64I Base Instruction Set (in addition to RV32I)					
imm[11:0]		rs1	110	rd	0000011 LWU
imm[11:0]		rs1	011	rd	0000011 LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011 SD
000000	shamt	rs1	001	rd	0010011 SLLI
000000	shamt	rs1	101	rd	0010011 SRLI
010000	shamt	rs1	101	rd	0010011 SRAI
imm[11:0]		rs1	000	rd	0011011 ADDIW
0000000	shamt	rs1	001	rd	0011011 SLLIW
0000000	shamt	rs1	101	rd	0011011 SRLIW
0100000	shamt	rs1	101	rd	0011011 SRAIW
0000000	rs2	rs1	000	rd	0111011 ADDW
0100000	rs2	rs1	000	rd	0111011 SUBW
0000000	rs2	rs1	001	rd	0111011 SLLW
0000000	rs2	rs1	101	rd	0111011 SRLW
0100000	rs2	rs1	101	rd	0111011 SRAW

Figure 2.4: Instructions in the RV64I base instruction set. [22]

The compiler and calling conventions maintain the invariant that all 32-bit values are stored in sign-extended format within 64-bit registers. Even 32-bit unsigned integers will have bit 31 extended into bits 63 through 32. Consequently, converting between 32-bit signed and unsigned integers is a no-operation, as is converting from a 32-bit signed integer to a 64-bit signed integer. Existing 64-bit instructions like **SLTU** (set less than unsigned) and branch comparisons continue to operate correctly on 32-bit unsigned integers under this rule. Similarly, 64-bit logical operations on 32-bit sign-extended integers preserve the sign-extension property. A few new instructions (**ADD[I]W/SUBW/SxxW**) are necessary for addition and shifts to ensure reasonable performance for 32-bit values.

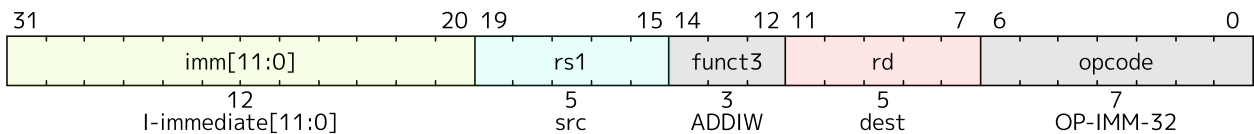


Figure 2.5: Instruction encoding format for ADDIW in 64-bit architecture. [22]

Some distinctly different instructions found only in RV64I include the **ADDIW** in-

struction, as shown in Figure 2.5. `ADDIW` performs an addition between a 12-bit sign-extended immediate value and register `rs1`, producing a proper 32-bit sign-extended result in `rd`. Overflows are ignored, and the result consists of the lower 32 bits of the sum, sign-extended to 64 bits. Note, the instruction "`addiw rd, rs1, 0`" writes the sign-extension of the lower 32 bits of register `rs1` into register `rd` (pseudo-instruction `SEXT.W`).

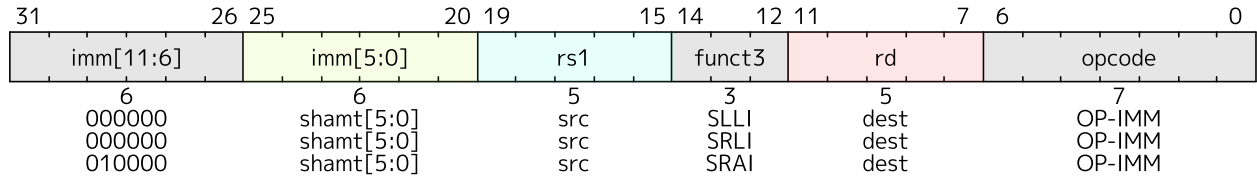


Figure 2.6: Encoding format for 64-bit I-type shift instructions. [22]

I-type shift operations in RV64I (Figure 2.6) use the same opcode as RV32I with some modifications. Register `rs1` contains the operand, and the shift amount is encoded in the lower 6 bits of the I-immediate field. Bit 30 determines the shift type: `SLLI` (logical left shift, zeros shifted into lower bits), `SRLI` (logical right shift, zeros shifted into upper bits), and `SRAI` (arithmetic right shift, sign bit copied into vacated upper bits).

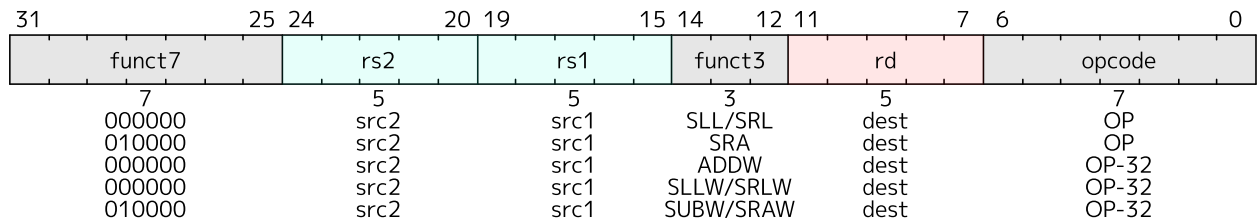


Figure 2.7: Encoding format for 64-bit R-type shift instructions for word operations. [22]

The `ADDW` and `SUBW` instructions in RV64I, similar to `ADD` and `SUB` for 64-bit operations, operate on 32-bit data and produce a 32-bit signed result, which is then sign-extended to 64-bits before being written to the destination register. Overflows are also ignored. The `SLL`, `SRL`, and `SRA` instructions perform logical left, logical right, and arithmetic right shifts on the value in `rs1`, with the shift amount determined by the lower 6 bits of `rs2`. The `SLLW`, `SRLW`, and `SRAW` instructions (Figure 2.7), unique to RV64I, operate on 32-bit values and use the lower 5 bits of `rs2` to determine the shift amount.

Finally, load-store instructions can be mentioned. RV64I extends the address space to 64 bits, but the number of accessible addresses may vary depending on the

execution environment.

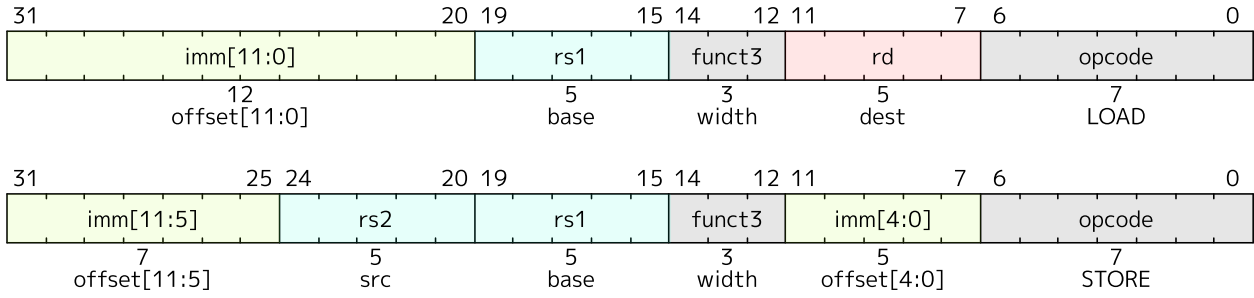


Figure 2.8: Encoding formats for 64-bit LOAD and STORE instructions. [22]

The instruction formats are shown in Figure 2.8. The LD instruction here will load a 64-bit value, LW loads a 32-bit value (sign-extended to 64-bits) from memory into a register, and LWU zero-extends a 32-bit value from memory into register `rd`. Similarly, LH/LHU and LB/LBU handle 16-bit and 8-bit values. The SD, SW, SH, and SB instructions store 64-bit, 32-bit, 16-bit, and 8-bit values from register `rs2` to memory, respectively.

2.2 RISC-V Processor Cores

With its open-source nature, system developers can freely implement designs based on the RISC-V architecture, supported by an increasingly robust and growing ecosystem. The RISC-V ISA ecosystem mentioned here includes a diverse range of components from hardware design tools, low-level firmware development, bootloaders, to fully functional operating system kernels. Currently, development tools such as GCC/LLVM compilers, QEMU and Verilator simulators, and even the Linux operating system have incorporated RISC-V architecture and are under active development. As of the time of writing this thesis, there are over 100 open-source hardware processor cores and more than 40 open-source SoC platforms using the RISC-V ISA that have been and continue to be developed.

This section will provide a general overview of pipeline, in-order, and out-of-order architectures commonly implemented in microprocessors. Subsequently, a brief survey of open-source RISC-V processor cores will be conducted to select the most suitable design for implementation.

2.2.1 Pipeline Architecture in Microprocessors

A modern general-purpose CPU is often designed using a pipeline architecture. This technique draws inspiration from industrial assembly lines, dividing the processing of an assembly instruction into multiple stages or segments, each performed by different specialized hardware units connected sequentially throughout the process and capable of continuous operation. This architecture is commonly found in many microprocessor designs using RISC in general and RISC-V ISA in particular, due to its higher performance compared to processing instructions sequentially and waiting for results.

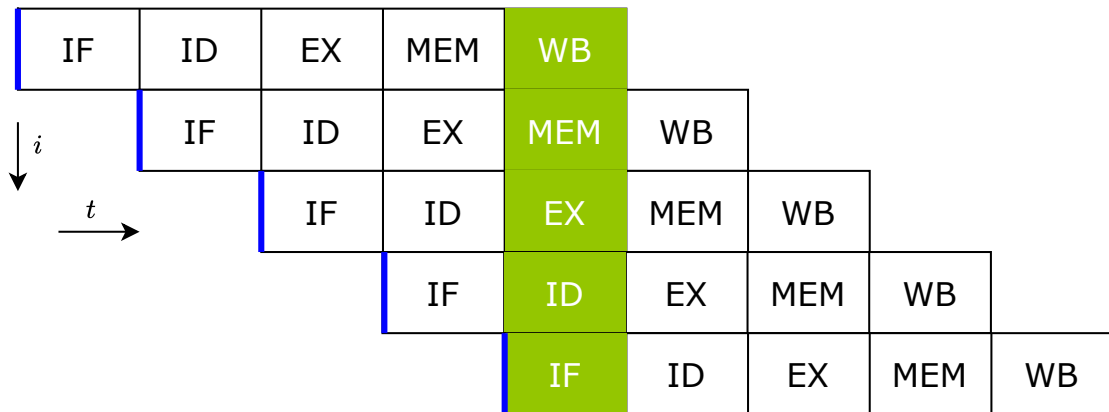


Figure 2.9: Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch.

The concept of pipelining is illustrated in Figure 2.9. While one assembly instruction is being processed in later stages, another assembly instruction begins processing in earlier stages, in parallel with previous instructions. It can be seen that at the fourth clock cycle, the pipeline is processing four different instructions in parallel, each in a distinct stage. A basic microprocessor pipeline consists of five stages. However, depending on the implementation and design, a microprocessor may use more or fewer stages, but still follows the sequence of stages explained below:

- **Instruction Fetch (IF):** Retrieves instructions from the instruction memory, also known as program memory. Each instruction code that the CPU must execute is stored in the program memory. These instruction codes are binary sequences compiled from C/C++ code or assembly code.
- **Instruction Decode (ID):** The binary of each instruction is decoded, and corresponding control signals are activated.

- **Execute (EX):** Performs calculations and processing corresponding to the decoded instruction, such as addition, subtraction, multiplication, division, shifts, assignments, branches, etc.
- **Memory Access (MEM):** Accesses memory and performs read/write operations of data from/to memory.
- **Writeback (WB):** Saves the results after instruction execution to the location specified by the instruction, such as registers.

2.2.2 Out-of-Order Architecture in Microprocessors

Alternatively, out-of-order execution architecture is a crucial paradigm in modern CPU design, delivering significant performance gains by allowing instructions to be executed as soon as their input operands are ready, rather than strictly adhering to the program order as in a pipeline. This architecture is also a core element in high-performance systems used in scientific computing and artificial intelligence research. This flexible execution model addresses performance bottlenecks caused by data dependencies, cache latencies, and pipeline stalls, ensuring more efficient utilization of computational resources.

Key components of this architecture include the instruction window, which stores multiple instructions for dependency analysis, and dynamic scheduling mechanisms such as Tomasulo's algorithm or scoreboarding, which track dependencies and resource availability. Another critical feature is the Re-order Buffer (ROB), which helps maintain program order during instruction retirement, ensuring correct architectural state updates and precise exception handling. By decoupling instruction fetch and execution stages, out-of-order processors achieve higher instruction throughput and sustain the performance of execution units, optimizing efficiency across various applications.

Despite its many strengths and implementation benefits, out-of-order processors also present significant design challenges. The hardware blocks required for components like the ROB and branch prediction increase complexity, chip area, and power consumption, making energy efficiency optimization an important research area. Furthermore, extending out-of-order architecture to meet the demands of heterogeneous processing and large-scale parallel computing requires innovative approaches in dynamic scheduling and resource management.

2.2.3 Open-Source RISC-V Processor Cores

Rocket [2], or Rocket Core, is a RISC-V instruction set architecture microprocessor core designed by the Berkeley Architecture Research group. Rocket features a 5-stage, in-order architecture and implements the RISC-V RV32G and RV64G instruction sets.

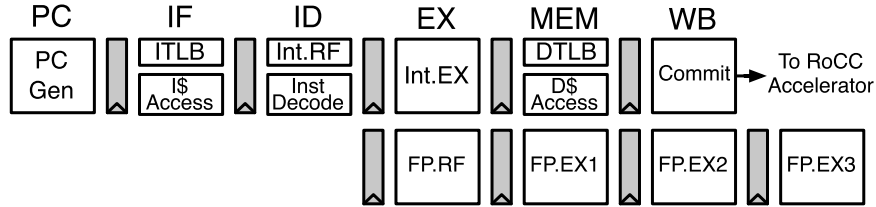


Figure 2.10: Pipeline of Rocket core. Source [2]

The design of this microprocessor integrates many advanced features, including a Memory Management Unit (MMU) that supports virtual memory, enabling efficient memory access and address translation. Besides its role as a general-purpose microprocessor, Rocket also serves as a library of reusable components, including functional units, caches, Translation Lookaside Buffer (TLB), Page Table Walker (PTW), and Control and Status Registers (CSRs), which are reused in many other microprocessor designs. This makes Rocket Core a foundational component in Chipyard - RocketChip, allowing for the rapid development and creation of experimental systems ranging from simple to complex.

Rocket is considered the most prominent microprocessor core in the open-source RISC-V development community due to its well-updated and stably supported development documentation. It is built using Chisel, a relatively new hardware construction language based on Scala, employing an object-oriented approach similar to software development. Therefore, the flexibility of the system development process and the applicability of Rocket Core are very high, making it suitable for implementing multi-core systems.

Along with Rocket Core, the Berkeley Out-of-Order Machine (BOOM) [5, 4, 6] is also a prominent open-source RISC-V core, designed for high performance and out-of-order execution. Also developed within the microprocessor architecture research group at the University of California, Berkeley, BOOM is aimed at applications in advanced academic research and industry, featuring a flexible design and high operational performance. BOOM is also written in the Chisel language as a generator, like Rocket Core, allowing for rapid deployment and customization of the core ac-

cording to the RV64GC RISC-V architecture. The latest version, SonicBOOM [25] (also known as BOOMv3), achieves a performance of 6.2 CoreMarks/MHz, capable of competing directly with high-performance commercial processor cores, setting a high standard for open-source processor design.

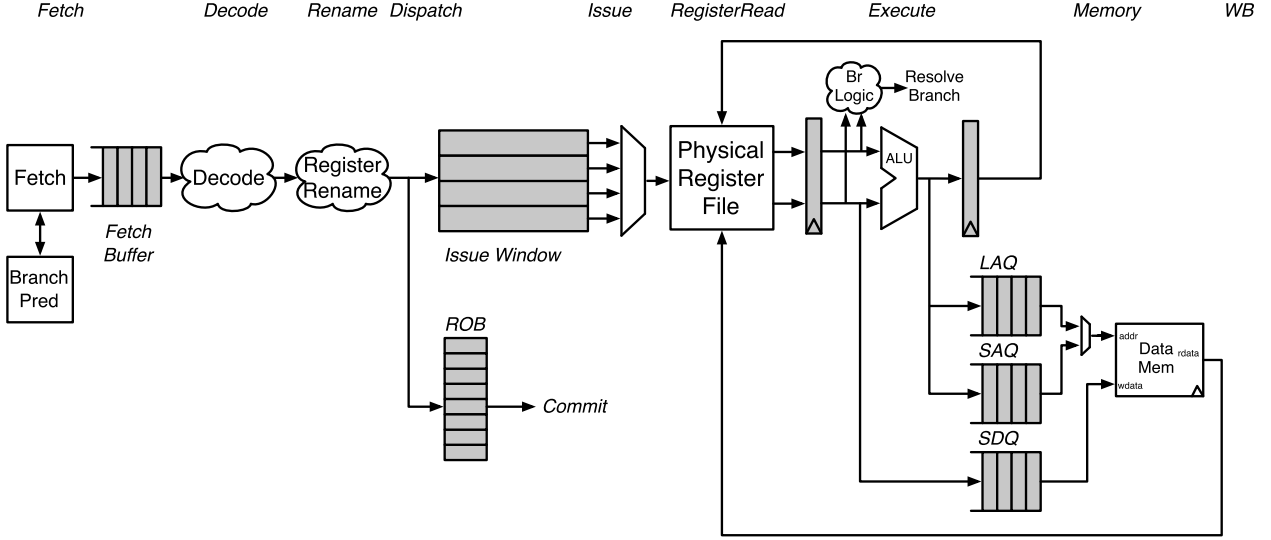


Figure 2.11: Berkeley Out-of-Order Machine Processor Pipeline Architecture. [6]

Structurally, BOOM features a 10-stage pipeline architecture including Fetch, Decode, Register Rename, Dispatch, Issue, Register Read, Execute, Memory, Writeback, and Commit. These stages allow for detailed control and optimization of the instruction flow, maintaining the "out-of-order" functionality at the high speeds necessary for modern processors. Specifically, each stage performs as follows:

- **Fetch:** Instructions are fetched from instruction memory and stored in a fetch buffer to prepare for processing.
- **Decode:** Instructions are converted into simpler micro-ops to optimize execution within the pipeline.
- **Register Rename:** Logical registers in the ISA are renamed to independent physical registers, reducing data conflicts and increasing processing efficiency.
- **Dispatch:** Micro-ops from the decode stage are sent to the Issue window, awaiting necessary operands.
- **Issue:** Instructions in the Issue window are checked for operand readiness, and when conditions are met, micro-ops are sent to an execution unit.

- **Register File (RF) Read:** Micro-ops read operands from the physical register file or bypass network, preparing data for the next stage.
- **Execute:** Micro-ops are processed by functional units, performing computational tasks and memory address calculations.
- **Memory:** Manages memory operations via Load Address Queue (LAQ), Store Address Queue (SAQ), and Store Data Queue (SDQ), performing loads when addresses are ready and stores when both address and data are available.
- **Writeback:** Results from operations are written back to the physical register file, ensuring updated values are stored for future instructions.
- **Commit:** The Reorder Buffer tracks the status of instructions in the pipeline and commits results when instructions complete, including storing data to memory.

Similar to Rocket Core, this is also considered one of the most prominent open-source microprocessor cores. However, BOOM’s design is more geared towards ASIC implementation, and its numerous features along with the use of Chisel for implementation might pose difficulties in creating a multi-core design in Verilog and deploying it on FPGAs.

2.3 System-on-Chip Design Framework: Chipyard - Rocket Chip

The Rocket Chip project, also developed by the Berkeley Architecture Research (BAR) group at the University of California, Berkeley, marked a significant advancement in open-source processor design. This project provides a flexible, scalable, and configurable platform, enabling the creation of custom RISC-V microprocessors tailored to diverse performance and application requirements. Rocket Chip is essentially a library of hardware generators that can be configured and interconnected in various ways, producing diverse SoC designs. This library can generate an in-order core (Rocket) or an out-of-order core (BOOM), and these cores can be connected to co-processors via an interface named RoCC (Rocket Custom Coprocessor) [2].

Subsequently, to develop and expand the capabilities of Rocket Chip, Chipyard was created. Chipyard is an open-source framework that integrates system design, simulation, and implementation for SoC development, and it has become one of the most actively embraced and developed frameworks by the community. Chipyard, also



Figure 2.12: Chipyard, developed by UC Berkeley. Source: [24]

originating from UC Berkeley, is the result of continuous development and integration of Chisel, Rocket Chip, and the Rocket Core.

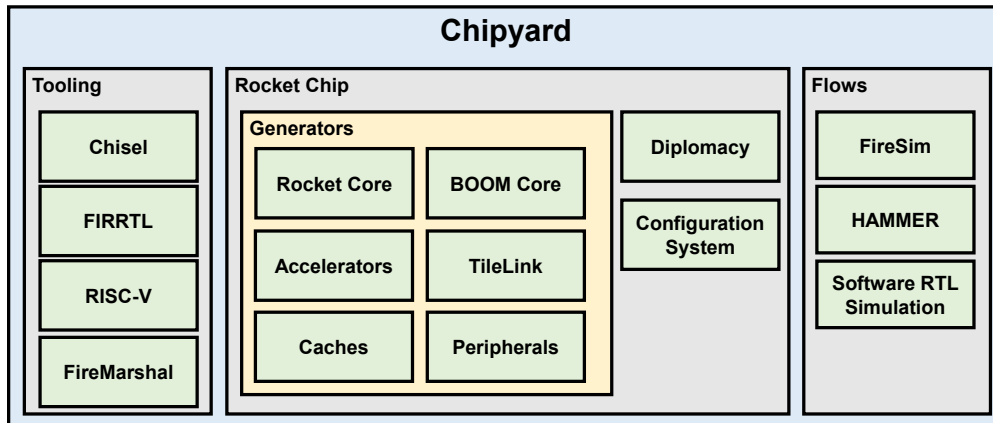


Figure 2.13: Component structure of Chipyard. Source: [24]

By utilizing Chisel, the FIRRTL compiler, the Rocket Chip generator, and other tools, Chipyard facilitates the development of highly customizable RISC-V SoCs. Chipyard includes a diverse set of processor cores (Rocket Core, BOOM, Ariane), accelerators (Hwacha, Gemmini, SHA3), along with comprehensive memory systems, peripheral devices, and tools to support the construction of a full-featured SoC. Chipyard supports multiple development flows, allowing for flexible hardware extension and experimentation. These development flows include software-based Register Transfer Level (RTL) simulation, FPGA-accelerated simulation using FireSim, automated VLSI flows with Hammer, and FireMarshal, which enables the creation and loading of software on both bare-metal systems (similar to microcontrollers) and Linux-based systems.

Rocket Chip supports deployment on FPGAs, with operational frequencies reaching 25-100 MHz depending on the configuration and the type of FPGA used. Currently, Rocket Chip is being utilized by SiFive [18], a company founded by some of the authors of RISC-V. SiFive produces RISC-V processors on silicon based on Rocket and provides tools for developing, testing, and evaluating their designs.

Using Chipyard (or Rocket Chip) as a foundation for system construction can pose several challenges due to its complexity in terms of the number of components and the design language used—Chisel. However, overcoming this barrier yields numerous benefits due to Rocket Chip’s high flexibility and system customization capabilities. Its components are developed independently, have detailed development documentation, and are easily extensible. Furthermore, the Rocket Chip support community is very active, with more frequent updates and improvements compared to other frameworks.

Chapter 3: A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System

From the surveys and analyses in the preceding sections regarding processor cores and the two SoC development frameworks, it is evident that each of these research subjects possesses distinct advantages and limitations. However, for the research orientation and implementation of a processor system featuring a custom accelerator, the Chipyard framework and its library components are deemed most suitable for the objectives of this thesis. Not only does Chipyard effectively meet current requirements, but it also holds the potential for expansion to support more complex systems in the future.

This chapter will present a more detailed exposition of the component structure and the process of constructing an SoC system, while also delving into the architecture of the Rocket processor core. The thesis will utilize high-level design languages Chisel/Scala to implement a single-core processor system integrated with the Gemmini RoCC accelerator, focusing on leveraging the components within an SoC system, and building the hardware and software design based on the Chipyard framework.

3.1 The Chipyard Compilation and Elaboration Flow

A key advantage of the Chipyard framework is its unified compilation and elaboration flow, which decouples high-level hardware design from low-level simulation and implementation targets [1]. The process begins with a **Custom SoC Configuration**, a set of Scala classes that define the parameters for the entire system. These parameters are consumed by the **RTL Generators**—hardware libraries written in Chisel—which programmatically construct the SoC design. Rather than directly emitting Verilog, the Chisel compiler generates a circuit description in the **FIRRTL Intermediate Representation**. This crucial IR acts as a pivot point for the entire framework, allowing a single design description to be consumed by various backend "transforms" tailored for specific goals, as illustrated in Figure 3.1.

For instance, the **FireSim Transforms** can apply modifications necessary for mapping the design onto cloud FPGAs for fast, cycle-exact simulation. The **VLSI Transforms** prepare the design for an ASIC implementation flow by replacing logical memories with technology-specific macros and preparing the hierarchy for place-and-route tools.

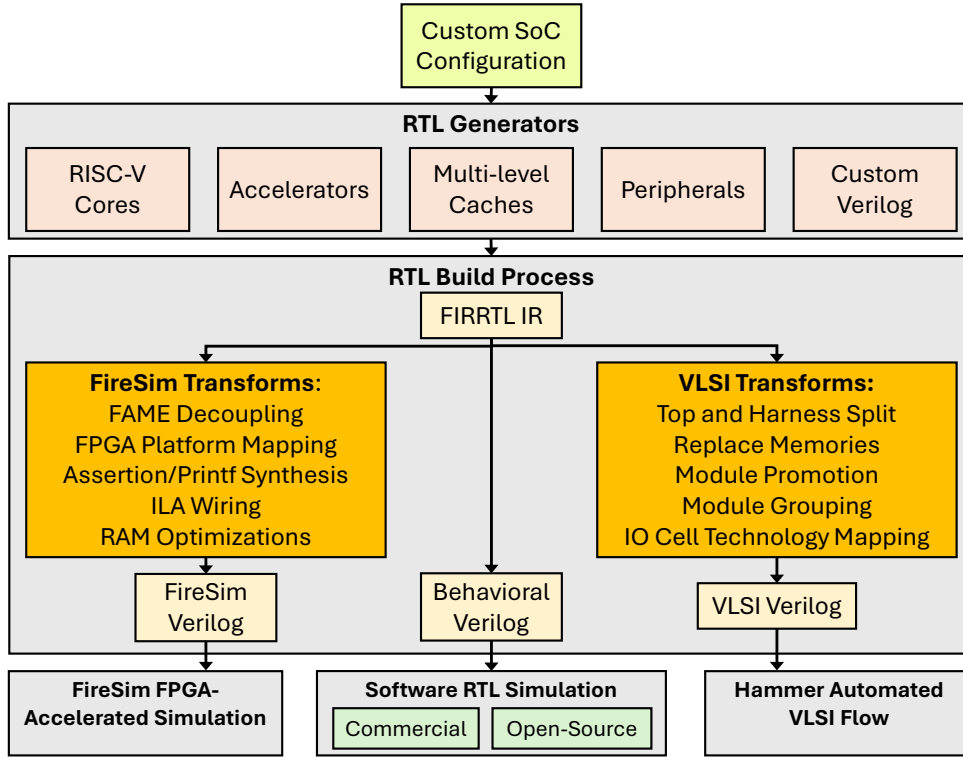


Figure 3.1: The overall development flow within Chipyard. Design begins with high-level configuration and Chisel generators, which produce FIRRTL. The FIRRTL IR acts as a pivot point, enabling different transformation paths for software simulation, FPGA-accelerated simulation, or VLSI layout. [1].

For the purposes of this thesis, this flow is primarily used to generate synthesizable Verilog for two key targets:

1. **Software RTL Simulation:** The FIRRTL is compiled into C++ code for cycle-accurate simulation using Verilator, enabling functional verification and initial performance analysis.
2. **FPGA Implementation:** The generated Verilog is consumed by the Xilinx Vivado toolchain to be synthesized, placed, and routed for deployment on the target VC707 FPGA board.

This structured flow demonstrates the power of a generator-based approach, allowing a single, high-level design description to target multiple platforms while maintaining design consistency.

3.2 System Design Methodology with Chisel

As previously introduced, this thesis undertakes system design based on the components and tools within Chipyard. This framework and its library components are constructed from a diverse range of elements, all based on Chisel/Scala.

3.2.1 Chisel and Scala



Figure 3.2: Chisel as a library within Scala.

Chisel is a Hardware Construction Language (HCL) developed at UC Berkeley, initially intended to enable hardware design through highly parameterized and hierarchically specific code generators. Chisel is used to define these generators, which then produce designs in Verilog, suitable for industry-standard RTL design flows. Chisel is classified as a Domain-Specific Language (DSL); more precisely, it is implemented as a library package for Scala, a high-level, multi-paradigm programming language that integrates features of both Object-Oriented Programming (OOP) and functional programming on the Java Virtual Machine (JVM). Furthermore, Scala’s functional programming capabilities support higher-order functions and immutable data structures, facilitating more maintainable code, which is crucial for reproducibility and collaboration in scientific research.

Chisel’s design aims to address the lack of object-oriented programming and pre-processing structures in existing Hardware Description Languages (HDLs), supporting the construction of more flexible generators. By being based on Scala, Chisel gains the flexibility to describe hardware with high levels of abstraction and modularity. This helps hardware designers describe circuits in a way that is easy to maintain, modify, and extend, especially for large and complex designs such as microprocessors and specialized digital signal processing cores. Scala’s robust type system, diverse numerical representations and variables, immutability, and support for functional programming make it an ideal foundation for Chisel, aiding in error checking during hardware and circuit description and encouraging reusable design patterns.

In the context of hardware design based on Chisel and Scala, `sbt` (Simple Build Tool) plays a crucial role, optimizing the design management and development process. Built specifically for Scala and Java projects, `sbt` is the build tool used by

the majority of Scala developers. Essentially, `sbt` acts as the compiler when working with Chisel and Scala. Using `sbt` simplifies the process of structuring projects for hardware design with Chisel in Chipyard, as well as enabling integration with many other open-source projects.

The design elaboration process using Chisel is illustrated in Figure 3.3. A design written in Chisel is first executed by a Scala program, which invokes the Chisel compiler to generate a circuit representation in FIRRTL. This intermediate representation is then processed by a series of FIRRTL transformers and optimizers, which ultimately produce synthesizable Verilog or VHDL code. This final HDL can then be consumed by standard industry tools for synthesis and physical design, leading to a final chip layout. This flow abstracts away many tedious aspects of hardware design, allowing architects to focus on the high-level structure and parameterization of their designs.

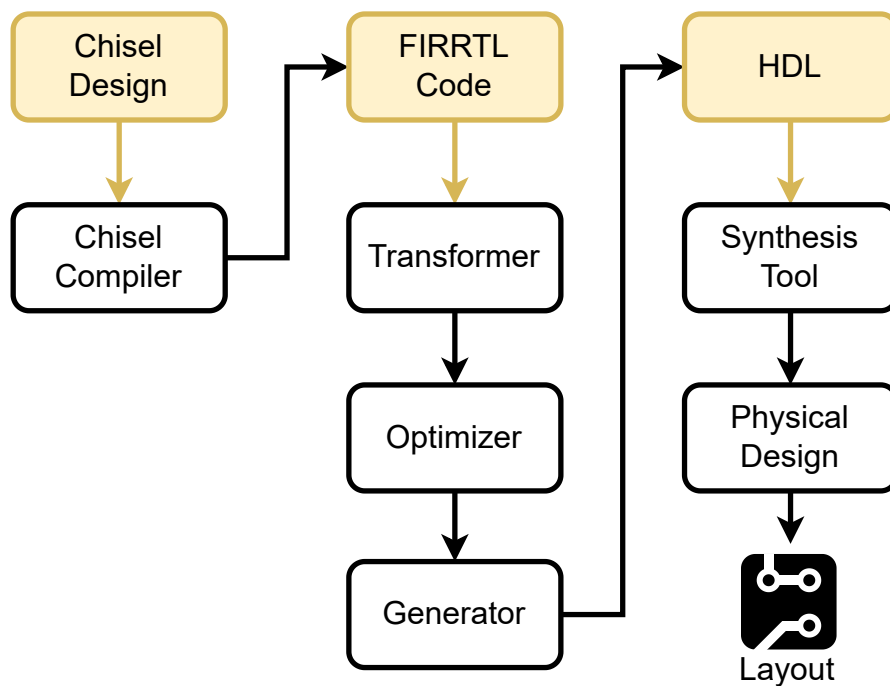


Figure 3.3: The Chisel to HDL design elaboration flow. A design is described in Chisel, compiled to FIRRTL, and finally transformed into a standard HDL for synthesis and physical design. [14]

3.2.2 Verilator Simulation Tool

In the hardware design process, evaluating the functionality of the design is critically important. With Chipyard, system designs, from simple to complex structures, can be effectively realized and tested thanks to integrated tools and RTL simulators like Verilator, which are readily available within the project framework.

Verilator is an open-source program, prominent in the field of RTL simulation due to its unique method of converting ("Verilating") `Verilog` or `SystemVerilog` into `C++` or `SystemC`, creating a highly optimized executable program. Unlike traditional simulators, Verilator does not directly interpret `Verilog` during simulation. Instead, it generates `C++` or `SystemC` program files by parsing the hardware code, performing syntax checking, linting, and optionally inserting debug points and coverage analysis to enhance the correctness and accuracy of evaluation and verification. This "Verilated" output file can be configured to run in single-threaded or multi-threaded mode on a development server, offering an advantage for simulating complex or computationally demanding designs. After the "Verilated" code is generated, it is compiled by a standard `C++` compiler such as `GCC`, `Clang`, or `MSVC++`, and is typically combined with a user-provided wrapper file to manage the initialization of the Verilated model.

Verilator offers high simulation performance, especially for designs requiring substantial resources, due to its high speed and cycle accuracy. Verilator's compilation approach makes it up to 100 times faster than interpretive simulators like Icarus Verilog (which is also open-source), and with multi-threading, performance can increase by an additional 2-10 times, achieving a total improvement of up to 1000 times. This makes Verilator suitable for RISC-V multi-core processor designs ranging from simple to complex, and competitive even with commercial simulators such as Synopsys VCS, Cadence Incisive, or Mentor ModelSim.

3.3 The Rocket Processor Core

As introduced in the previous chapter, Rocket is a 5-stage, in-order processor core compliant with the RISC-V instruction set architecture, with RV32G and RV64G variants. Currently, the development of the Rocket core is maintained by the CHIPS Alliance. Rocket is written in the Chisel hardware design language, based on the Scala programming language, and is a key component of the Rocket Chip SoC Generator toolchain.

When implemented, each Rocket core is combined with L1 data and instruction

caches. This memory operates locally for each core and, in a multi-core system, is responsible for storing temporary variables generated during the execution of the pipeline, helping to reduce congestion when multiple cores attempt to store data. Together with the Page Table Walker (PTW), a Tile within the SoC is formed. This Rocket Tile (Figure 3.4) is considered a fundamental unit in the Rocket Chip SoC system and can be replicated to create multi-core processor systems, meeting the demands for performance and flexibility in modern SoC design.

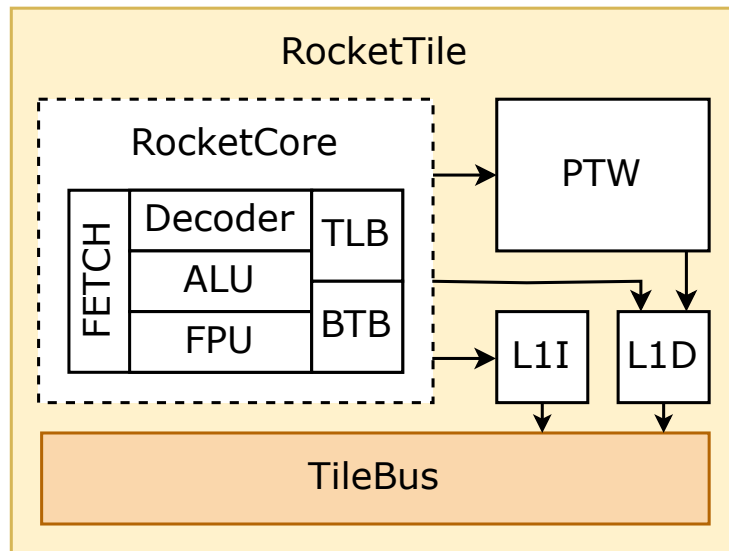


Figure 3.4: Structure of a Rocket Tile.

Due to the object-oriented nature of Chisel, the language used for its design, Rocket itself is written from object classes that can be defined and reconfigured according to implementation needs, such as `WithNBigCore()`, `WithNMedCore()`, `WithNSmallCore()`, and `WithNTinyCore()`. Additionally, instead of implementing the Rocket core, the BOOM core can also be integrated in its place, and many other peripherals can be integrated into a Tile unit.

3.4 Rocket Chip SoC Structure

This system generator includes multiple components, not just the processor core but also the on-chip system bus (SoC bus), specifically TileLink. TileLink comprises a peripheral bus, control bus, and memory bus, along with arbiters and system controllers. Additionally, the system integrates peripheral components such as DRAM, GPIO, UART, and, critically for this thesis, supports tightly-coupled accelerators like Gemmini via the RoCC interface. Figure 3.5 below depicts the generalized structure

of the single-core RISC-V processor system with a Gemmini RoCC accelerator, which forms the basis for the development in this thesis.

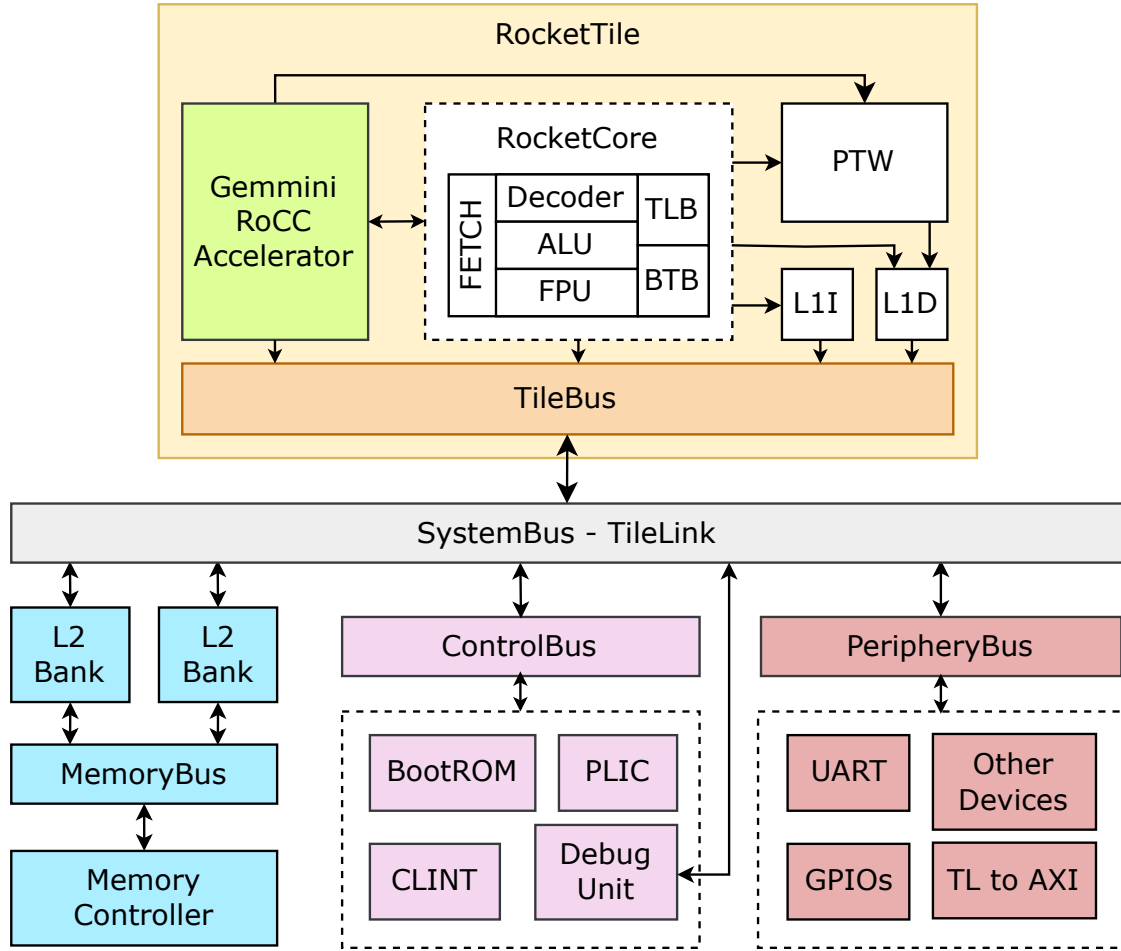


Figure 3.5: Proposed structure of a single-core RISC-V processor system with a Gemmini RoCC accelerator.

3.4.1 Bus and Interconnect

In the design of Systems-on-Chip (SoCs), bus architecture plays a backbone role, connecting and coordinating the operation of all components within the system. Particularly for modern multi-core processor systems, development trends focus on using crossbar-type bus architectures instead of Time Shared Buses. A crossbar, specifically a fully-connected crossbar structure, is considered a form of hardware switching fabric, allowing any input port to connect directly to any output port simultaneously. The outstanding advantage of this architecture lies in its ability to provide high bandwidth and low latency, effectively meeting performance requirements in applications such as networking equipment, multi-core processors, and high-performance storage

systems. The optimization in data transmission offered by this architecture is a key factor in improving the overall performance of modern SoC systems.

For Rocket Chip, the TileLink [19] system communication bus was developed concurrently with the project and the RISC-V instruction set, and is widely used in open-source system research at present.

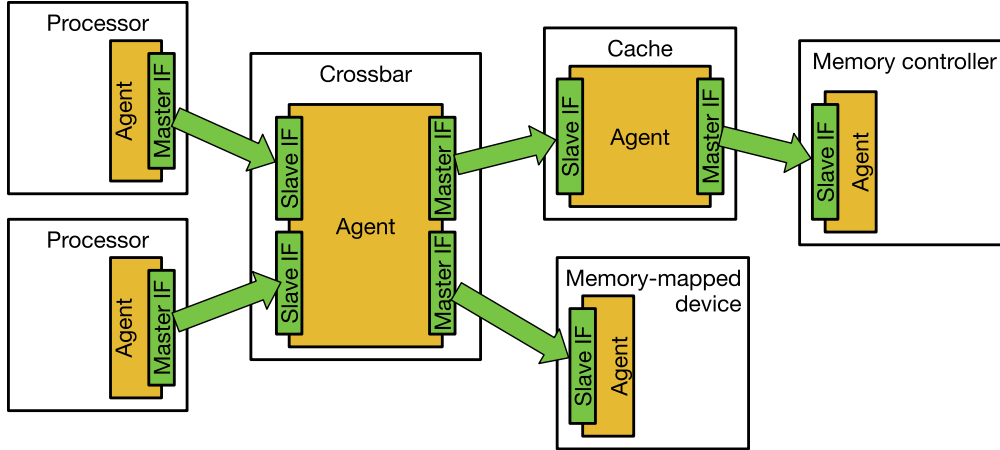


Figure 3.6: Connection model in TileLink and its interfaces. Source: [19]

TileLink is a highly parameterizable, open-source, shared memory interconnect protocol based on the aforementioned crossbar structure, designed to connect diverse modules for SoCs. TileLink is implemented via a hierarchical, point-to-point network that can be easily scaled. This protocol provides memory-mapped access for multiple controllers, ensuring memory coherency while supporting efficient communication between processors, DMAs, and peripheral devices.

A TileLink bus system can support a combination of multiple communicating agents, each capable of supporting different configured subsets of the protocol. The TileLink specification includes three levels of operational configuration for agents, presented in Table 3.1.

Table 3.1: TileLink operational configurations. Source: [19]

	TL-UL	TL-UH	TL-C
Read/Write operations	y	y	y
Multibeat messages	.	y	y
Atomic operations	.	y	y
Hint operations	.	y	y
Cache block transfers	.	.	y
Channels B+C+E	.	.	y

TileLink clearly distinguishes between cached and uncached communications. The simplest level is TileLink Uncached Lightweight (TL-UL), which only supports basic read and write operations to memory, for individual words. The next, more complex level is TileLink Uncached Heavyweight (TL-UH), which adds features such as "hints," "atomic" operations, and burst accesses but does not support coherent caching. Here, "hints" refers to a feature where a request command seeks data information external to the executing command from masters, and "atomic" refers to operations that lock access rights in a system with multiple masters, which could be microprocessors. Finally, TileLink Cached (TL-C) is the full protocol, supporting the use of coherent caches within the system.

For integrating peripheral hardware, all three bus configuration levels can be used. The communication connection diagram, at its most complete level, between a custom peripheral hardware and an agent is shown in Figure 3.7.

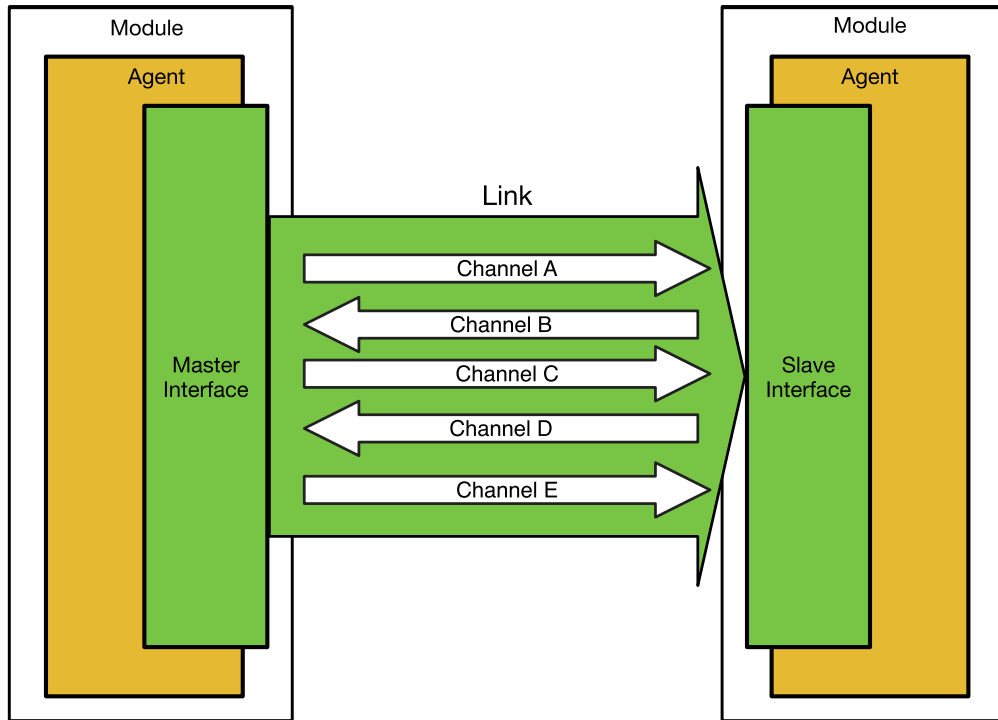


Figure 3.7: Channels in a TileLink connection between two agents. Source: [19]

In this TileLink protocol, the defined communication channels include A, B, C, D, and E, each undertaking specific roles in the interaction process between a master (M) and a slave (S), and all have a defined direction of transmission as depicted in the figure above.

- **Channel A:** This is the initiation channel, where M sends requests to S,

establishing the initial step of the communication process.

- **Channel B:** This channel is used when M requests a cache block from S, ensuring necessary data is queried.
- **Channel C:** Used for S to respond to M's request for a cache block, playing a crucial role in providing necessary data to M.
- **Channel D:** This is the general response channel, where S sends the final response to M after processing requests.
- **Channel E:** This channel ensures transaction completion by performing the final handshake for cache block transfer between the two parties.

The priority order for these channels is as follows: $A < B < C < D < E$, where channel A has the lowest priority and E has the highest. Prioritization in the TileLink bus system ensures that messages transmitted through the system do not fall into a hold-and-wait loop. In other words, the message flow through all channels between agents is maintained as a directed acyclic graph, enabling the TileLink protocol to operate without encountering deadlocks, ensuring continuous and uninterrupted data flow within the system.

The choice of TileLink bus configuration is relevant for connecting various memory-mapped peripherals within the system. For general-purpose MMIO peripherals, which might include cryptographic algorithm accelerators designed in Verilog HDL (if they were to be integrated as MMIO devices), the TL-UL configuration, comprising only channels A and D between M and S (as shown in Figure 3.8), is often sufficient. However, for the primary accelerator in this thesis, the Gemmini core, integration is achieved through the RoCC interface, which provides a more tightly-coupled path to the processor core, rather than as a TileLink-attached MMIO peripheral. Details on the RoCC interface are discussed further in Section 3.4.4. For other standard MMIO peripherals, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to connect their signals to the appropriate system bus, typically the peripheral bus.

At the system configuration level, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to convert the signals of the accelerator core to the system bus, in this case, the peripheral bus. Details for this interface will be further elaborated in subsequent sections of this thesis.

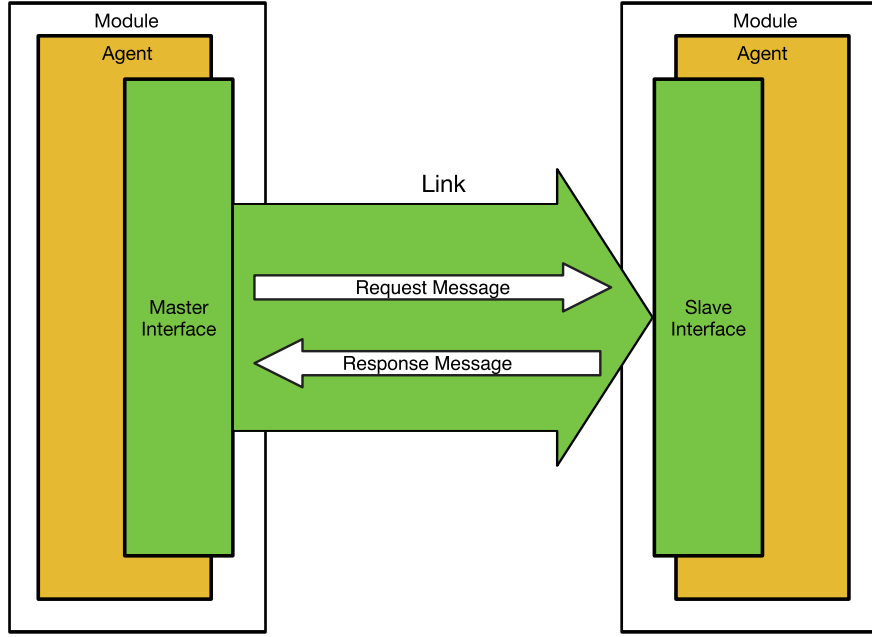


Figure 3.8: TileLink TL-UL connection structure. Source: [19]

3.4.2 Memory and Cache

Each Tile in the system is integrated with a processor core and an L1 cache that is flexibly configurable. By default design with Chisel, the L1 cache has a size of 16 KiB, organized multi-dimensionally with sets and ways. The L1 cache uses a set-associative cache structure, dividing the memory region into sets, each set containing a fixed number of data blocks (ways). When a memory address access request occurs, the index derived from the address maps it to a specific set. The cache controller then searches for the data block within the ways of that set. This design supports both instruction and data storage, optimizing access performance and increasing the hit rate compared to caches with lower associativity.

In addition to the L1 cache, the system also integrates a shared L2 cache, connected to all cores via the TileLink bus system. The L2 cache acts as an intermediate storage repository, allowing processor cores and other components in the system to perform read/write data requests. This is a coordination mechanism that helps reduce pressure on the main memory and improves overall processing performance. The L2 cache controller also ensures fairness in resource allocation, using a queuing system to manage access requests.

At a higher level, the system also supports the integration of external memory, such as DRAM or SRAM, through open-source controllers or specialized IPs from FPGA implementation toolkits. Additionally, an interface with DRAMSim is sup-

ported for integration when performing full-system simulation. This provides flexibility and scalability, meeting various requirements in practical applications.

3.4.3 Core-Complex

As previously mentioned, the Chipyard framework and library consist of many component definition classes that can form a Rocket Chip system. This allows for the formation of a complex system with diverse processor cores and peripherals, specifically heterogeneous architecture systems comprising multiple different processors like Rocket and BOOM coexisting on the system.

In these multi-core processor structures, each individual core will be uniquely identified with a different hart ID, ordered according to the system configuration. An example can be referenced below with a configuration comprising one Rocket core and one BOOM core. The corresponding block diagram configuration when generating the system will be as shown in Figure 3.9 below.

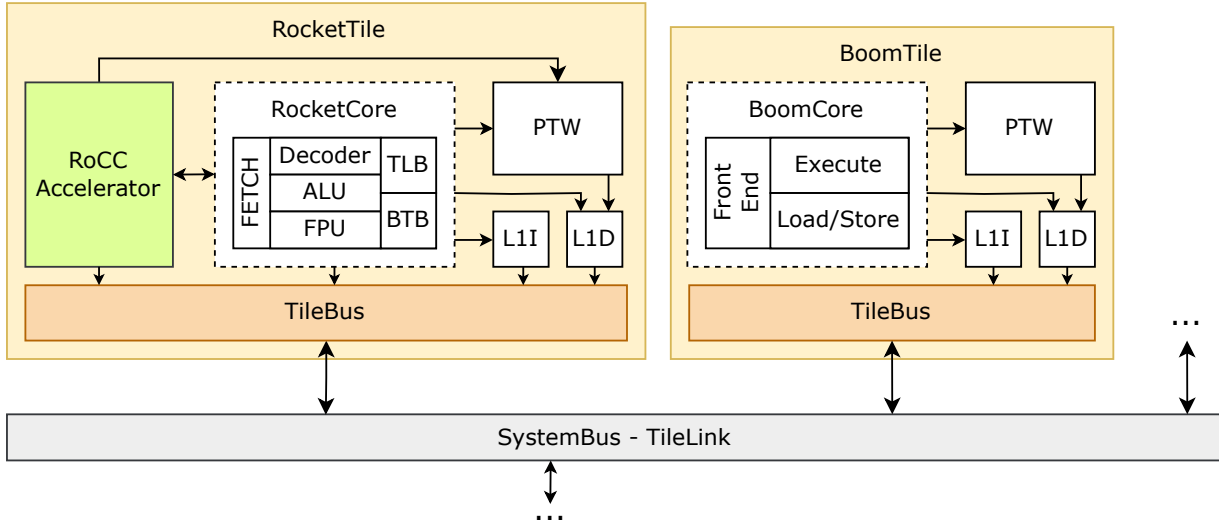


Figure 3.9: Implementation structure of a heterogeneous multi-core processor system.

A heterogeneous SoC system is an implementation approach also considered in this thesis to combine different processor cores to optimize performance and energy efficiency. At the application scale, lightweight cores can handle simple tasks with low power consumption, while high-performance cores support hyper-threading and out-of-order execution to process complex tasks. This design ensures flexibility, resource optimization, and superior performance for diverse applications.

3.4.4 Peripheral Components

In the base system configuration class of Rocket Chip, many peripheral components are initialized and integrated by default to support both simulation and deployment on FPGA hardware for the SoC system. This configuration includes basic I/O devices such as PWM, interrupt controllers, JTAG, ROM, external memory, and standard peripheral interfaces, ensuring that basic communication, storage, and control requirements are met. In particular, common peripheral devices like UART, SPI Flash, and GPIO are implemented to support serial communication, storage management, and flexible control within the system.

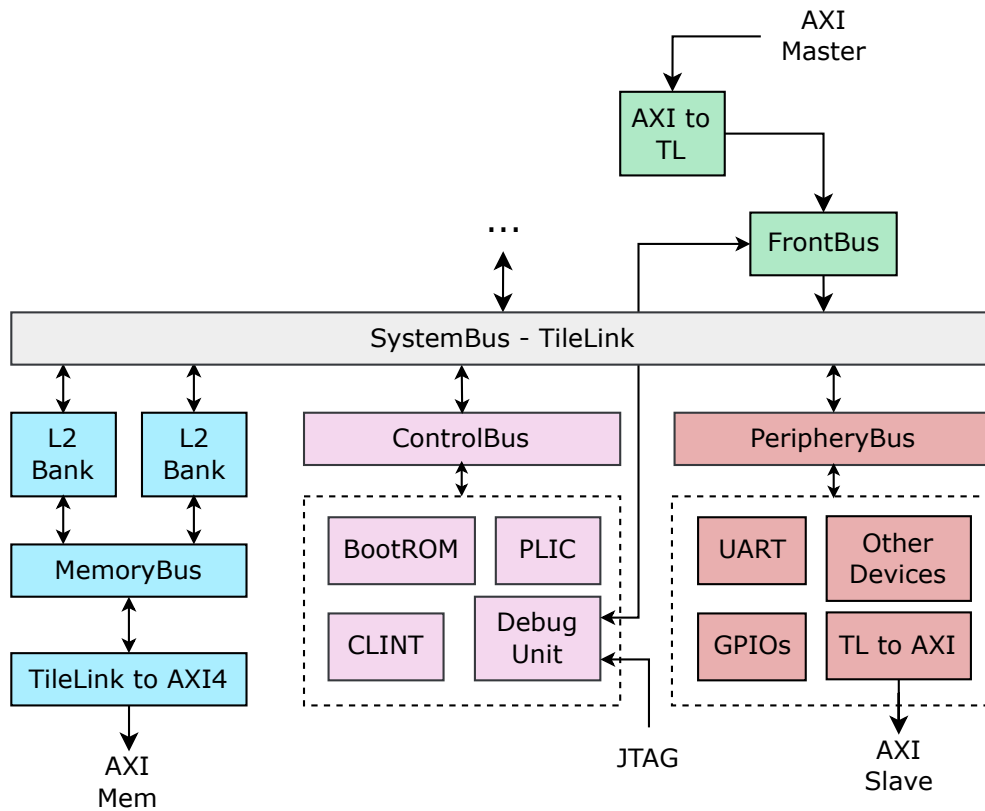


Figure 3.10: Peripherals in the Rocket Chip base system.

All these components are developed based on the open-source Rocket Chip library, with hardware designs like Rocket and BOOM realized on the Chisel platform. When external memory is needed in design and simulation, the **AXI4** (Advanced eXtensible Interface 4) interface can be integrated as an intermediate layer, allowing effective connection with DRAM or SPI Flash models. Concurrently, serial protocols like JTAG are integrated to ensure accurate debugging capabilities and efficient serial communication, configured to connect with the TileLink bus. The Rocket Chip

system also supports flexible configuration through special control ports, including custom boot pins and chip identification ports. Furthermore, the Rocket Chip system provides the capability to directly map bus interfaces such as AXI4 Memory and MMIO (Memory-Mapped IO), allowing for expanded compatibility and optimized performance in complex SoC applications.

To expand the capability of integrating components and peripheral cores to form highly parameterized custom systems suitable for specific applications, the Rocket Chip architecture provides two flexible methods for peripheral integration. Specifically, the system can integrate a peripheral core as a RoCC (Rocket Custom Co-processor), located within each tile, or integrate at the system communication level via the MMIO mechanism. These methods facilitate in-depth customization, meeting various requirements for each specific application.

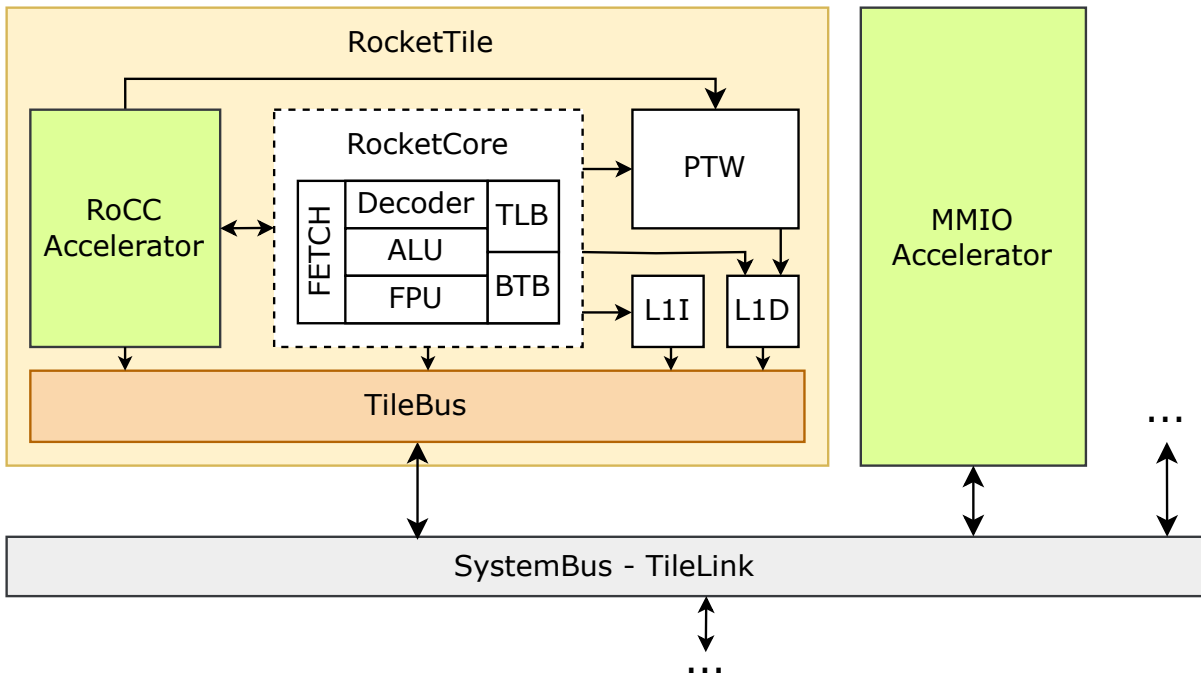


Figure 3.11: RoCC co-processor core and MMIO peripheral core in Rocket Chip.

MMIO Peripherals are custom peripherals attached directly to the TileLink bus, while Tightly-Coupled RoCC Accelerators are custom peripherals attached directly to the arbiter and processor within each Tile, in parallel with the processor core. With the TileLink-Attached MMIO method, the processor communicates with MMIO devices through registers mapped into memory at addresses on the system. Through these registers, control mechanisms or data transfer can be customized to suit the application of the peripheral core. Conversely, in the RoCC Accelerator method, com-

munication is performed via a custom protocol and non-standard ISA instructions, which are configured and defined separately within the encoding space of the RISC-V ISA. For Rocket and BOOM, each core can control up to four accelerators through custom opcodes, sharing resources with the microprocessor. Instructions executed with RoCC cores have the format `customX rd, rs1, rs2, funct`. Here, `X` is a number from 0-3, determining the opcode that controls the routing of the instruction to a specific accelerator. The fields `rd`, `rs1`, `rs2` are the register numbers for the destination register and two source registers, and the `funct` field is a 7-bit integer that the accelerator can use to differentiate various instructions.

Peripherals connected as RoCCs have a distinct advantage due to their high customizability and direct connection and control via custom ISA instructions. This allows for specialized performance optimization, particularly useful for cores requiring large, complex computational workloads or applications needing extremely fast response times. Communication via a custom protocol and leveraging the RISC-V ISA helps RoCC utilize CPU resources efficiently, while also offering flexibility in designing specialized accelerator processors. The Gemmini accelerator, central to this thesis, is a prime example of a RoCC-based component, designed for high-throughput matrix multiplication and tightly integrated with the Rocket core.

While RoCC implementation can require a custom software toolchain for its specialized instructions, the performance benefits for demanding applications like those targeted by Gemmini often justify this investment. For this thesis, the integration of the Gemmini accelerator via the RoCC interface is deemed the most appropriate approach to achieve the desired performance for deep learning workloads.

Conversely, the MMIO TileLink-Attached method is widely used for integrating general-purpose memory-mapped peripheral devices. In this model, peripheral devices provide a set of registers (e.g., Chisel Registers) to communicate with microprocessors or other masters on the system bus. This method is suitable for peripherals where the tight coupling and custom instruction set of RoCC are not necessary.

To minimize design complexity for MMIO peripherals, the Rocket Chip architecture provides the `regmap` interface, which allows for the automatic generation of much of the necessary linking logic. Designers can use `TLRegisterRouter` or create a `LazyModule` and initialize `TLRegisterNode` to integrate such devices. Since TileLink is the official protocol used in SoCs based on Rocket Chip and Chipyard, designing MMIO peripheral devices (other than the primary RoCC accelerator) would typically focus on integration with this protocol.

Chapter 4: Architectural Foundations of DNN Acceleration

The efficient execution of Deep Neural Networks (DNNs) presents one of the most significant challenges and opportunities in modern computer architecture. While today’s general-purpose CPUs are highly optimized for a wide range of tasks, their fundamental design principles are often mismatched with the unique computational and data movement patterns of large-scale neural networks. This chapter will establish the foundational concepts that motivate the need for specialized hardware. We will begin by exploring the core problem—the immense cost of data movement—and then introduce the classic architectural paradigm designed to solve it: the systolic array. Finally, we will examine how this timeless principle is embodied in state-of-the-art industrial accelerators, providing the necessary context to understand the design of the Gemmini accelerator, which is the central subject of this thesis.

4.1 The Data-Compute Divide: The Memory Wall

The performance of any computing system is ultimately limited by two factors: the speed of its computations and the speed at which it can supply data to its computational units. For decades, Moore’s Law drove exponential growth in the number of transistors on a chip, leading to dramatic increases in processing power. However, the speed of off-chip memory systems, such as DRAM, has improved at a much slower pace. This growing disparity between processor speed and memory speed is famously known as the **Memory Wall** [23].

For data-intensive workloads like DNNs, this is the primary performance bottleneck. The problem can be understood with a simple analogy: imagine a master chef (the processor) who can perform culinary tasks with superhuman speed. However, their ingredients (the data) are stored in a large warehouse across the street (DRAM). Even though the chef is exceptionally fast, their overall productivity is dominated by the time spent walking to and from the warehouse to fetch each ingredient.

This bottleneck is not just about latency; it is fundamentally about energy. As quantified by Horowitz [12], the energy required to perform a complex 32-bit floating-point computation is dwarfed by the energy needed to fetch its operands from off-chip DRAM (Figure 4.1). This immense energy cost of data movement means that for DNNs, which perform billions of operations, any viable high-performance hardware solution must be designed with a primary objective: to minimize data movement.

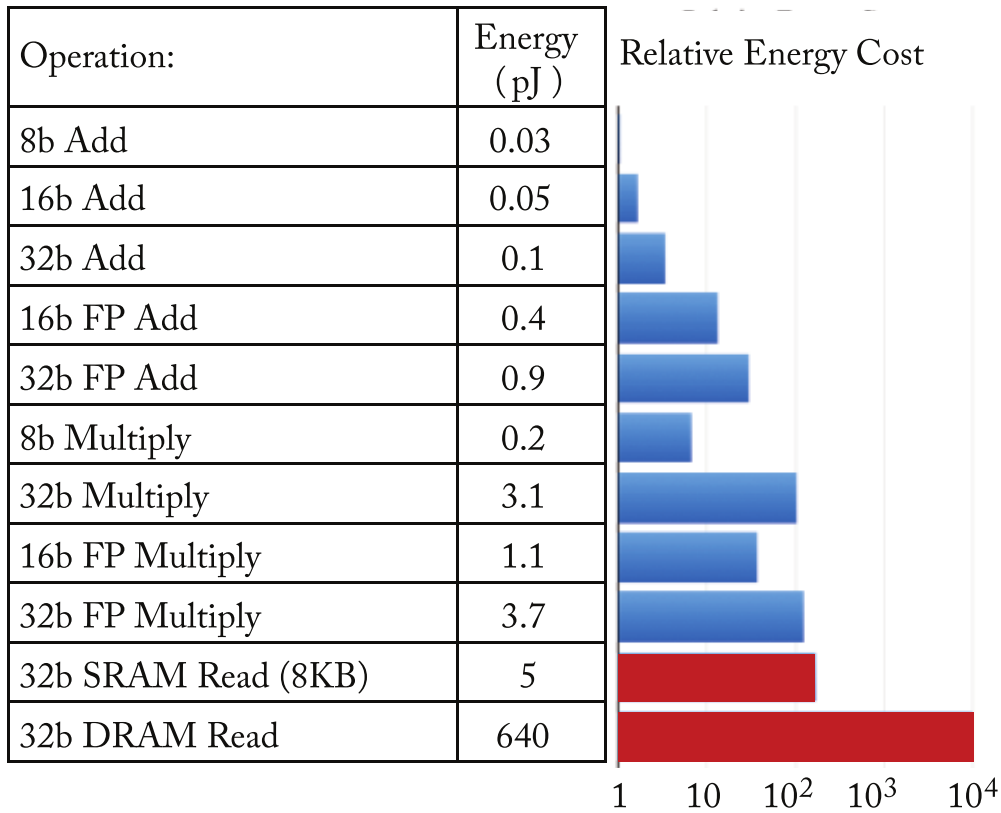


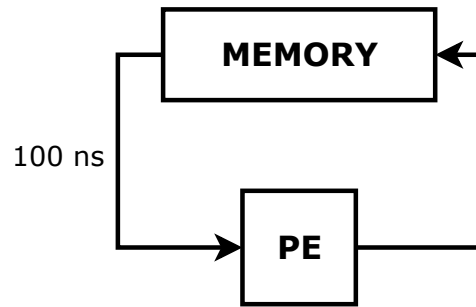
Figure 4.1: The relative energy cost of various arithmetic and memory operations. A 32-bit DRAM read is orders of magnitude more costly than a 32-bit floating-point addition. This disparity is the primary motivation for specialized accelerator architectures. Adapted from Horowitz, as presented in Sze et al. [20, 12].

4.2 A Classic Solution: The Systolic Principle

In his seminal 1982 paper, H.T. Kung proposed a novel architectural paradigm designed specifically to address the memory wall: the **systolic array** [15]. The architecture is named by analogy to the human circulatory system. Just as the heart rhythmically pumps blood to the body's cells, in a systolic system, memory "pumps" data through a regular grid of simple Processing Elements (PEs).

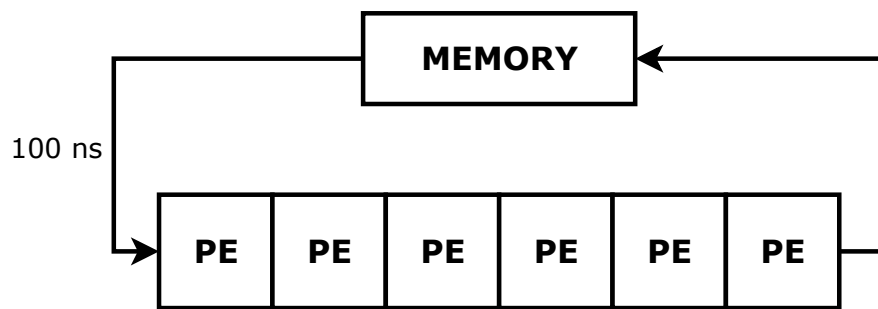
The core principle is to **maximize the number of computations performed for each data item fetched from memory**. Instead of fetching an operand, using it once in a central ALU, and then discarding it (the conventional approach), a systolic array passes an operand from one PE to the next in a pipelined fashion. At each step of its journey, the data element participates in another computation. This structure, illustrated conceptually in Figure 4.2, amortizes the high cost of the initial memory access over dozens or even hundreds of useful operations.

INSTEAD OF:



5 MILLION
OPERATIONS
PER SECOND
AT MOST

WE HAVE:



30 MOPS
POSSIBLE

THE SYSTOLIC ARRAY

Figure 4.2: A conceptual comparison of a conventional von Neumann architecture and a systolic architecture. The conventional approach (top) is bottlenecked by the memory bus. The systolic approach (bottom) uses an array of PEs to achieve high throughput with the same memory bandwidth by reusing data internally. Adapted from the presentation of Kung's work [15].

4.3 Case Studies in Modern Systolic Design

While the systolic principle is four decades old, it has become the dominant paradigm for modern high-performance DNN accelerators. We will examine two landmark industrial designs that showcase this principle in practice.

4.3.1 Google’s TPUv1: A Datacenter-Scale Design

Google’s Tensor Processing Unit (TPU) was one of the first large-scale custom ASICs designed specifically for accelerating DNN inference in datacenters [13]. As shown in Figure 4.3, its architecture is a direct and powerful embodiment of the systolic principle. The heart of the TPU is a massive 256x256 Matrix Multiply Unit, a systolic array of 65,536 8-bit multiply-accumulate (MAC) units.

The TPUv1’s design philosophy is a masterclass in optimizing for a specific domain: high-throughput datacenter inference. Several key decisions reflect this focus. First, the use of 8-bit integer arithmetic was a critical trade-off, sacrificing the precision of floating-point numbers for immense gains in area and energy efficiency. This was based on the insight that for inference, the resilience of neural networks to quantization allows for acceptable accuracy with lower-precision data. Second, the enormous on-chip Unified Buffer (a 24 MiB software-managed scratchpad) is essential to "keep the beast fed"—it buffers entire layers of weights and activations on-chip to ensure the massive systolic array is never starved for data. This design choice implicitly favors large batch sizes, where the cost of loading the model can be amortized over many simultaneous inference requests, prioritizing throughput over single-request latency.

4.3.2 NVIDIA NVDLA: An Open-Source Accelerator IP

In contrast to the monolithic, datacenter-focused TPU, the NVIDIA Deep Learning Accelerator (NVDLA) is an open-source, configurable IP core designed for integration into a wide range of Systems-on-Chip (SoCs), particularly for embedded systems [7]. The NVDLA’s design philosophy prioritizes **modularity and integration flexibility** over raw scale.

As shown in Figure 4.4, the architecture is partitioned into distinct, specialized hardware units for convolution, post-processing, and memory management. This modularity is a key feature, allowing an SoC designer to integrate only the necessary components. A significant design choice differentiating the NVDLA from the TPU is its reliance on industry-standard bus protocols like AXI. While this may introduce

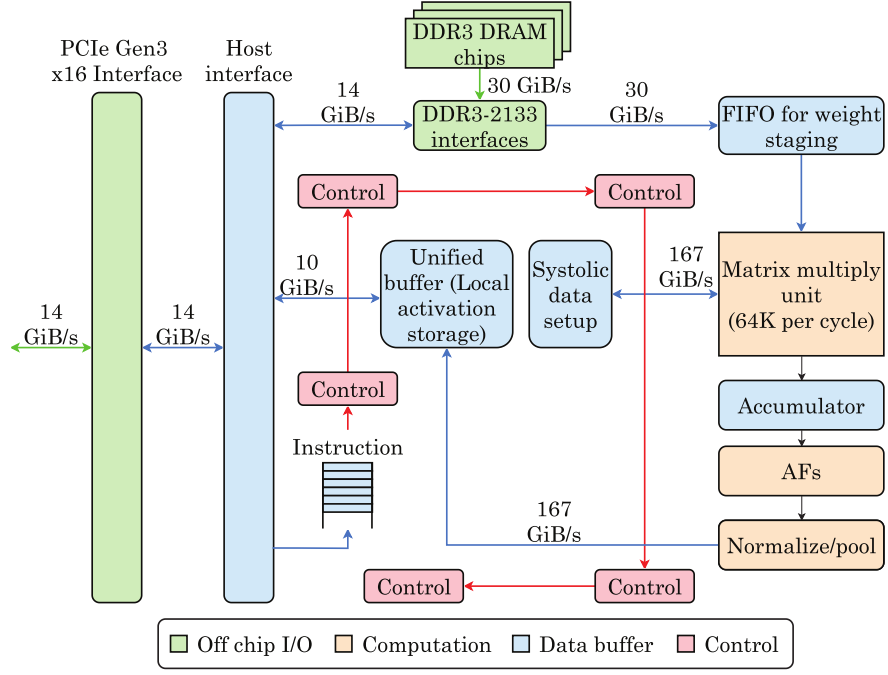


Figure 4.3: A high-level block diagram of the Google TPUv1. The design is dominated by the systolic Matrix Multiply Unit and its large on-chip Unified Buffer. Adapted from Jouppi et al. [13, 16].

more protocol overhead than a custom interface, it dramatically simplifies the process of integrating the NVDLA into a third-party SoC from a different vendor. Furthermore, its configurability (e.g., in the number of MAC units from the `nv_small` to `nv_large` configurations) makes it a versatile solution that can be tailored to the specific power, performance, and area (PPA) constraints of an embedded device.

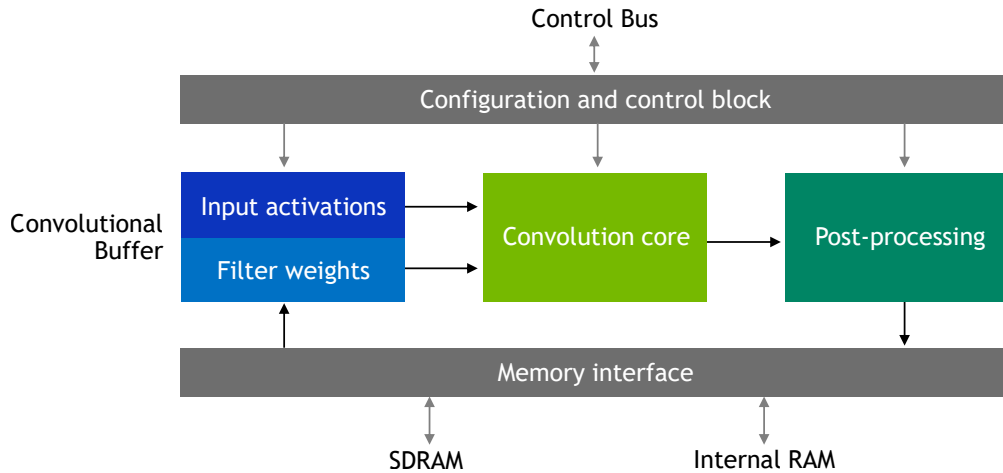


Figure 4.4: The high-level architecture of the NVDLA. The design is partitioned into distinct functional units connected via standard bus interfaces. [7].

4.4 A Formal Framework: Dataflows

The design choices made in architectures like the TPU and NVDLA can be formally described using the concept of a **dataflow**. The dataflow defines the strategy for moving data through the systolic array to maximize reuse. The effectiveness of a given dataflow depends on the structure of the DNN layer being executed. The two most common dataflows, which we will see are directly relevant to Gemmini, are:

- **Weight Stationary (WS):** This dataflow prioritizes weight reuse. Filter weights are pre-loaded into the PEs and remain stationary, while input activations are streamed in. This strategy, used by the Google TPU, is highly efficient for layers with large input feature maps and relatively small filters.
- **Output Stationary (OS):** This dataflow prioritizes minimizing the movement of partial sums. Each PE is responsible for accumulating the final value for a single output activation, which remains stationary in its accumulator. Inputs and weights are streamed to the PE as needed. This can be more efficient for layers where partial sum read/write energy is a dominant cost.

These strategies, visually contrasted in Figure 4.6 and 4.7, represent a fundamental trade-off in accelerator design. The choice of dataflow directly impacts which types of data reuse, shown in Figure 4.5, are most effectively exploited.

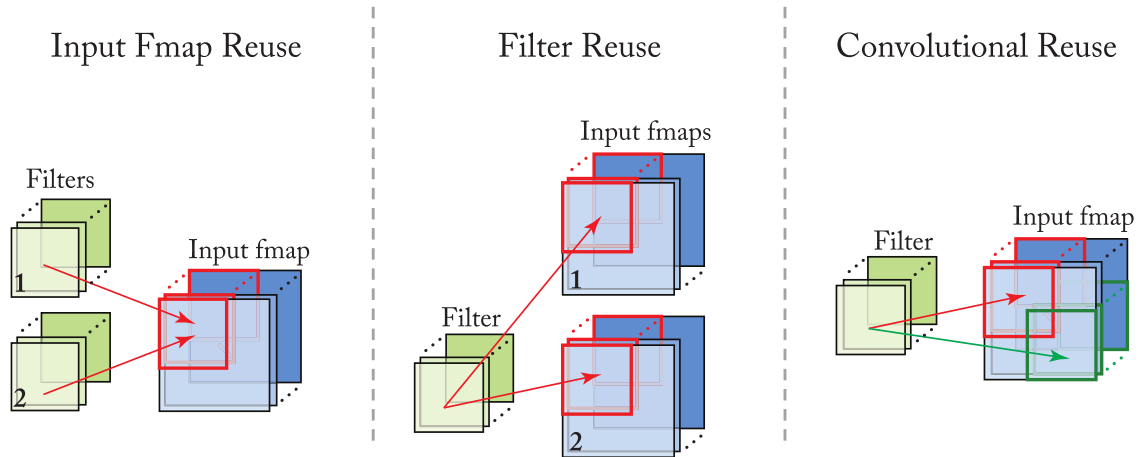


Figure 4.5: The three primary forms of data reuse in convolutional layers: convolutional, filter, and input reuse. [20].

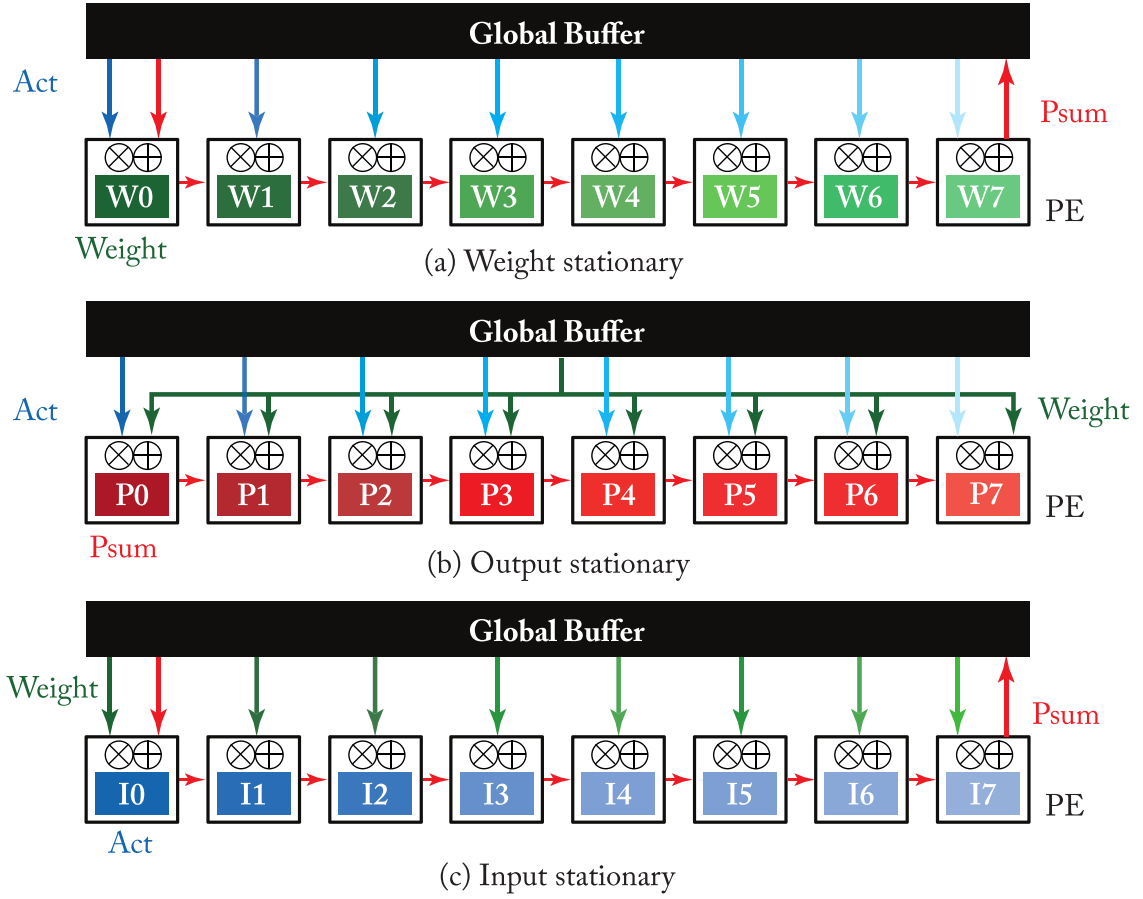


Figure 4.6: A visual comparison of Weight Stationary (left) and Output Stationary (right) dataflows. [20].

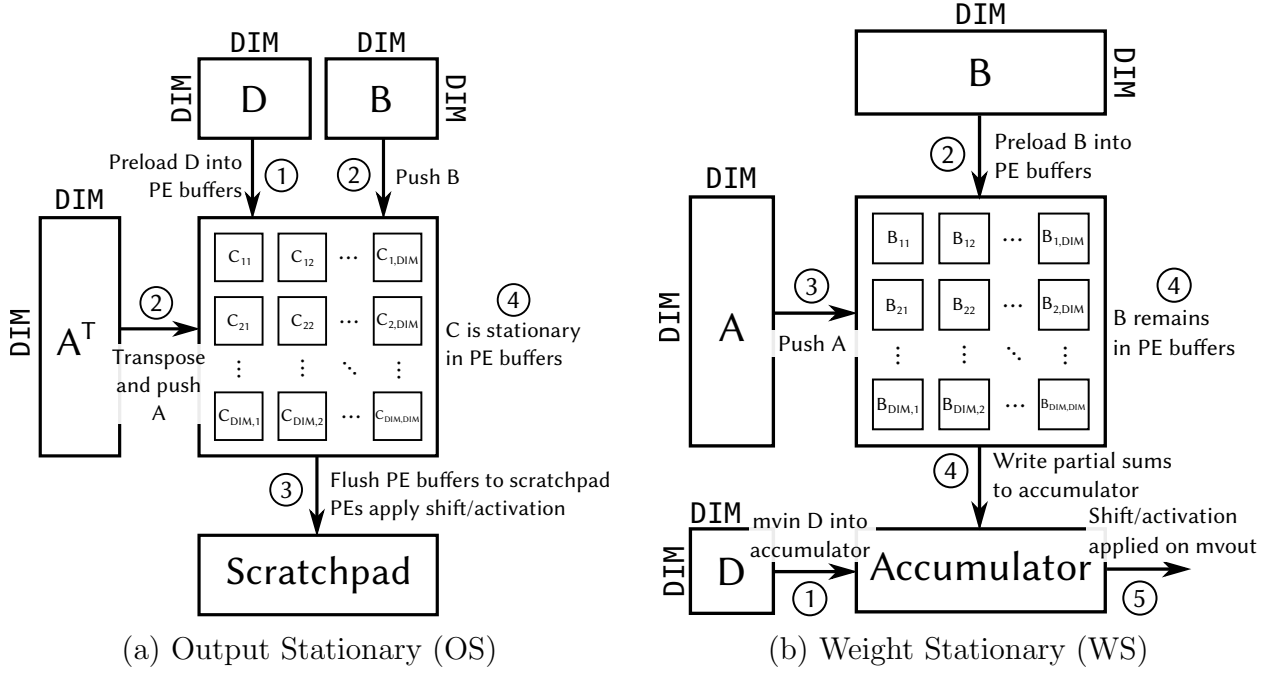


Figure 4.7: A visual comparison of the two primary dataflows supported by Gemmini. (a) In the **Output Stationary** dataflow, the result matrix C remains stationary in the PEs to accumulate partial sums. (b) In the **Weight Stationary** dataflow, the weight matrix B is preloaded and remains stationary in the PEs to maximize weight reuse. The choice between these strategies represents a fundamental trade-off in accelerator design. Source: [10].

4.5 The Gemmini Approach: A Full-Stack Research Platform

While the TPU demonstrates the immense potential of large-scale systolic arrays and the NVDLA showcases the value of modular, open-source IP, both present challenges for comprehensive, system-level academic research. The TPU is a proprietary, closed-source design, limiting deep architectural inspection. The NVDLA, while open, is often evaluated as an isolated hardware block, which can obscure the real-world performance bottlenecks that arise from interactions with the host CPU, operating system, and shared memory hierarchy [10].

To address these challenges, this thesis utilizes **Gemmini**, an open-source, *full-stack DNN accelerator generator* from the University of California, Berkeley [10]. Gemmini’s design is founded on the philosophy that to truly understand an accelerator’s performance, it must be evaluated as part of a complete, functioning system. It achieves this by leveraging the Chipyard framework to generate not only the accelerator RTL but also a Linux-capable SoC, enabling the study of critical cross-stack effects such as:

- **SoC Resource Contention:** Competition for shared resources like the L2 cache and the main memory bus between the CPU and the accelerator.
- **Operating System Overhead:** The performance impact of OS services, including context switches and page fault handling.
- **Virtual Memory Interaction:** The latency and performance implications of the accelerator’s DMA engine interacting with the system’s TLBs and Page Table Walker.

Crucially, as a generator written in Chisel, Gemmini is not a single, fixed hardware design. It is a highly flexible template that can be configured at generation time to produce a wide spectrum of systolic architectures. As illustrated in Figure 4.8, by tuning its parameters, Gemmini can generate designs ranging from highly-pipelined, TPU-like structures to more loosely-coupled, NVDLA-like parallel vector engines. This architectural flexibility makes it an exceptionally powerful platform for academic research and design-space exploration.

Therefore, for the objectives of this thesis—to implement and evaluate a custom accelerator within a realistic SoC environment—Gemmini provides the ideal platform. It combines the computational power of a systolic array with the verifiability of a full-stack, open-source framework, enabling a rigorous and holistic analysis.

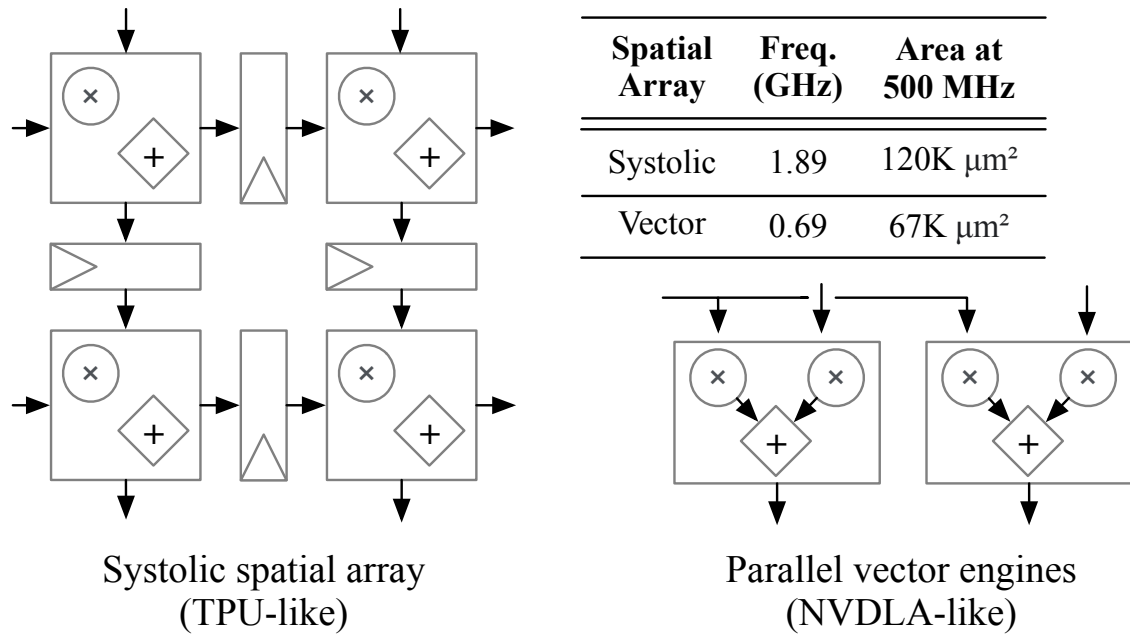


Figure 4.8: The design space enabled by the Gemmini generator, allowing it to produce a wide variety of spatial array structures, from systolic (TPU-like) to parallel vector engines (NVDLA-like). Source: [10].

4.6 Chapter Summary

This chapter has built a foundational argument for the necessity of specialized hardware for DNN acceleration. We began with the core architectural challenge, the Memory Wall, and introduced the systolic array as a classic and enduring solution that minimizes costly data movement by maximizing data reuse. By examining modern industrial accelerators like the Google TPU and NVDLA, we have seen this principle in action. Finally, we formalized these strategies with the concept of dataflows, such as Weight Stationary and Output Stationary. These foundational concepts provide the necessary framework to understand the design and operation of the Gemmini accelerator, which will be the focus of the subsequent chapters.

Chapter 5: The Gemini DNN Accelerator: An Architectural Overview

As established in the previous chapter, the System-on-Chip (SoC) developed in this thesis utilizes a single Rocket Core augmented with a specialized accelerator for deep learning workloads. This chapter delves into the architecture of this accelerator: Gemini, a sophisticated component designed to overcome the fundamental limitations of general-purpose processors for Deep Neural Network (DNN) tasks.



Figure 5.1: Gemini Logo. Source: [10].

5.1 Top-Level Architectural Blueprint

The Gemini generator can produce a wide range of accelerator instances based on a flexible architectural template. To understand the Gemini architecture, we first examine the high-level template shown in Figure 5.2. This blueprint illustrates the three primary components of the accelerator: the Controller, the on-chip memory (Scratchpad and Accumulator), and the compute core (Spatial Array), and their interface with the host CPU and system DRAM. The following sections provide a detailed analysis of each of these blocks.

5.2 Analysis of Core Architectural Components

5.2.1 The Systolic Compute Engine

The computational heart of Gemini is its systolic array, a highly configurable spatial array that functions as a powerful General Matrix-Matrix Multiplication (GEMM) engine. The array's microarchitecture, as detailed in Figure 5.3, features a two-level hierarchy composed of Tiles and individual Processing Elements (PEs). Each PE is capable of performing a Multiply-Accumulate (MAC) operation in a single clock cycle. This hierarchical and configurable design allows Gemini to efficiently execute the dense matrix multiplications fundamental to modern Deep Neural Networks (DNNs).

The key parameters of the systolic array can be specified at generation time:

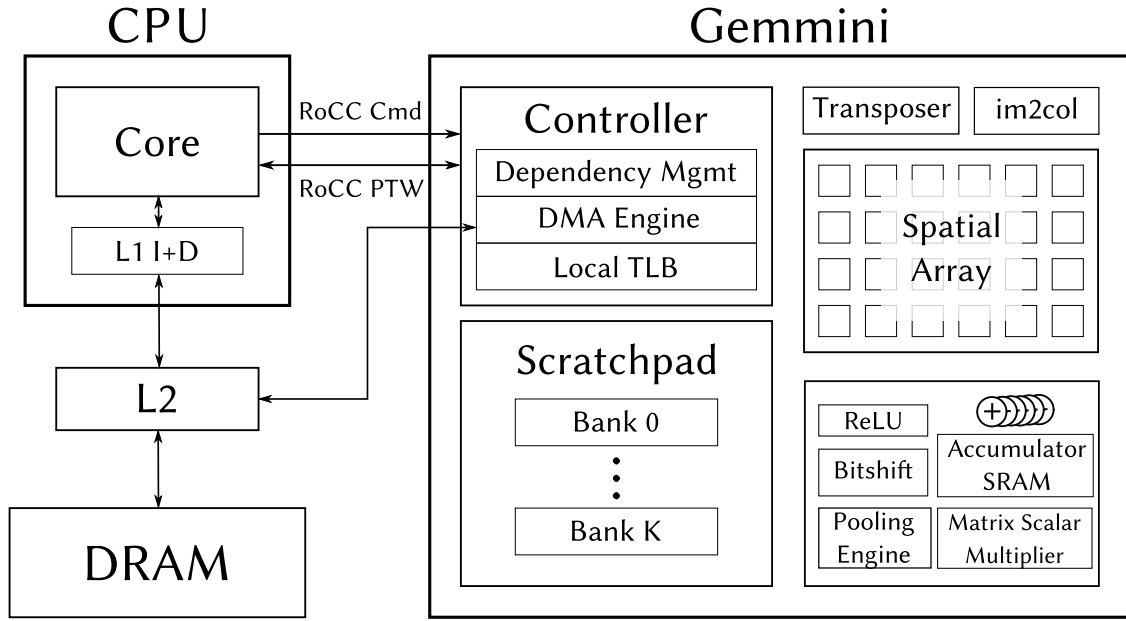


Figure 5.2: High-level overview of the Gemini accelerator template integrated with a host CPU and system memory hierarchy. Source: [10].

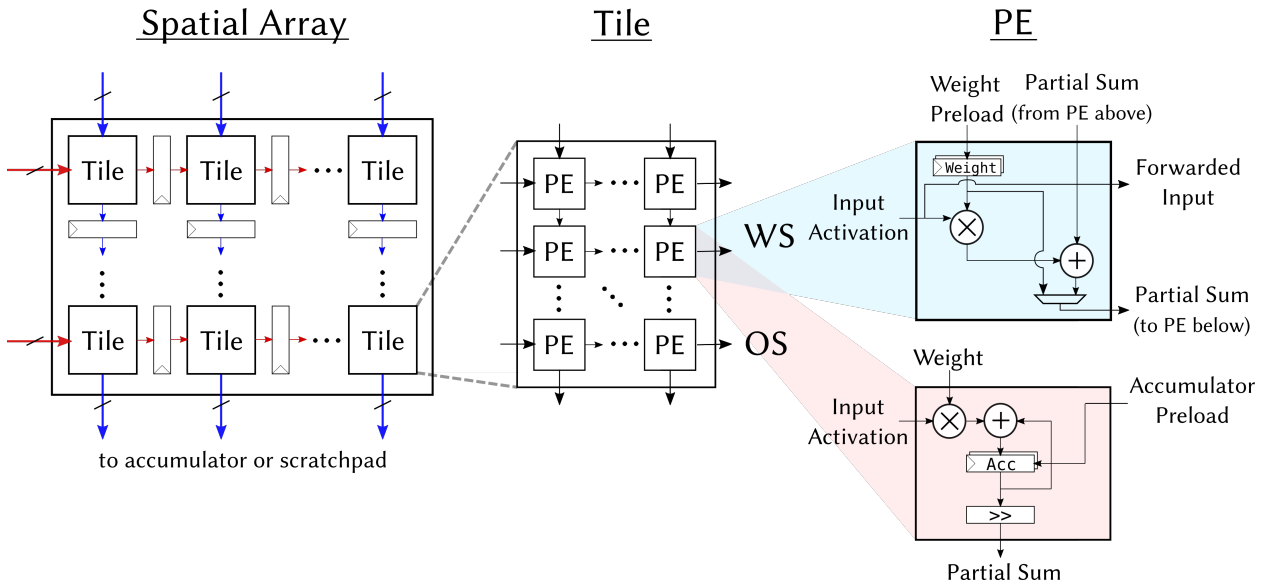


Figure 5.3: The two-level hierarchy of Gemini's spatial array, composed of tiles and Processing Elements (PEs). Source: [10].

- **Dimensions:** The size of the PE grid, structured into a hierarchy of tiles, can be configured (e.g., a 16x16 array composed of 2x2 tiles, each with 8x8 PEs). This allows for a trade-off between hardware area and parallel processing capability.
- **Dataflow:** Gemmini primarily supports two dataflow strategies: Weight Stationary (WS) and Output Stationary (OS).
 - In **WS dataflow**, the DNN weights are pre-loaded into the PEs and remain stationary, maximizing weight reuse as input activations are streamed through.
 - In **OS dataflow**, partial sums of the output activations remain stationary within the PEs, while both inputs and weights are streamed. This is particularly beneficial for layers with low weight reuse.

The choice of dataflow has a significant impact on performance and energy efficiency.

- **Pipelining:** The degree of pipelining within the array and its PEs can be configured, affecting the maximum clock frequency and overall throughput.

5.2.1.1 Mapping Convolution to Hardware: The `im2col` Transformation

While systolic arrays are fundamentally designed to accelerate General Matrix-Matrix Multiplication (GEMM), they are adept at accelerating 2D convolutions through a crucial data-restructuring technique known as `im2col` ("image-to-column"). This process unrolls the overlapping input image patches from a convolutional layer into the columns of a new, larger matrix. As shown in Figure 5.4, this new matrix can then be multiplied with a corresponding weight matrix, effectively **converting the convolution into a GEMM operation** that the systolic array can process with maximum efficiency.

Recognizing the computational cost of this pre-processing step, Gemmini can be configured with a dedicated hardware unit to perform the `im2col` transformation on-the-fly. This offloads the data restructuring from the host CPU, freeing it for other tasks and streamlining the overall computation pipeline. The decision to include this unit represents a classic area-versus-performance trade-off in the accelerator design.

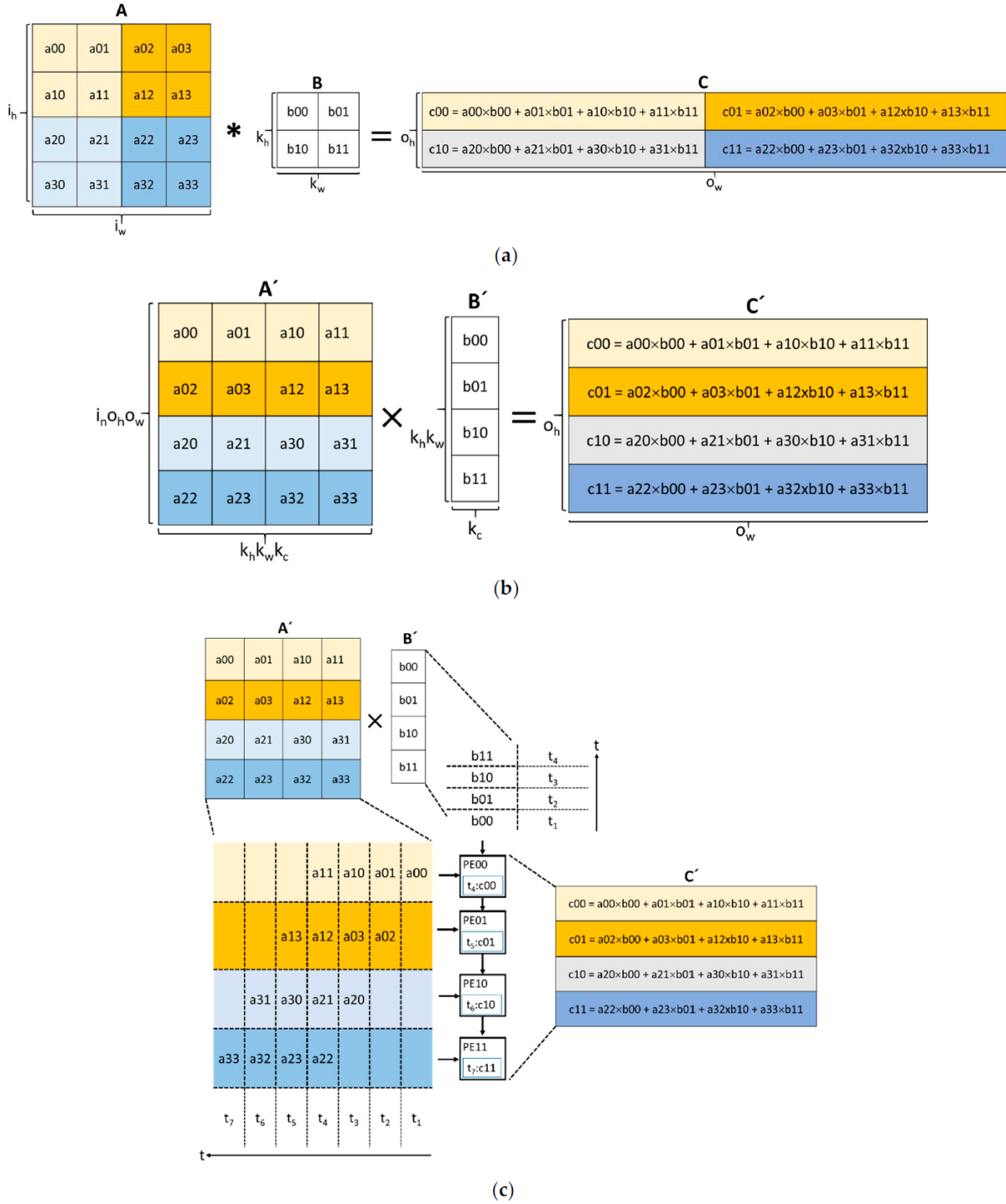


Figure 5.4: From convolution to systolic array computation: (a) direct convolution operation; (c) example of a systolic array operation; (b) A visualization of the `im2col`-based GEMM operation. The 3D input tensor (left) is transformed into a 2D matrix (center) that can be processed by the systolic array. Adapted from Gookyi et al. [11].

5.2.2 The On-Chip Memory Subsystem

To feed its high-throughput systolic array, Gemmini includes a specialized on-chip memory system. This design represents a deliberate trade-off between hardware-

managed caches and software-managed memories.

- **Scratchpad Memory:** A software-managed SRAM used for staging input activations, weights, and intermediate feature maps close to the systolic array. Its purpose is to reduce the latency and energy consumption associated with fetching data from higher levels of the memory hierarchy, such as the shared L2 cache or main memory (DRAM). Its capacity and banking are configurable.
- **Accumulator Memory:** Separate from the main scratchpad, Gemmini features a dedicated memory to store the partial sums generated by the MAC operations. This memory typically uses a higher precision data type (e.g., 32-bit integers) than the input data (e.g., 8-bit integers) to maintain accuracy during accumulation.

5.3 SoC Integration and System-Level Features

5.3.1 The RoCC Coprocessor Interface

Gemmini integrates into the Chipyard SoC via the **Rocket Custom Coprocessor (RoCC)** interface. As a tightly-coupled coprocessor, it receives custom RISC-V instructions dispatched directly by the host Rocket Core's pipeline. This allows for low-latency command and control.

This integration uses a decoupled access-execute model, as illustrated conceptually in Figure 5.5. The host CPU issues three main types of commands to Gemmini: load (from main memory to scratchpad), execute (perform computation on data within the scratchpad), and store (from scratchpad to main memory). These commands are placed into separate hardware queues, allowing Gemmini's internal controller then processes these commands asynchronously, and Gemmini's DMA engine to handle data movement in parallel with the systolic array's computations. This overlapping of communication and computation is crucial for hiding memory latency and maximizing the utilization of the systolic array.

5.3.2 Virtual Memory Support

To operate correctly under a full-featured operating system like Linux, an accelerator's DMA engine must be able to translate virtual memory addresses into physical addresses. Gemmini is designed with this "full-stack" capability. It can be configured

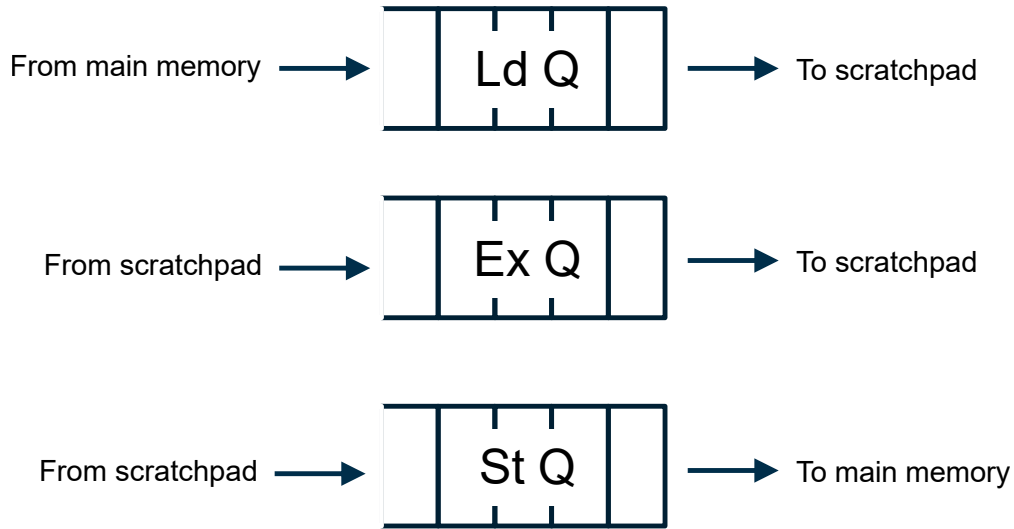


Figure 5.5: Conceptual view of Gemini’s decoupled access-execute pipelines, managed by separate hardware command queues for Load, Execute, and Store operations. Source: [10].

to share the host CPU’s Page Table Walker (PTW) to perform its own address translations, and it includes its own local Translation Lookaside Buffer (TLB) to cache these translations and reduce latency.

5.4 Summary

In summary, Gemini is a sophisticated, highly configurable DNN accelerator generator that embodies the principles of systolic computation. Its key architectural features include a powerful systolic array, a specialized on-chip memory hierarchy, and a tightly-coupled integration with a host processor via the RoCC interface, including support for system-level features like virtual memory. This design enables the high-performance execution of DNN workloads within a full-stack, system-level context.

Having established this architectural overview, the following chapter will now perform a deep dive into the specifics of Gemini’s internal command processing and its multi-level programming model.

Chapter 6: Gemini's Internal Architecture and Programming Model

While Chapter 5 described the conceptual architecture of Gemini, this chapter explores its internal structure and the software interfaces used to control it. A deeper understanding of these implementation-level details is necessary to effectively program the accelerator and analyze its performance.

6.1 Internal Command and Control Flow

A custom RoCC instruction sent from the Rocket core is not executed directly by the systolic array. Instead, it triggers a complex sequence of events within the Gemini accelerator's internal controller modules. The overall structure of the generated hardware, is shown in Figure 6.1. This section details the key components of this internal pipeline.

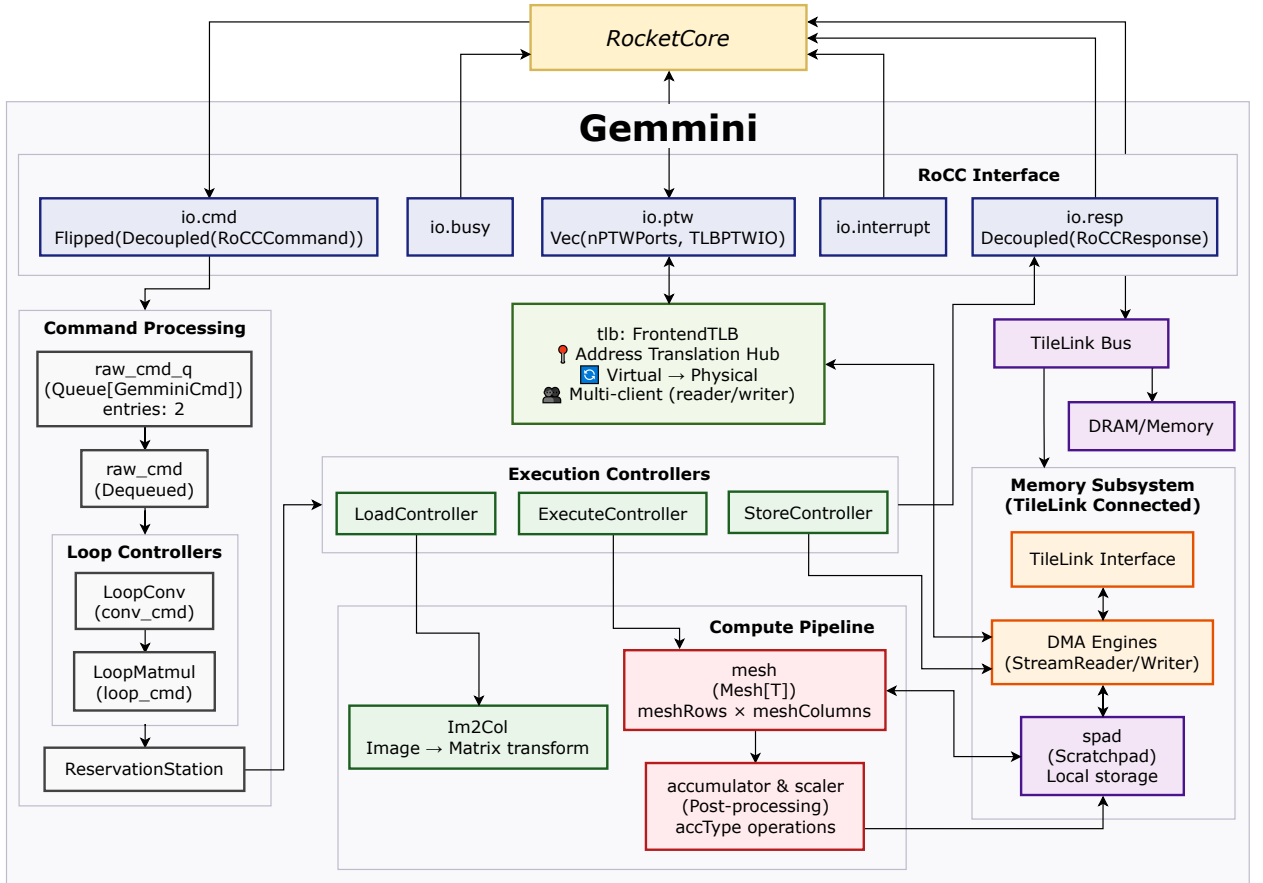


Figure 6.1: Overview of the internal modules in the generated Gemini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers.

6.1.1 Command Unrolling and Buffering for Decoupled Execution via LoopConv and LoopMatmul

This hardware-based command unrolling is a defining characteristic of a powerful, tightly-integrated co-processor. This design offloads what would otherwise be thousands of CPU instructions—for loop control, address calculation, data dependency checking, and synchronization—into a single accelerator command. By handling this complexity autonomously within its own hardware controllers, Gemmini frees the Rocket Core to perform other tasks, demonstrating a highly efficient division of labor between the general-purpose CPU and the specialized accelerator.

The stream of micro-operations unrolled by the high-level controllers is not fed into a rigid, lock-step pipeline. Instead, Gemmini employs a more sophisticated and powerful paradigm: decoupled execution managed by a set of independent, communicating Finite State Machines (FSMs). This design allows the various stages of a complex operation—loading data, executing computation, and storing results—to proceed concurrently and asynchronously, maximizing hardware utilization.

The initial RoCC instructions received from the CPU are often "complex" commands that encapsulate entire loops (e.g., 'loop_matmul'). As shown in Figure ??, modules like LoopMatmul and LoopConv are responsible for unrolling these high-level tasks into a stream of simpler, primitive micro-operations for loading, storing, and executing on tiled sub-matrices. These primitive commands are then buffered in a queue before being sent to a central dispatcher.

Due to convolution operations inherently require two levels of decomposition to be efficiently executed on the systolic array hardware, the interface between LoopConv and LoopMatmul reflects a fundamental architectural principle: hierarchical loop unrolling.

Once a `RoCCCommand` crosses the interface into Gemmini, it is not executed directly by the systolic array. Instead, the CPU often issues high-level, almost CISC-like commands such as `LOOP_WS` or `LOOP_CONV_WS`, which encapsulate entire, complex operations. These commands are received by dedicated hardware controllers — `LoopMatmul` and `LoopConv` — which act as micro-code engines. Their primary function is to interpret these high-level instructions and "unroll" them into a sequence of simpler, primitive micro-operations that can be dispatched to the underlying hardware units.

At the first level, `LoopConv` decomposes high-level convolution operations into a sequence of matrix multiplication primitives. This transformation handles the complex six-dimensional loop structure inherent in convolutions, including batch

dimensions, input/output channels, and spatial dimensions. At the second level, **LoopMatmul** further breaks down these matrix multiplication operations into smaller tiles that can be efficiently processed by the systolic array’s fixed dimensions.

This hierarchical approach ensures that the complex nested loop structures of convolution operations are systematically reduced to primitive operations that can be directly executed by the underlying hardware. The **LoopMatmul** module thus acts as an intermediary controller, translating matrix-level operations into the tile-based computations required by the systolic array architecture.

6.1.1.1 LoopConv

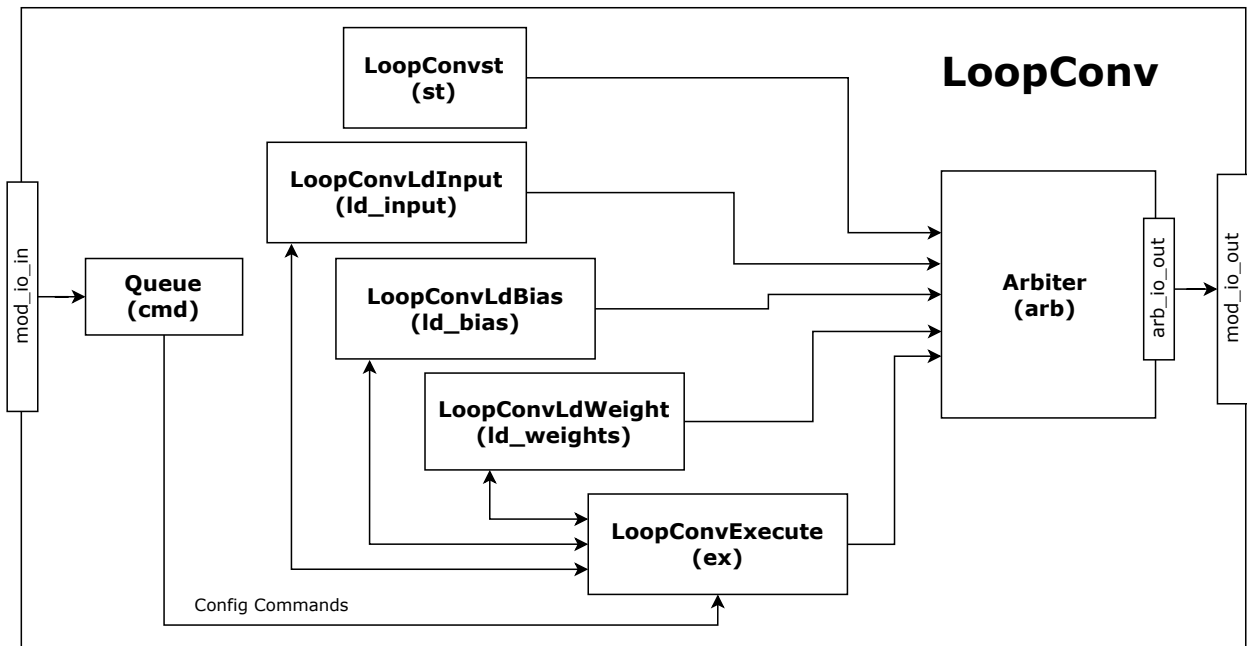


Figure 6.2: LoopConv module

The **LoopConv** block is a **Convolution Loop Unroller**, or in other words, it is the High-Level Command Unrolling module within the Gemini systolic array accelerator. Usually, convolution operations involve complex nested loops over: Batch dimensions (N), Input/Output channels (C), Spatial dimensions (H, W), and Kernel dimensions (R, S). Without **LoopConv**, programmers would need to manually implement these 6-7 nested loops, handle data tiling, and manage resource allocation. **LoopConv** provides a single high-level instruction (for example, `gemmini_loop_conv_ws` or `gemmini_loop_ws` in C code, ...) that can perform entire convolution operations automatically.

The internal architecture of the **LoopConv** controller, depicted in Figure 6.5b,

provides a clear illustration of this command unrolling process. As the figure shows, a single high-level convolution command is not executed serially. Instead, it triggers three independent, parallel load units to begin fetching the necessary tensors concurrently.

- **LoopConvLdInput**: Loads input feature maps from memory.
- **LoopConvLdWeight**: Loads convolution kernel weights from memory.
- **LoopConvLdBias**: Loads bias values from memory.

After the loaded data is fully received, it begins the execution phase.

- **LoopConvExecute**: Executes the convolution operation.
- **Arbiter**: A 5-to-1 arbiter that manages access to the memory interface, ensuring that the three load units and the execute unit do not contend for the same resources simultaneously.

6.1.1.2 LoopMatmul

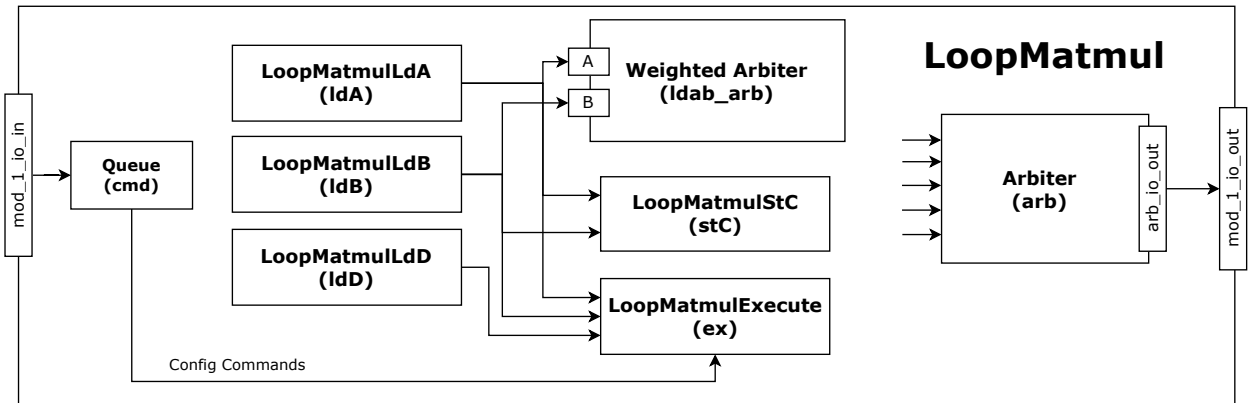


Figure 6.3: LoopMatmul module

The LoopMatmul module serves as the matrix multiplication command unroller within the Gemmini architecture. It receives commands from the LoopConv module and processes five distinct types of operations:

- **LOAD3_CMD (MVIN)**: Loads data from DRAM to scratchpad memory
- **STORE_CMD (MVOUT)**: Stores data from scratchpad memory to DRAM

- **PRELOAD_CMD**: Preloads weights into the systolic array
- **COMPUTE_CMD**: Executes matrix multiplication operations
- **CONFIG_CMD**: Configures the load/store/execute pipeline parameters

The internal structure of the `LoopMatmul` controller exemplifies this architecture. It is composed of several independent hardware modules:

- `LoopMatmulLdA`: Loads matrix A from DRAM to scratchpad memory
- `LoopMatmulLdB`: Loads matrix B (weights) from DRAM to scratchpad memory
- `LoopMatmulLdD`: Loads bias/accumulator data from DRAM to accumulator memory
- `LoopMatmulExecute`: Performs the actual matrix multiplication computation
- `LoopMatmulStC`: Stores results from accumulator back to DRAM
- `LoopMatmulStCSpad`: Stores results from accumulator to scratchpad memory

While these modules operate independently, they are coordinated through a lightweight handshake mechanism referred to in the design as iterator synchronization. For instance, the `LoopMatmulExecute` will not begin computation on a tile of data until it receives a signal that the `LoopMatmulLdA` and `LoopMatmulLdB` have successfully loaded the required operands into the scratchpad. This allows the Load modules to work ahead, fetching data for a future computation (`tile i+1`) while the `Execute` module is still busy processing the current one (`tile i`).

This decoupled architecture is a deliberate and critical design choice for overcoming the "memory wall". The high latency of fetching data from off-chip DRAM is a primary performance bottleneck in any accelerator. By overlapping the long-latency memory operations (Load and Store) with the core computation (Execute), the system effectively hides this latency. This ensures that the systolic array—the most valuable computational resource—is starved for data as infrequently as possible, maximizing its utilization and achieving a throughput that would be unattainable with a simple, rigid pipeline.

The Arbiter employs a fixed-priority scheduling policy to manage resource contention among the parallel execution units:

1. **Highest Priority - Store to DRAM:** Commands from the `LoopMatmulStC` module receive top priority to prevent accumulator buffer overflow and maintain pipeline flow.
2. **Execute Operations:** The `LoopMatmulExecute` module's preload and compute commands receive second priority to keep the systolic array actively computing.
3. **Load Bias/Accumulator:** The `LoopMatmulLdD` module's commands for loading bias data receive third priority.
4. **Load Input Matrices:** A weighted sub-arbiter manages requests from `LoopMatmulLdA` and `LoopMatmulLdB` modules with fourth priority.
5. **Lowest Priority - Store to Scratchpad:** The `LoopMatmulStCSpad` module receives lowest priority as scratchpad stores are less critical than DRAM operations.

This priority scheme ensures that result data is promptly written to DRAM to free accumulator resources, while maintaining sufficient data throughput to keep the computational units busy.

The impressive result, `LoopMatmul` transforms a single `LOOP_WS(matmul 1024x1024)` command into **thousands** of precisely calculated, optimally ordered, resource-aware hardware operations ready for immediate execution by the controllers.

6.1.2 Reservation Station and Execution Control

`LoopMatmul` provides `ReservationStation` with:

- **Optimally Sequenced Commands:** Thousands of individual operations from high-level matrix specifications
- **Complete Address Calculation:** DRAM offsets, scratchpad addresses, stride handling
- **Resource-Aware Flow Control:** Prevents overwhelming downstream controllers
- **Dependency Management:** Ensures proper ordering without explicit dependencies in `ReservationStation`

- **Concurrent Loop Support:** Enables double-buffering for maximum throughput
- **Attribution Metadata:** Marks commands for proper tracking and debugging

The ReservationStation receives commands from the LoopMatmul:

- **LOAD_CMD:** Load matrix A block (4x4) from DRAM to scratchpad
- **LOAD2_CMD:** Load matrix B block (4x4) from DRAM to scratchpad
- **LOAD3_CMD:** Load bias (4x4) from DRAM to accumulator
- **PRELOAD_CMD:** Preload B matrix into systolic array
- **COMPUTE_AND_FLIP_CMD:** Perform matrix multiplication $A*B$
- **STORE_CMD:** Store result (4x4) from accumulator to DRAM

The stream of primitive commands is received by the ReservationStation module, which reorders and dispatches them to the appropriate back-end controllers: LoadController, StoreController and and ExecuteController.

- **LoadController:** Handles all data loading operations, fetching data from DRAM to the scratchpad memory.
- **StoreController:** Manages data storage operations, writing results from the scratchpad back to DRAM.
- **ExecuteController:** Manages the actual computation. It issues commands to the systolic array and orchestrates the movement of data between the scratchpad and the computational units. It also contains logic to manage optional units, such as the hardware `im2col` unit or data transposer (Figure 6.5).
 - **Im2Col Module:** Specialized SRAM controller for convolution operations that converts image data into column format for efficient convolution, manages sliding window operations over input data, handles address generation for accessing input feature maps, and provides specialized data access patterns for CNN operations.

- **Queue (sram_read_signals_q)**: Pipeline synchronization buffer that buffers control signals for SRAM read operations, provides timing alignment between address generation and data availability, and handles pipeline delays in the Im2Col data path.
- **Queue (mesh_cntl_signals_q)**: Mesh control signal buffer that queues control signals fed into the computational mesh, synchronizes computation control with data arrival timing, and provides pipeline buffering for mesh operations.
- **MultiHeadedQueue**: Multi-port queue for command buffering that allows multiple commands to be read simultaneously, provides flexible command scheduling and issue capabilities, and supports out-of-order execution patterns.
- **TransposePreloadUnroller**: Command transformation unit that unrolls complex operations into simpler primitive operations, handles matrix transpose operations for weight-stationary dataflow, and manages preload command sequences for optimal data placement.
- **MeshWithDelays (Computational Mesh)**: The main systolic array for matrix computations that performs actual matrix multiplication operations, implements systolic array dataflow (input/weight/output stationary), handles multiple simultaneous matrix multiplications, and manages timing and data flow through the PE array.

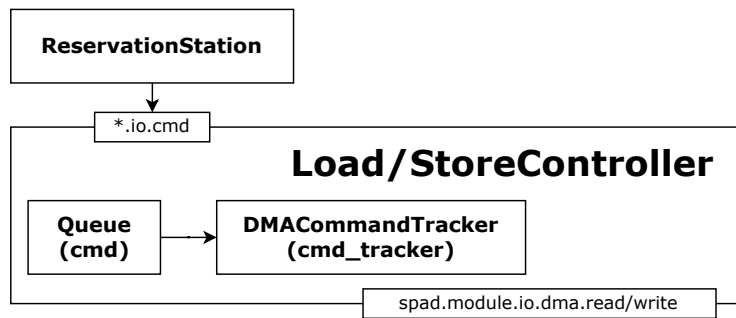


Figure 6.4: The Load and Store Controller module, which manages data movement between the scratchpad and DRAM. Both controllers load the commands from the ReservationStation into the DMACommandTracker, which is connected to the DMA read and write interfaces of the scratchpad memory.

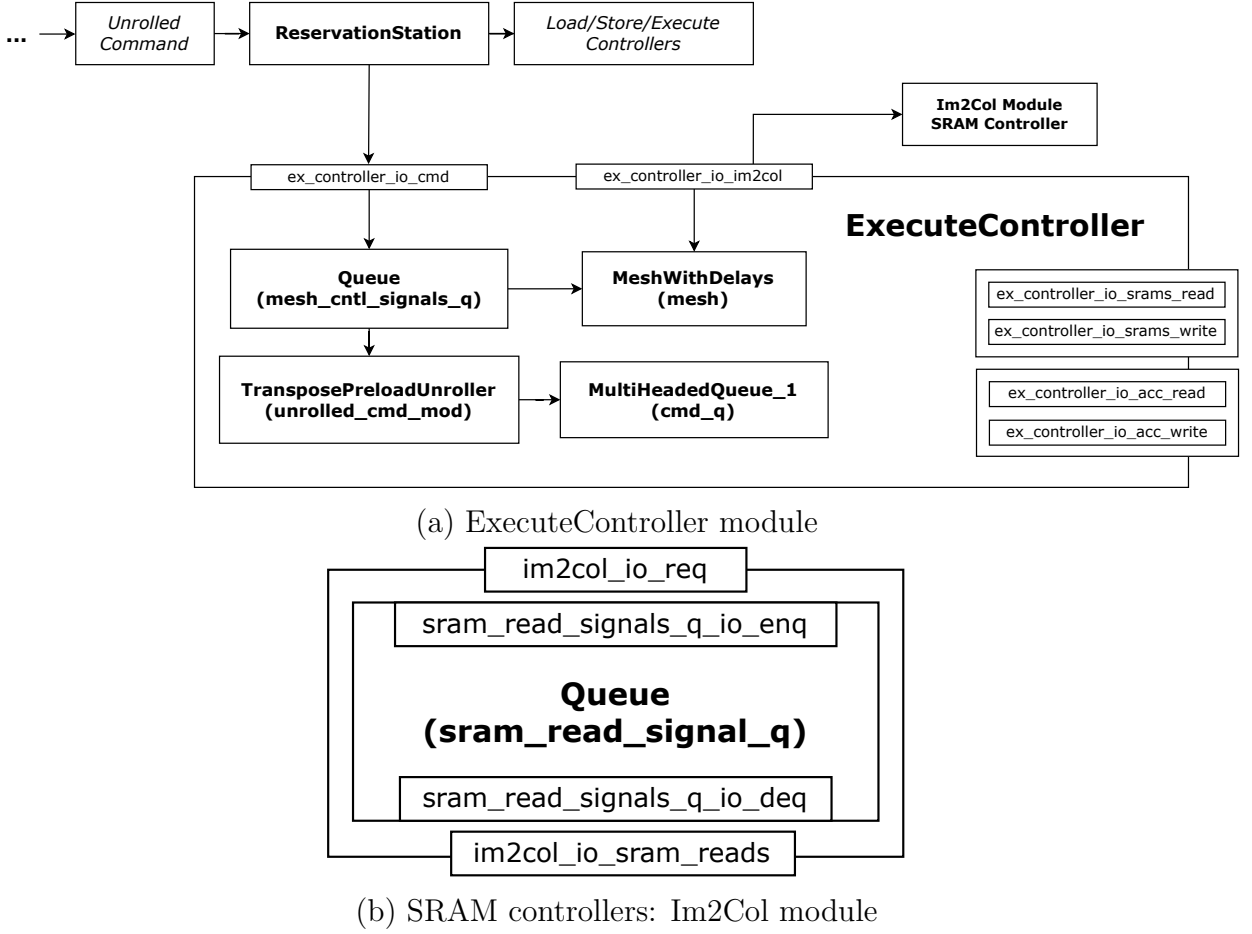


Figure 6.5: The ExecuteController module and its associated SRAM control interfaces.

6.1.3 Scratchpad: the Local Memory Subsystem

The Scratchpad (Figure 6.6) implements the on-chip memory, containing the SRAM banks and logic for interfacing with the DMA engine and the systolic array’s accumulator. It serves as the central data hub of the matrix multiplication accelerator, acting as a high-speed, local memory subsystem that bridges the gap between external DRAM and the compute units.

Far more than simple storage, the Scratchpad implements a sophisticated data movement engine with integrated processing capabilities, dual data paths, and complex arbitration logic, enabling the high-throughput, low-latency data access patterns essential for efficient matrix computation. Its design demonstrates the critical importance of specialized memory subsystems in achieving peak performance in domain-specific accelerators.

The Scratchpad sits at the heart of the Gemini data flow architecture. There’s

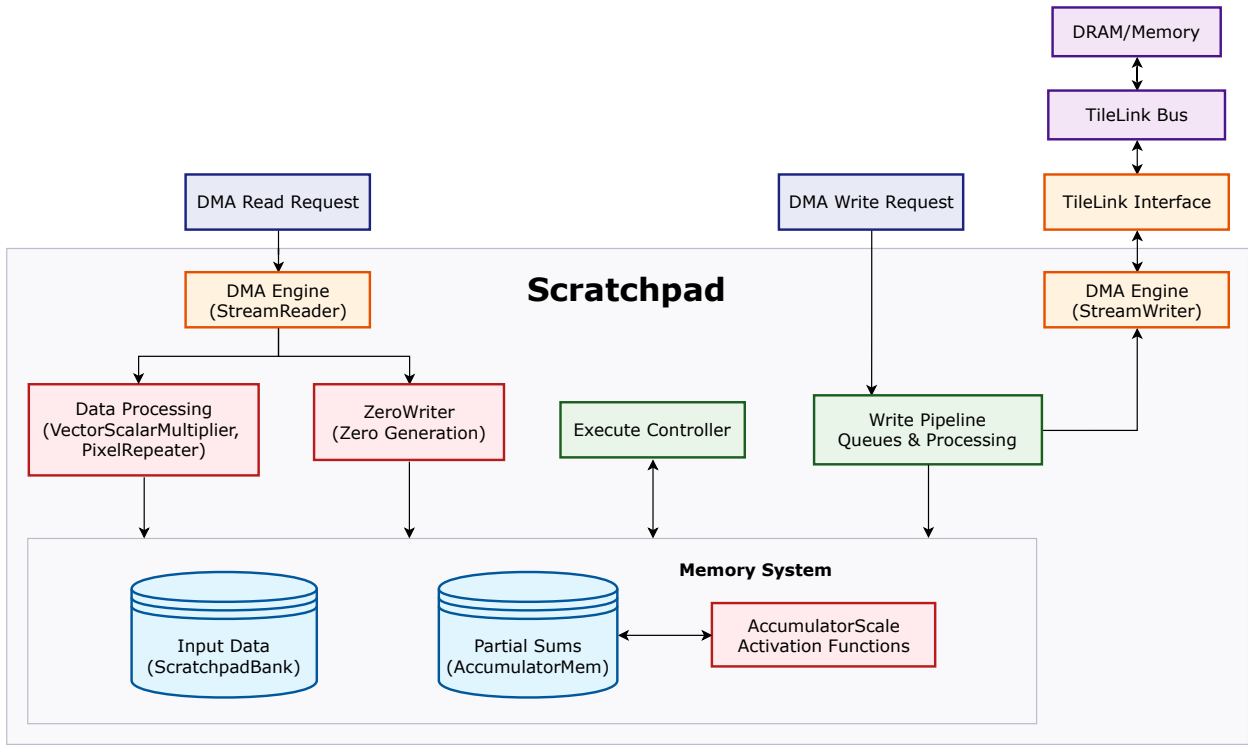


Figure 6.6: The internal structure of the Scratchpad module, showing the various interfaces to the TLB, DMA, and internal memory banks.

two primary interfaces that the Scratchpad exposes to work with the rest of the system:

- **External Memory Interface:** Via TileLink and DMA for bulk data transfers with DRAM
- **Compute Unit Interface:** Direct access for the systolic array and accumulator units

The Scratchpad's internal architecture reveals its complexity as a data processing subsystem rather than passive storage. The key components include:

- **Memory Banks:**
 - **ScratchpadBank** (`spad_mems_*`): Multiple SRAM banks storing input data (weights, activations) for the systolic array, enabling parallel access patterns
 - **AccumulatorMem** (`acc_mems_*`): Specialized banks storing partial sums and accumulation results, with integrated support for activation functions and scaling operations

- **DMA Engines:** `StreamReader` and `StreamWriter` for data transfers between external memory and internal banks
- **Integrated Data Processing Units:** Built-in data transformation units to reduce CPU overhead
 - `VectorScalarMultiplier` (`vsm/vsm_1`): Applies scaling operations to data during movement, with separate scalers for scratchpad and accumulator data paths
 - `PixelRepeater`: Handles pixel repetition optimizations for first-layer convolution operations
 - `AccumulatorScale` unit (`acc_scale_unit`): Critical component applying scaling and activation functions to accumulator data before memory writes
 - `Normalizer` unit (`acc_norm_unit`): Performs normalization operations on accumulator data
 - `ZeroWriter`: Generates zero data when the `all_zeros` flag is set, avoiding unnecessary memory reads
- **Pipeline Infrastructure:** Built with multiple queue stages to ensure smooth data flow and prevent pipeline stalls.

6.1.4 Mesh: the Systolic Array

The Mesh (Figure 6.7) is the direct hardware implementation of the systolic array, containing the grid of PEs.

6.2 The RoCC Interface that controls Gemmini

The Gemmini accelerator is controlled through a custom RoCC interface, which allows the Rocket Core to issue commands and receive results. This interface is designed to be flexible and efficient, enabling high-throughput data transfers and command execution.

6.2.1 Resource Management and Arbitration

The decoupled, parallel architecture described previously, while powerful, creates an inherent engineering challenge: contention for shared hardware resources. With

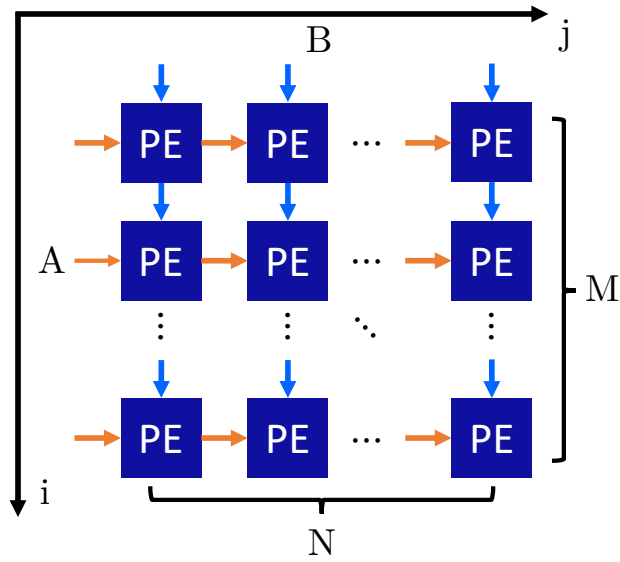
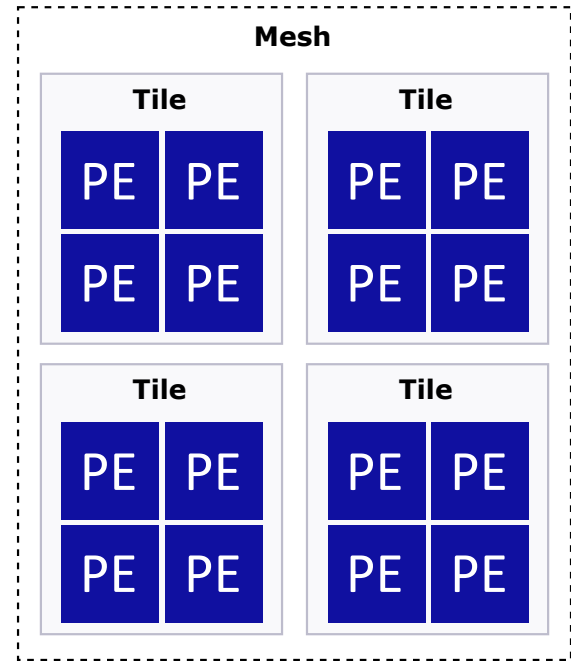


Figure 6.7: The Mesh module.



(a) Mesh 4x4, 4 meshRows and 4 meshCols, made up 16 Tiles. Each Tile has 1 PE.



(b) Mesh 2x2, 2 meshRows and 2 meshCols, made up 4 Tiles. Each Tile has 4 PE.

Figure 6.8: Different Mesh configurations, while maintaining the same total number of 16 PEs.

multiple independent Load, Execute, and Store modules operating concurrently, they

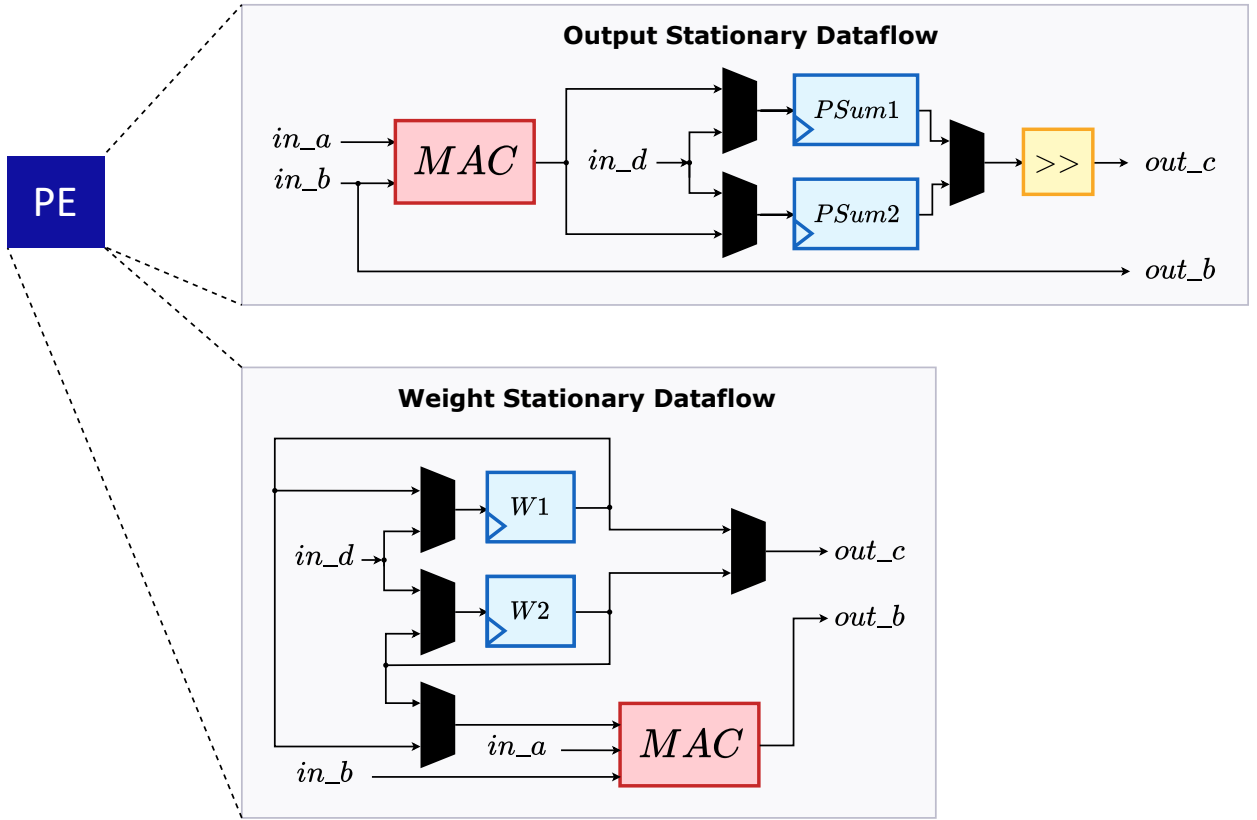


Figure 6.9: PE supports different dataflows (Output Stationary and Weight Stationary).

will inevitably attempt to access the same resources—such as the scratchpad memory ports or the primary DMA engine—at the same time. Gemmini addresses this challenge through a hierarchy of dedicated hardware arbiters.

These arbiters act as traffic controllers for internal data and command pathways. For instance, the **LoopConv** controller, as shown previously in Figure ??, features a central 5-to-1 arbiter that serializes requests from the five parallel units before they are dispatched. The **LoopMatmul** controller employs an even more complex, two-stage arbitration scheme which includes a **Global 5-Way Arbiter**. This hardware explicitly enforces a fixed-priority policy, granting higher precedence to **Store** operations to free up internal result buffers, followed by **Execute** and **Load** operations. This ensures that the system does not deadlock and that resource access is allocated in a deterministic and efficient manner.

Beyond simply resolving simultaneous requests, the system must also manage its overall load to prevent its internal queues and buffers from overflowing. This is achieved through a feedback mechanism managed by resource utilization counters. These hardware counters track the number of load, store, and execute micro-

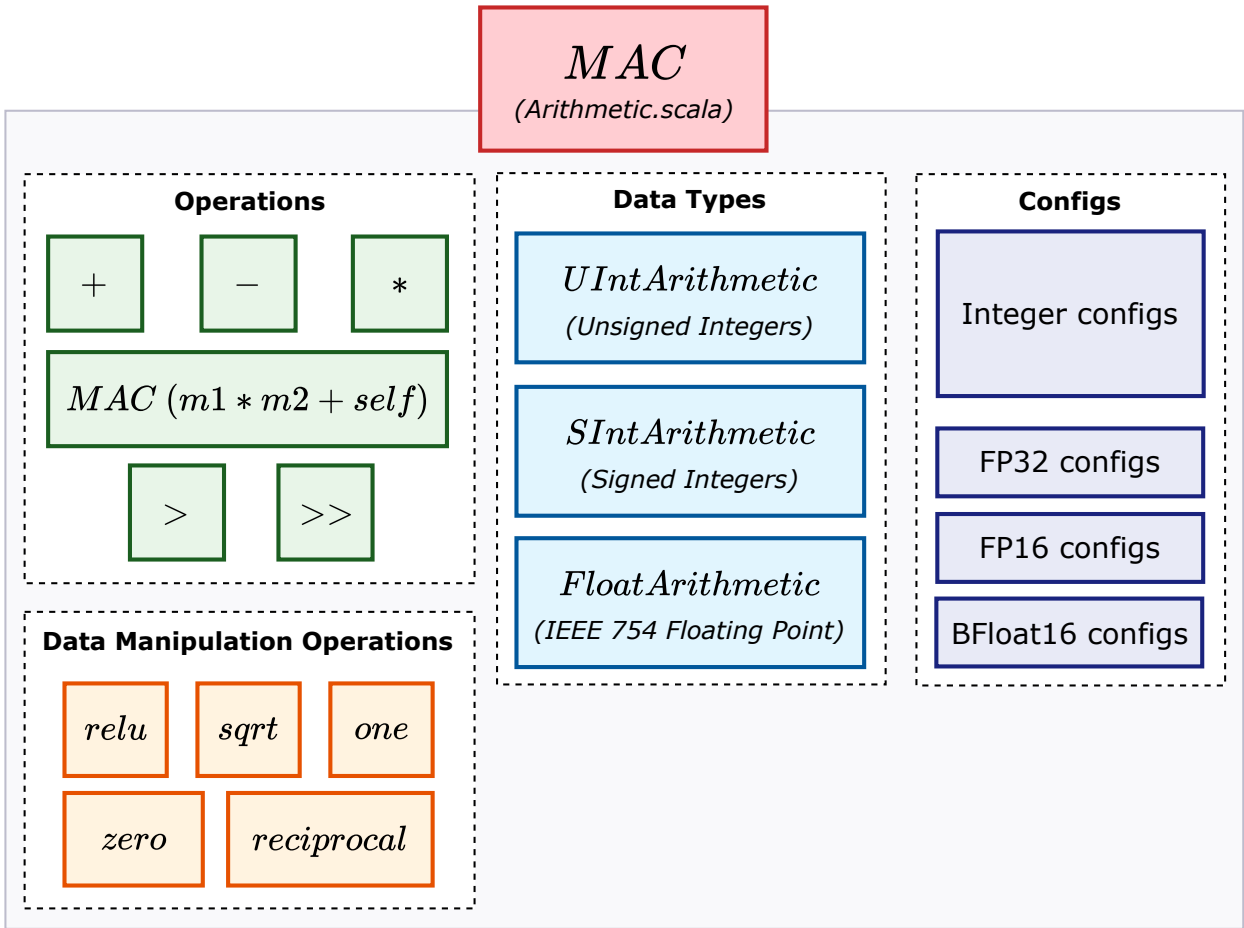


Figure 6.10: MAC module, or Multiply-Accumulate unit, performs the core computation in the systolic array.

operations currently in flight. If the number of pending commands in the Re-Order Buffer (ROB) exceeds a hardware-defined threshold, the controller asserts a backpressure signal (e.g., `rob_overloaded`) to the upstream FSMs, temporarily stalling them from issuing new commands until resources become available.

The inclusion of explicit hardware for arbitration and backpressure-based flow control is a critical indicator of the design’s sophistication. While a simplistic proof-of-concept accelerator might function correctly under ideal, low-load conditions, it would likely fail or deadlock under the high-throughput, concurrent scenarios that this system is designed for. Therefore, the presence of this robust resource management infrastructure proves the maturity of the Gemmini accelerator, demonstrating an architecture designed not just for theoretical correctness but for practical, high-performance operation.

6.3 The Multi-Level Programming Model

To cater to different user needs, from application developers to hardware architects, Gemmini supports a multi-level programming model. This stack provides layers of abstraction, allowing users to work at a level appropriate for their task, from direct hardware control to high-level framework integration.

6.3.1 Low-Level: Gemmini ISA

At the lowest level, Gemmini is controlled by its custom RoCC instruction set. These instructions are typically wrapped in C preprocessor macros for easier use from C/C++ code. The instruction set includes primitive commands like `mvin` (move data in), `mvout` (move data out), `preload` (load data into the PE array), and `compute`. It also features more "complex" instructions like `loop_matmul` that abstract away the manual tiling and scheduling of primitive commands, offloading the loop control to hardware state machines [9].

6.3.2 Mid-Level: Hand-Tuned C Library

To abstract the complexities of direct ISA programming, Gemmini provides a C library (typically accessed via `gemmini.h`). This library offers functions for common DNN operations, such as `tilled_matmul` and `tilled_conv`. These functions implement optimized heuristics for loop tiling and data movement to maximize scratchpad utilization based on the specific hardware parameters of the generated Gemmini instance.

6.3.3 High-Level: Compiler Support (Conceptual)

The Gemmini ecosystem is designed to support higher-level compilation flows. This involves tools that can take DNN models described in standard frameworks like ONNX (Open Neural Network Exchange) and compile them down to Gemmini instructions, either by calling the mid-level C library or by directly generating low-level ISA calls. This further simplifies the deployment of DNNs on Gemmini-accelerated SoCs.

Chapter 7: System Implementation and Testing

This chapter details the implementation of the RISC-V System-on-Chip (SoC) featuring a Gemmini accelerator. The preceding chapters have established the theoretical foundations of the RISC-V ISA, the Chipyard framework, and the architectural principles of DNN acceleration. We now transition from theory to practice, documenting the complete process from hardware configuration and generation to final system validation on an FPGA platform. This chapter will cover the specific SoC configuration, the integration of the Gemmini accelerator as a Rocket Custom Coprocessor (RoCC), the software stack required to boot a full Linux operating system, the automated build process, and finally, the verification of the complete system on a Xilinx VC707 FPGA board. This practical account serves as a tangible demonstration of the maturity of the open-source, full-stack design methodology adopted for this research.

7.1 Hardware Implementation: SoC Configuration

The foundation of our hardware design is the Chipyard SoC generation framework, which provides a flexible and powerful environment for constructing complex RISC-V systems. The result of this process is a complete, Linux-capable SoC, whose top-level architecture is illustrated in Figure 7.1. This diagram encapsulates the full-stack integration central to this thesis, depicting the system from the processor core and accelerator down to the specific peripheral IP cores and their connections to the physical hardware on the FPGA board.

At the heart of the system is the **RocketTile** (Figure 7.1), which contains the 64-bit RISC-V **RocketCore** and its private L1 instruction and data caches. Tightly coupled to the processor pipeline, as discussed in Chapter 5, is the **Gemmini RoCC Accelerator**. This integration via the RoCC interface allows for low-latency command dispatch from the CPU to the accelerator, a critical feature for high-performance operation.

The **RocketTile** communicates with the rest of the system through the coherent **SystemBus**, which is implemented using the **TileLink** protocol. This bus serves as the main interconnect for all major subsystems. The memory hierarchy extends from the L1 caches to a unified L2 cache and finally to the off-chip DDR3 memory. To bridge the on-chip **TileLink** protocol with the industry-standard **AXI** protocol used by the external memory controller IP, a **TileLink to AXI4 bridge** is automatically

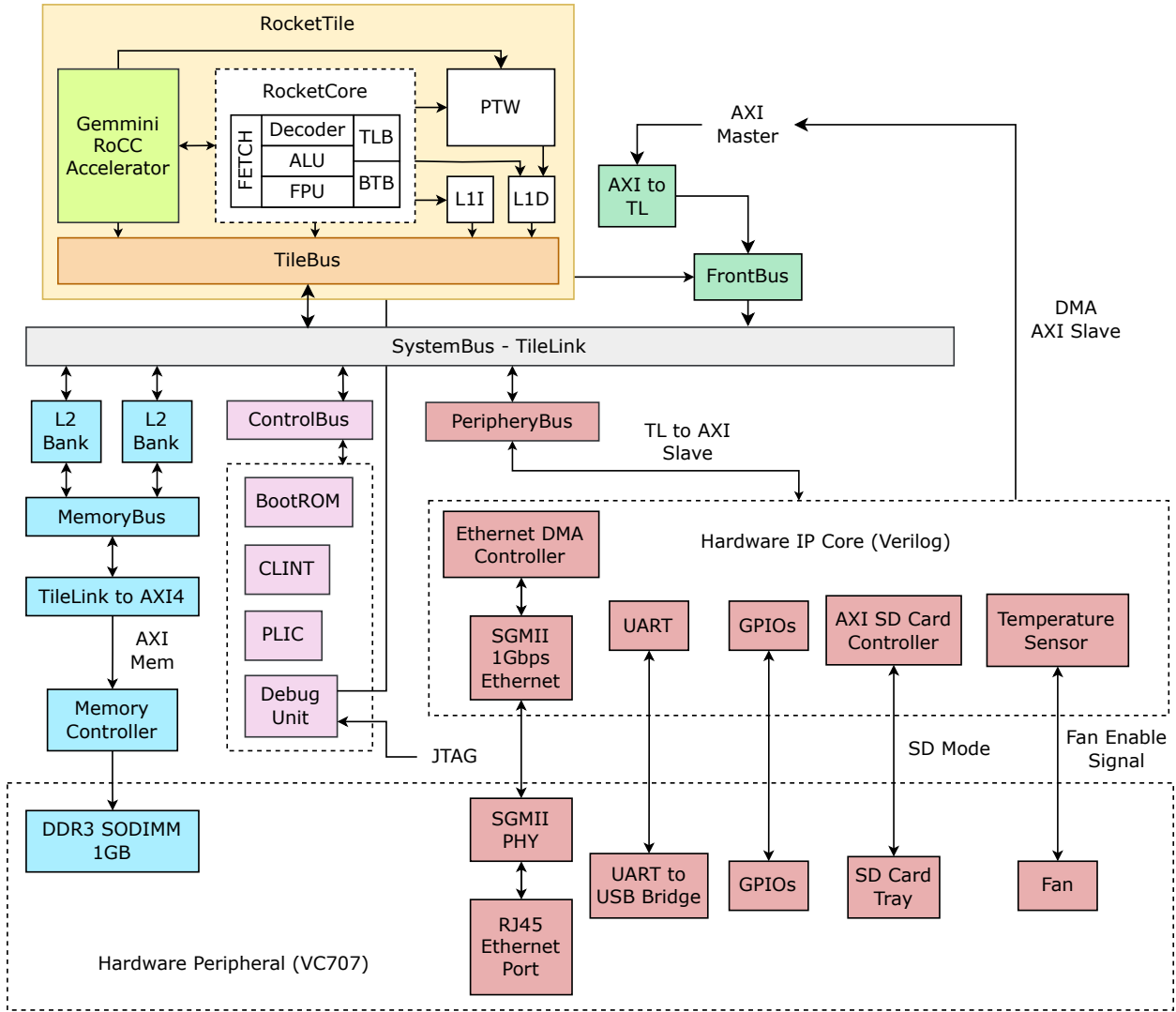


Figure 7.1: The implemented Rocket SoC with Gemini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemini systolic array, and various peripherals are integrated via AXI interfaces.

instantiated by the framework. This demonstrates a practical aspect of modern SoC design where components with disparate interface standards are seamlessly integrated.

To create a fully functional, Linux-capable computing platform, a rich set of peripherals is integrated via the `PeripheryBus`. Similar to the memory interface, a `TL to AXI Slave` bridge connects the `TileLink` bus to a set of AXI-based peripheral controllers. These include an `AXI SD Card Controller` to enable booting from an SD card, a `UART` controller for console I/O, and a complete Ethernet subsystem for network connectivity. These controller IPs, written in `Verilog`, are then wired to their corresponding physical counterparts on the VC707 board, such as the SD card tray and the RJ45 Ethernet port.

7.1.1 The Rocket64b1gem16jtag Configuration

The comprehensive architecture shown in (Figure 7.1) is not designed by hand in Verilog, but is programmatically generated from a high-level Scala configuration. As outlined in Chapter 3, this generator-based approach allows for rapid architectural exploration and ensures design consistency.

The specific configuration for our SoC is named `Rocket64b1gem16jtag`. This name concisely describes its key features: a 64-bit Rocket Core (`Rocket64`), one "big" core (`b1`), and a 16x16 Gemmini systolic array (`gem16`). This configuration is defined in Scala using Chipyard's `Config` classes, which employ a `mixin`-based approach to compose system components. This powerful paradigm allows features to be incrementally added to a base configuration through a series of traits, each beginning with the `With...` prefix. The core of the configuration is shown in Listing 7.1.

```
1 class Rocket64b1gem16jtag extends Config(  
2   new WithGemmini(16, 64) ++  
3   new WithInclusiveCache ++  
4   new WithNBreakpoints(8) ++  
5   new WithNBigCores(1) ++  
6   new WithBootROMFile("workspace/bootrom.img") ++  
7   new WithExtMemSize(0x380000000L) ++  
8   new WithNExtTopInterrupts(8) ++  
9   new WithDTS("freechips,rocketchip-vivado", Nil) ++  
10  new WithDebugSBA ++  
11  new WithJtagDTM ++  
12  new WithEdgeDataBits(64) ++  
13  new WithCoherentBusTopology ++  
14  new WithoutTLMonitors ++  
15  new BaseConfig)
```

Listing 7.1: The `Rocket64b1gem16jtag` configuration class.

The `WithGemmini(16, 64)` trait is the most critical for this work. It instructs the generator to instantiate the Gemmini accelerator with a specific set of hardware parameters derived from the `defaultConfig` within the Gemmini generator source code. The generated accelerator is a sophisticated system in its own right, comprising several key hardware blocks:

- `WithNBigCores(1)`: Instantiates a single, high-performance 64-bit Rocket Core. The Rocket Core was selected for its maturity and stability within the open-

source ecosystem, providing a reliable foundation for our system.

- **WithGemmini(16, 64)**: This is the crucial trait that integrates the Gemmini DNN accelerator, specifying a 16x16 systolic array mesh and a 64-bit wide DMA bus. This trait instructs the generator to instantiate a complete accelerator system based on its internal `defaultConfig`. The resulting hardware is a sophisticated system-within-a-system, comprising:
 - A Systolic Array Core: A 16x16 grid of Processing Elements (PEs), each capable of an 8-bit integer multiply and a 32-bit integer accumulate per cycle. The array supports both Weight Stationary (WS) and Output Stationary (OS) dataflows.
 - An On-Chip Memory System: A 256KB, 4-banked scratchpad memory for staging data and a separate 64KB, dual-ported accumulator memory for partial sums.
 - Control and Execution Logic: A Reservation Station to enable out-of-order execution of primitive commands, managed by decoupled Load, Store, and Execute controllers.
 - Specialized Hardware Units: A hardware Im2Col unit to accelerate convolution preprocessing and a 4-entry Translation Lookaside Buffer (TLB) to support virtual memory.
- **WithInclusiveCache**: Adds a last-level inclusive L2 cache to the memory hierarchy. This component is vital for performance, serving as a large, shared buffer that reduces the frequency of costly off-chip memory accesses, thereby helping to mitigate the "memory wall" problem.
- **WithExtMemSize(0x380000000L)**: Configures the size of the external DRAM to 14 GiB. A large main memory is essential for running a full operating system like Debian Linux, which requires significant space for the kernel, user-space applications, and runtime data.
- **WithBootROMFile(...)**: Specifies the path to the boot ROM image. This ROM contains the first-stage bootloader code that the processor will execute on reset, initiating the system boot sequence.

This Scala configuration is then consumed by the Chipyard generator-based framework to produce the concrete hardware implementation. The resulting top-level ar-

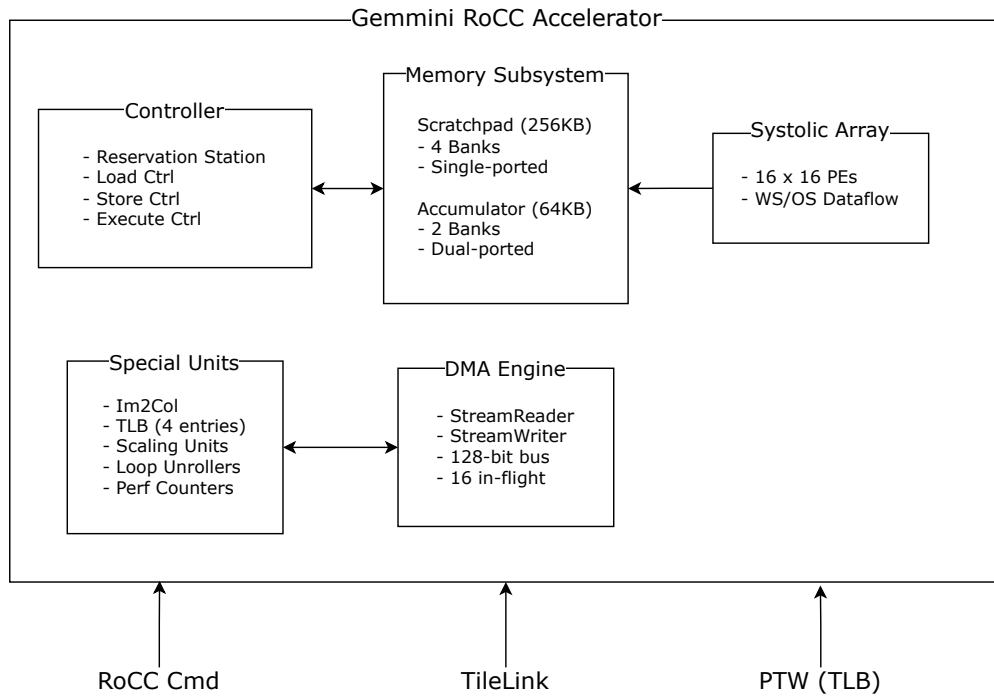


Figure 7.2: Gemmini Configuration on Rocket64b1gem16jtag SoC.

chitecture of the SoC, as deployed on the Xilinx VC707 FPGA, is illustrated in Figure 7.1.

7.2 Gemmini Integration via RoCC

While the RoCC interface provides a clean, standardized gateway for the CPU, it abstracts away a sophisticated internal command and control hierarchy within Gemmini itself. This internal micro-architecture functions as a hardware-based micro-code engine; it is responsible for autonomously unrolling high-level, CISC-like commands received from the CPU into a stream of primitive micro-operations, thereby orchestrating the intricate dataflows and resource management essential for high-performance systolic computation. This section delves into this multi-layered system, detailing the journey of a command from the initial RoCC dispatch to its fully decoupled execution within the accelerator’s asynchronous hardware modules.

7.2.1 The RoCC Interface

The primary conduit for communication between the Rocket Core and the Gemmini accelerator is the RoCC interface. This is a standardized feature of the Rocket Chip generator that allows for the tight coupling of co-processors. As part of this stan-

standard, the RISC-V ISA reserves four instruction opcode spaces for custom extensions. The Gemmini accelerator is assigned the `custom3` opcode space, which corresponds to the binary value `0b1111011`. When the Rocket Core’s instruction decoder encounters this specific opcode, it forgoes standard execution and instead initiates a dispatch to the attached RoCC unit.

```

1 class WithGemmini(mesh_size: Int, bus_bits: Int) extends Config((site,
  here, up) => {
2   case BuildRoCC => up(BuildRoCC) ++ Seq(
3     (p: Parameters) => {
4       implicit val q = p
5       implicit val v = implicitly[ValName]
6       LazyModule(new gemmini.Gemmini(gemmini.GemminiConfigs.
  defaultConfig.copy(
7         meshRows = mesh_size, meshColumns = mesh_size, dma_buswidth =
  bus_bits)))
8     }
9   )
10  case SystemBusKey => up(SystemBusKey).copy(beatBytes = bus_bits/8)
11 })

```

Listing 7.2: The `WithGemmini` configuration trait, which instantiates the accelerator as a RoCC device.

The consequence of this configuration is that the Gemmini accelerator is assigned a reserved block of custom instruction opcodes. As defined in its internal configuration, Gemmini uses `OpcodeSet.custom3`. During program execution, when the Rocket Core’s instruction decoder encounters an instruction with an opcode from this set, it recognizes it as a RoCC instruction. Rather than attempting to execute it, the core dispatches the entire instruction—including operands from the processor’s register file (`rs1`, `rs2`) and the funct field—directly to the Gemmini unit over the dedicated RoCC command channel. The RocketTile hardware then automatically wires the command and response channels between the core and the Gemmini module.

This direct connection demonstrates a successful accelerator integration. It is what distinguishes the RoCC methodology from a standard memory-mapped peripheral, providing the low-latency communication that is a cornerstone of this system’s design for high-performance DNN acceleration. The complete RoCC interface architecture, showing the connection pathways between the Rocket Core and the accelerator, is illustrated in Figure 7.3.

Upon recognizing a `custom3` instruction, the Rocket Core pipeline packages the

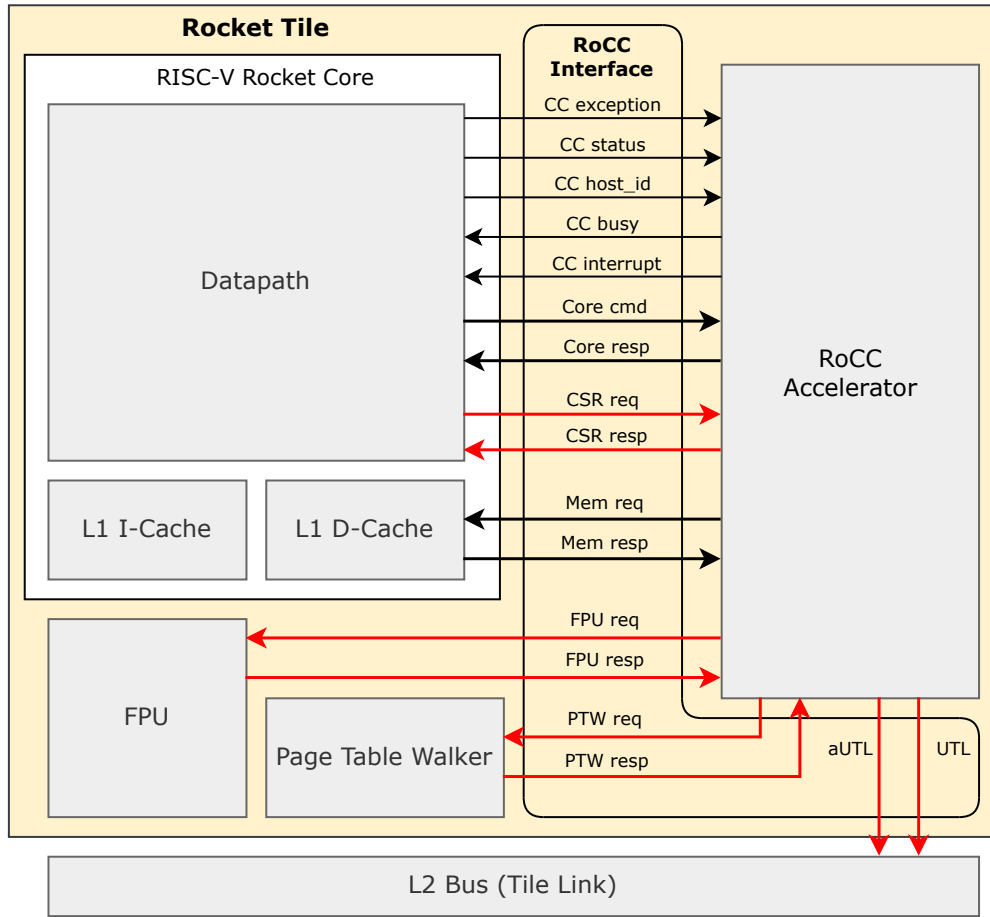


Figure 7.3: Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core’s datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels.

necessary information into a `RoCCCommand` bundle. This Chisel data structure, shown in Listing 7.3, acts as the payload for the interface. It contains the raw instruction bits in the `inst` field (including the `funct` sub-field that specifies the particular Gemmini operation), the operand values from the CPU’s integer register file in `rs1` and `rs2`, and the processor’s current status flags.

```

1 class RoCCCommand(implicit p: Parameters) extends CoreBundle()(p) {
2   val inst = new RoCCInstruction // Instruction encoding
3   val rs1 = Bits(xLen.W) // Source register 1 data
4   val rs2 = Bits(xLen.W) // Source register 2 data
5   val status = new MStatus // Processor status
6 }
7
8 class RoCCInstruction extends Bundle {
9   val funct = Bits(7.W) // Function code (specifies Gemmini

```

```

    operation)
10  val rs2 = Bits(5.W)           // Source register 2 address
11  val rs1 = Bits(5.W)           // Source register 1 address
12  val xd = Bool()               // Destination register valid
13  val xs1 = Bool()              // Source register 1 valid
14  val xs2 = Bool()              // Source register 2 valid
15  val rd = Bits(5.W)            // Destination register address
16  val opcode = Bits(7.W)        // Opcode (e.g., custom3 = 0b1111011)
17 }

```

Listing 7.3: The `RoCCCommand` Chisel Bundle, which defines the interface between the Rocket Core and the Gemmini accelerator.

The dispatch of this `RoCCCommand` bundle marks the end of the CPU’s direct involvement. However, the dispatched instruction is often not a simple, primitive operation. It can be a high-level, CISC-like command such as `LOOP_CONV_WS`, which instructs the accelerator to perform an entire tiled convolution. This single command triggers a cascade of parallel, asynchronous hardware processes within Gemmini’s internal controllers, such as the `LoopConv` module. These processes include the parallel loading of inputs, weights, and biases; decoupled computation in the systolic array; and the eventual storing of results, all managed autonomously by the accelerator. The CPU remains unaware of this internal complexity, having successfully offloaded the entire high-level task. The subsequent sections will now delve into this internal command processing hierarchy.

7.3 Peripheral Integration

The peripherals of the SoC are integrated firstly by having an controller, which is created by IP blocks (using Verilog). These controller are the middle-man for translating the AXI bus transactions from the Rocket Core to the specific peripheral protocols. The peripherals include ...

To operate these peripherals, the Operating Systems (barebone or Linux) will has a driver for each peripheral. The driver will interact with the controller, which in turn communicates with the peripheral hardware. The drivers are typically implemented in C and are part of the Linux kernel or the bare-metal runtime environment.

7.3.1 SD Card: AXI SD Card Controller and SD Mode

The SoC includes an AXI SD Card controller, which allows the system to boot from an SD card. This controller is integrated into the system using the Chipyard framework's diplomacy mechanism, which automatically wires up the necessary AXI interfaces.

The SD card is accessed in SD mode, which is a common mode for embedded systems. The controller supports standard SD commands and allows the system to read and write data blocks, making it suitable for booting the operating system and storing user data.

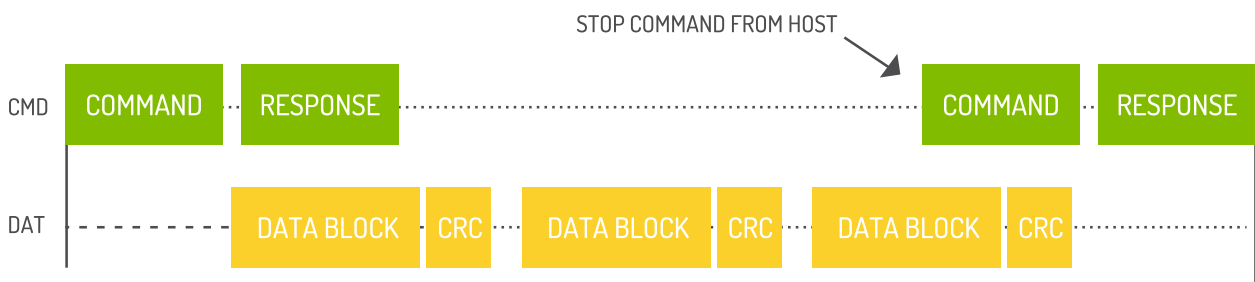


Figure 7.4: SD Mode: Multi-Block Read Operation. [3]

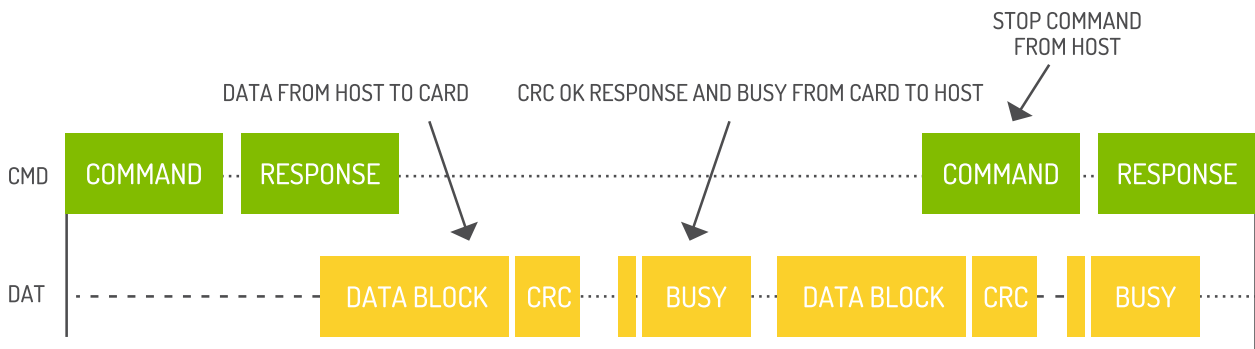


Figure 7.5: SD Mode: Multi-Block Write Operation. [3]

For comparison, here is how the same operation looks in SPI mode, which is another common mode for SD cards. The SPI mode is often used for compatibility with older systems or when a simpler interface is required. The SPI Mode offers lower speed compared to SD mode, but it is simpler to implement and can be used in systems that do not require high-speed data transfer.

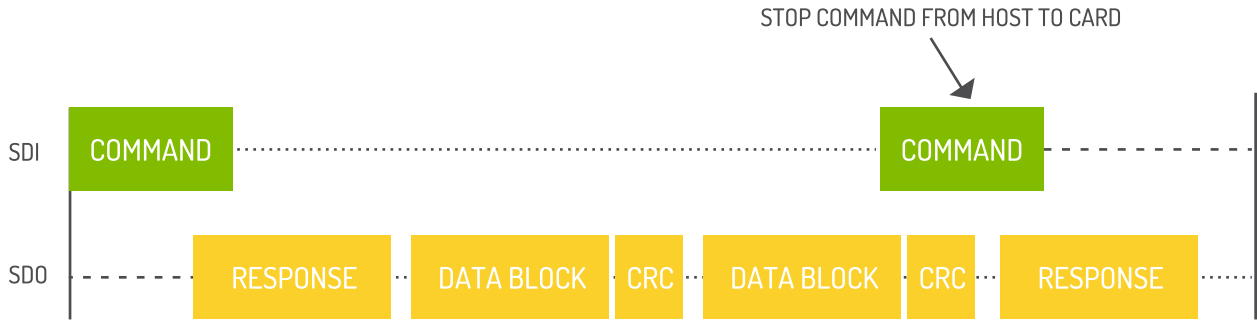


Figure 7.6: SPI Mode: Multi-Block Read Operation. [3]

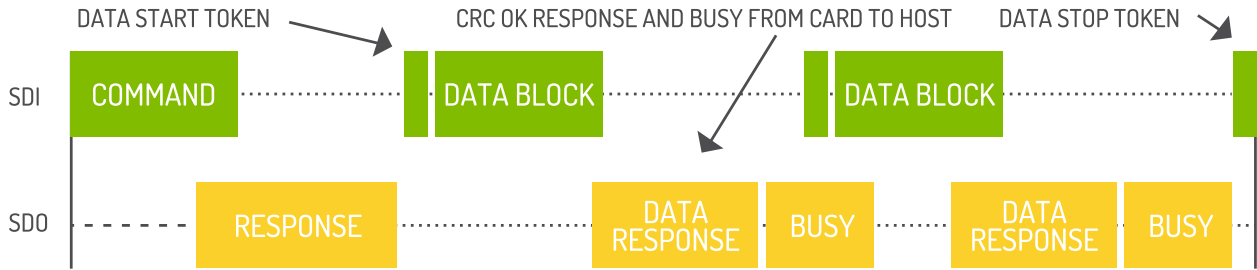


Figure 7.7: SPI Mode: Multi-Block Write Operation. [3]

7.3.2 UART: AXI UART Controller

7.3.3 Ethernet: AXI Ethernet Controller

The Ethernet subsystem provides high-speed network connectivity for the RISC-V RocketChip SoC, enabling communication with external systems and networks. The implementation consists of a four-layer architecture that bridges the processor's AXI4 bus interface to the physical Ethernet connection on the VC707 FPGA board.

The Ethernet subsystem is implemented as a hierarchical design with four distinct functional blocks, as illustrated in Figure 7.1.

1. **Ethernet DMA Controller:** serves as the primary interface between the RocketChip SoC and the Ethernet MAC layer. Provide functionality for managing data transfers between the SoC and the Ethernet MAC, including packet buffering, flow control, and interrupt handling.
2. **SGMII 1Gbps Ethernet MAC:** The MAC (Media Access Control) layer is implemented using the open-source `eth_mac_1g_fifo` module from the verilog-ethernet project [8]. This component provides: IEEE 802.3 Compliance, integrated frame-level FIFO Buffering, Frame Processing (Automatic frame padding, FCS generation and checking, and frame validation), AXI4-Stream Interface, Standard GMII Interface (Gigabit Media Independent Interface).

3. **SGMII PHY Interface:** implemented using Xilinx’s integrated SGMII IP core within the VC707 FPGA. This layer provides: SGMII Protocol (Serial Gigabit Media Independent Interface), Auto-negotiation, Clock Generation (125 MHz clock), Signal Conditioning.
4. **Physical RJ45 Interface:** The physical layer terminates at the RJ45 connector on the VC707 board.

7.4 Development Environment

The project requires a specific development environment to ensure that all tools, compilers, and dependencies are correctly configured. The primary environment is an Ubuntu 20.04 LTS workstation.

The complete project is included with all submodules for the RISC-V ecosystem components (Chipyard, U-Boot, Linux, OpenSBI) which are required for building the SoC and running the operating system.

7.5 Software Stack and Build Process

To run a complete operating system and user applications, a comprehensive software stack is required. This project utilizes a standard boot flow for embedded Linux systems and automates the entire build process with a **Makefile**.

7.5.1 Boot Flow: OpenSBI, U-Boot, and Linux

The system boots in several stages:

1. **Boot ROM:** On power-up, the Rocket Core begins execution from an on-chip Boot ROM. This initial code is responsible for basic hardware initialization and loading the next boot stage from an external device, in this case, an SD card.
2. **OpenSBI:** The first piece of software loaded from the SD card is the RISC-V Open Source Supervisor Binary Interface (OpenSBI). OpenSBI provides a crucial abstraction layer, handling low-level hardware control (like inter-processor interrupts and timers) so that the operating system kernel doesn’t need to be machine-specific.

3. **U-Boot:** OpenSBI then loads and jumps to the Das U-Boot bootloader. U-Boot is a more powerful bootloader responsible for further hardware initialization (e.g., DDR memory) and loading the Linux kernel into RAM.
4. **Linux Kernel:** Finally, U-Boot passes control to the Linux kernel. The kernel boots, mounts the root filesystem from the SD card, and starts the user-space services, providing a full Debian environment.

7.5.2 SoC Hardware Generation

The first stage of the build process is to generate the Verilog HDL for the SoC.

1. **Initial Verilog and Device Tree Generation:** The build system invokes the Scala Build Tool (SBT) to compile the Chisel sources using the `Rocket64b1gem16jtag` configuration. This generates the initial Verilog HDL and a base Device Tree Source (`.dts`) file, which describes the core SoC components as inferred by Chipyard’s diplomacy framework, as described in Chapter 6.
2. **Peripheral Integration:** This generated core is then integrated with several pre-existing, open-source peripheral controllers written in Verilog. These include the AXI UART controller, the AXI SD Card controller, and the AXI Ethernet controller.
3. **Device Tree Customization:** A crucial step is the creation of the final device tree. The base `system.dts` is combined with a board-specific overlay, `board/vc707/bootrom.dts`, which defines the VC707’s unique peripherals like the UART, MMC, and Ethernet controllers at their specific memory-mapped addresses.
4. **Boot ROM Generation:** The customized device tree is then passed to the boot ROM’s own build system, which compiles the C code for the first-stage bootloader and embeds the compiled device tree binary within the final ‘`bootrom.img`’ file.
5. **Final HDL Generation:** With the finalized ‘`bootrom.img`’ created, SBT is invoked a second time. This run uses the new boot ROM image to produce the final, correct Verilog HDL for the SoC, ensuring the processor’s memory map includes the fully configured boot ROM.

7.6 Gemmini Testing

We are going to use the gemmini-rocc-tests (<https://github.com/ucb-bar/gemmini-rocc-tests>) to test the Gemmini accelerator. The tests are designed to verify the functionality of the Gemmini RoCC interface and the correctness of the accelerator's operations. The tests cover a wide range of scenarios, including basic arithmetic operations, matrix multiplication, convolution, and more complex DNN workloads.

Chapter 8: Implementation Results and Discussion

This chapter presents the culminating results of the thesis, transitioning from theoretical design to tangible implementation and empirical validation. The System-on-Chip (SoC), designed programmatically in Chisel, was compiled through the FIRRTL intermediate representation and subsequently implemented using the Xilinx Vivado toolchain for the VC707 FPGA target. This chapter is organized into three main sections. First, it details the hardware implementation results, including the final system architecture and an analysis of on-chip resource utilization and power consumption. Second, it documents the successful deployment of the full software stack, demonstrating a boot-to-Linux environment. Finally, and most critically, it evaluates the performance of the integrated Gemmini DNN accelerator, providing quantitative evidence of its effectiveness.

8.1 Hardware Implementation Analysis

The hardware generation process produces a set of Verilog files that are synthesized, placed, and routed by the Vivado Design Suite. The following sections analyze the final hardware implementation from architectural, resource, and power perspectives.

8.1.1 System Block Design and Address Map

The generated system is a complex integration of the RISC-V core, the Gemmini accelerator, memory subsystems, and peripherals. The block diagrams in Figure 8.1, Figure 8.3, and Figure 8.2 provide a visual representation of the final hardware architecture as implemented in Vivado. These diagrams confirm the successful integration of all system components, from the top-level SoC (Figure 8.1) down to the specific connections for DDR memory (Figure 8.3) and I/O peripherals like UART and Ethernet (Figure 8.2).

The system’s memory-mapped architecture is confirmed by the Address Editor view in Figure 8.4, which shows the memory ranges assigned to the DDR memory controller, the Gemmini DMA interface, and various peripheral buses. This confirms that the diplomacy framework within Chipyard correctly generated the necessary hardware and address decoding logic.

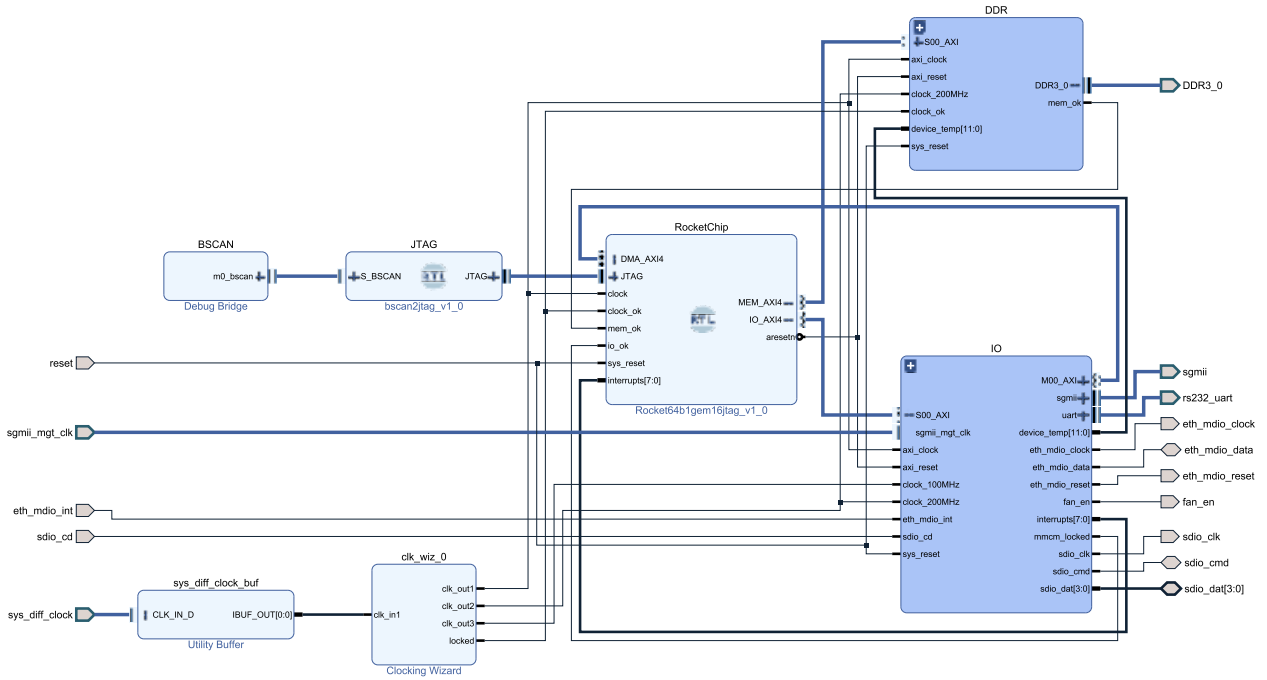


Figure 8.1: Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems.

8.1.2 Vivado Implemented Design Results

8.1.2.1 Device Utilization Analysis

Resource utilization is a critical metric for evaluating the cost and feasibility of an SoC implementation on a given FPGA. The post-implementation utilization report from Vivado provides a detailed accounting of the hardware resources consumed by the design.

As shown in Figure 8.6, the complete, Linux-capable SoC is a substantial design, consuming **80.92%** of the available Look-Up Tables (LUTs), **28.59%** of Block RAM (BRAM), and **8.96%** of DSP slices on the Xilinx VC707's Virtex-7 FPGA. The high LUT utilization is expected given the complexity of a 64-bit processor core with an L2 cache and numerous peripherals. The significant BRAM usage is primarily attributable to the memory-intensive components of the design, namely the L2 cache and the Gemmini accelerator's scratchpad and accumulator memories.

To isolate the resource cost of the Gemmini accelerator itself, a hierarchical utilization analysis was performed. Figure 8.8 shows that the Gemmini module ('riscv_tile/riscv_tile/Queue_Gemmini') consumes 180,142 LUTs. This represents approximately 73.5% of the logic within the 'riscv_tile' and underscores the fact that the specialized accelerator is, by far, the largest single component in the processor

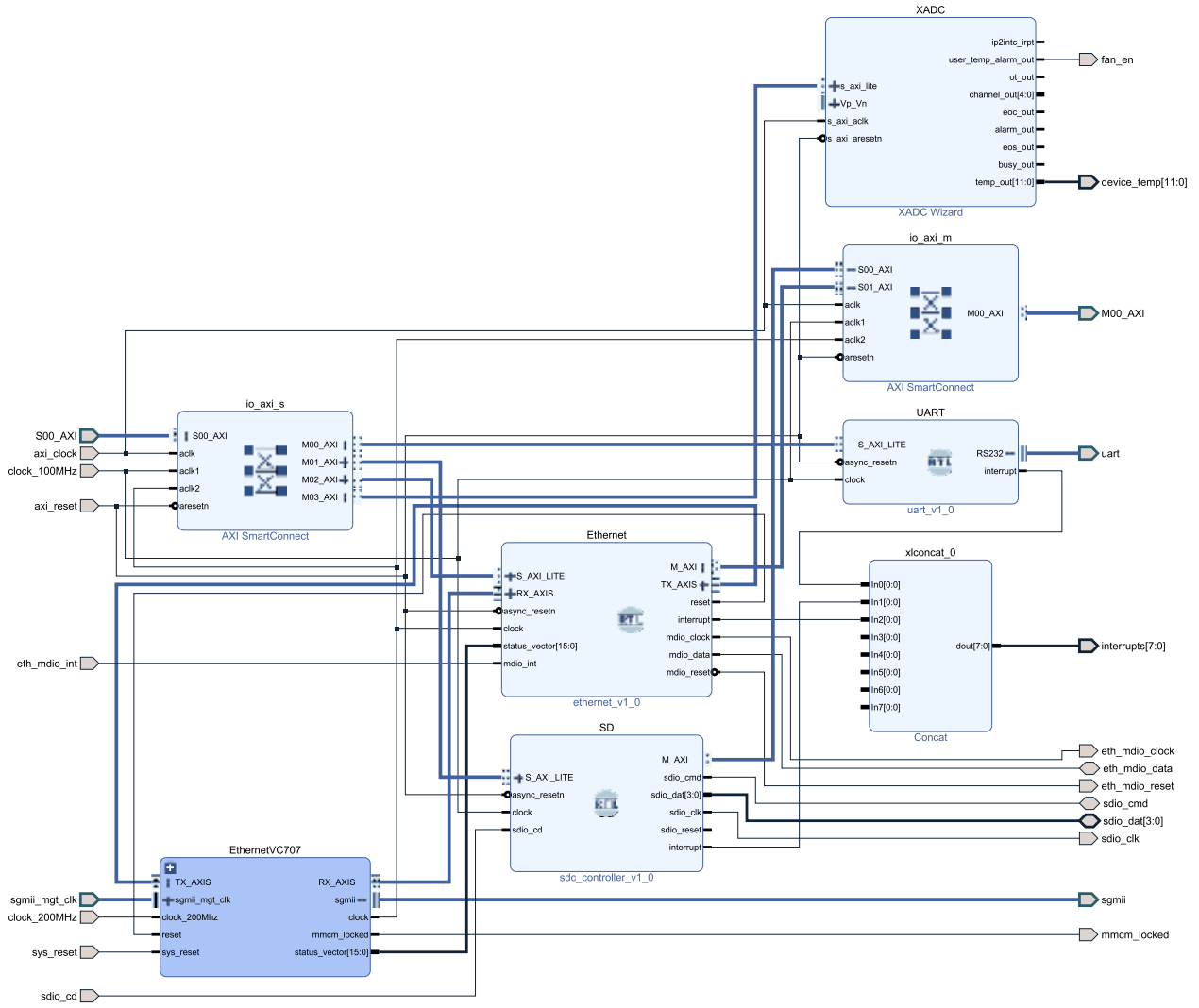


Figure 8.2: Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART.

tile, dwarfing the RISC-V core itself. This trade-off of silicon area for specialized performance is the central premise of hardware acceleration.

8.1.2.2 Power Consumption Analysis

Power analysis, performed by Vivado based on the placed-and-routed design and simulation-derived activity factors, is crucial for understanding the thermal and power-delivery requirements of the system.

The power summary in Figure 8.10 reports a **Total On-Chip Power of 3.856W**. A key observation is the breakdown between static and dynamic power. Dynamic power, consumed by switching logic, accounts for 3.309W (86% of the total), while static power is only 0.237W (6%). This is characteristic of a high-performance digi-

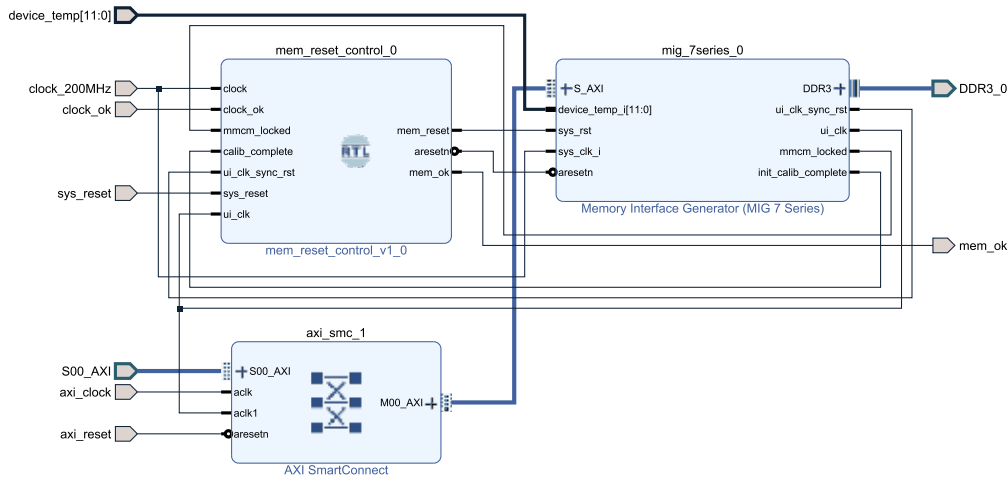


Figure 8.3: Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection.

Diagram × Address Editor × Address Map ×						
☒ Assigned (7) ☒ Unassigned (0) ☒ Excluded (0) Hide All						
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address	
Network 0						
IO/Ethernet						
IO/Ethernet/M_AXI (34 address bits : 16G)						
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF	
IO/SD						
IO/SD/M_AXI (34 address bits : 16G)						
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF	
Network 1						
/RocketChip						
/RocketChip/IO_AXI4 (31 address bits : 2G)						
IO/Ethernet/S_AXI_LITE	S_AXI_LITE	reg0	0x6002_0000	64K	0x6002_FFFF	
IO/SD/S_AXI_LITE	S_AXI_LITE	reg0	0x6000_0000	64K	0x6000_FFFF	
IO/UART/S_AXI_LITE	S_AXI_LITE	reg0	0x6001_0000	64K	0x6001_FFFF	
IO/XADC/s_axi_lite	s_axi_lite	Reg	0x6003_0000	64K	0x6003_FFFF	
Network 2						
/RocketChip						
/RocketChip/MEM_AXI4 (34 address bits : 16G)						
DDR/mig_7series_0/memmap	S_AXI	memaddr	0x0	16G	0x3_FFFF_FFFF	

Figure 8.4: Address Editor view, confirming the memory map for the integrated peripherals.

tal design where power consumption is dominated by computational activity rather than leakage. The report identifies that the most significant contributors to dynamic power are Clocks, Signals, and I/O, which is expected for a large, high-frequency SoC with multiple external interfaces. The power hierarchical report (Figure 8.12) further shows that the Gemini module contributes significantly to the overall power budget, consistent with its large area and high level of computational activity.

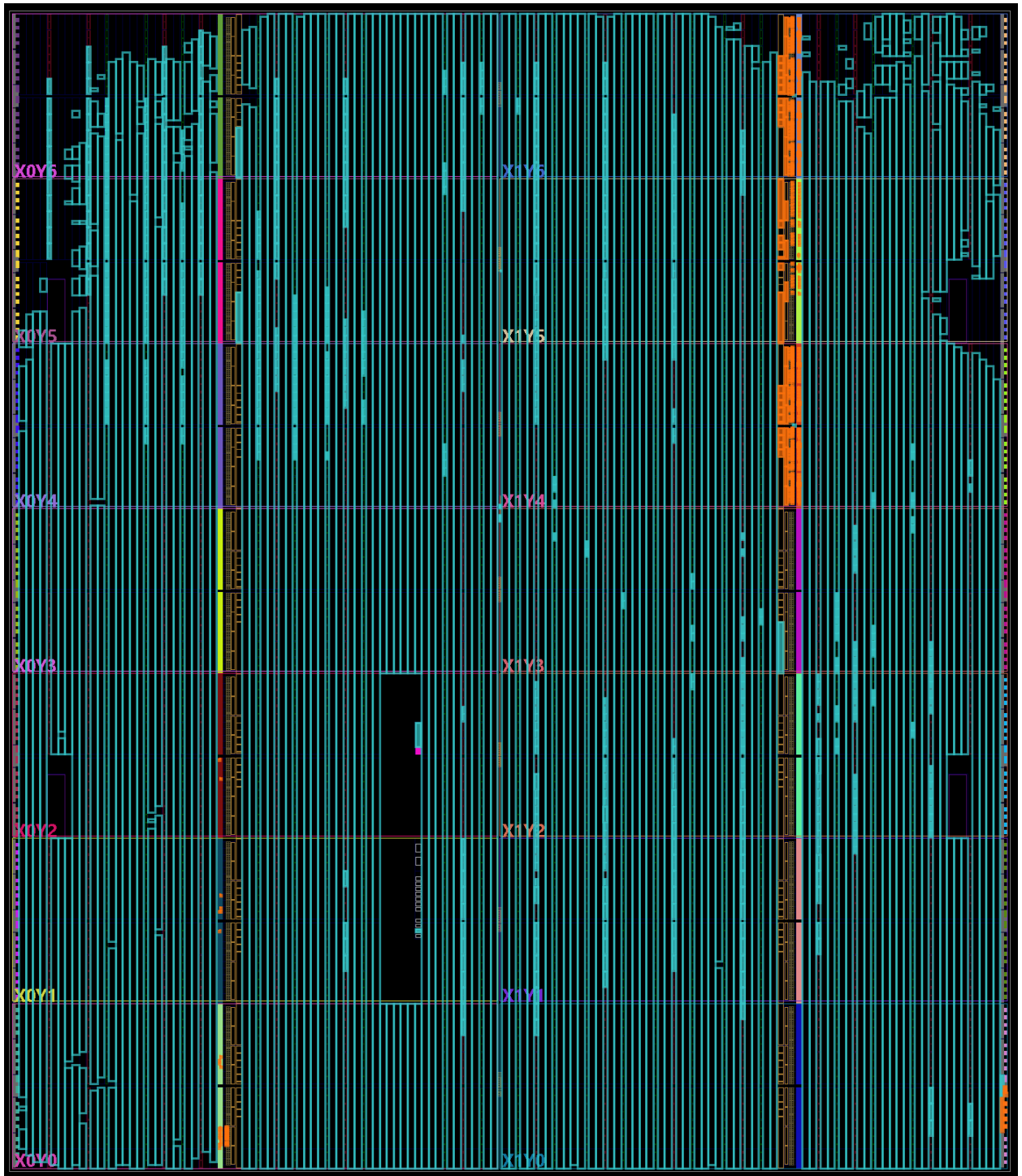


Figure 8.5: Device View of the implemented design on the FPGA fabric.

Summary

Resource	Utilization	Available	Utilization %
LUT	245673	303600	80.92
LUTRAM	9365	130800	7.16
FF	106058	607200	17.47
BRAM	294.50	1030	28.59
DSP	251	2800	8.96
IO	133	700	19.00
GT	1	28	3.57
MMCM	4	14	28.57
PLL	1	14	7.14

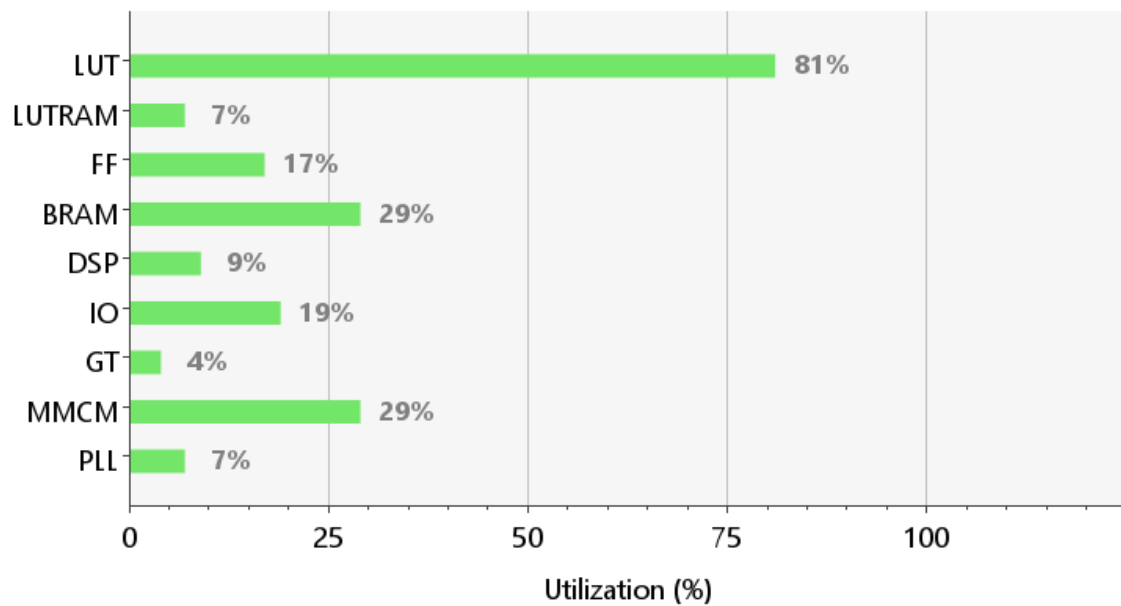


Figure 8.6: Detailed Utilization Summary for the entire SoC on the VC707 FPGA.

Name	Slice LUTs (303600)	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Slice (75900)	LUT as Logic (303600)	LUT as Memory (130800)	Block RAM Tile (1030)	DSPs (2800)
▼ riscv_wrapper	245673	106058	3692	791	71517	236308	9365	294.5	251
▼ riscv_i (riscv)	245673	106058	3692	791	71517	236308	9365	294.5	251
> BSCAN (riscv_BSCAN_0)	125	125	0	0	46	125	0	0	0
> clk_wiz_0 (riscv_clk_wiz_0_0)	0	0	0	0	0	0	0	0	0
> DDR (DDR_imp_10I4PB3)	13893	12217	371	8	4991	10769	3124	0	0
> IO (IO_imp_44488Y)	7549	9921	22	6	3501	6494	1055	6.5	0
> JTAG (riscv_JTAG_0)	28	27	0	0	14	28	0	0	0
> RocketChip (riscv_RocketChip_0)	224082	83768	3299	777	64577	218896	5186	288	251
> sys_diff_clock_buf (riscv_sys_diff_clock_buf_0)	0	0	0	0	0	0	0	0	0

Figure 8.7: Hierarchical Utilization, showing the breakdown of resources by major system modules.

Name	Slice LUTs	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Slice (75900)	LUT as Logic (303600)	LUT as Memory (130800)	Block RAM Tile (1030)	DSPs (2800)
▼ riscv_wrapper	245673	106058	3692	791	71517	236308	9365	294.5	251
▼ riscv_i (riscv)	245673	106058	3692	791	71517	236308	9365	294.5	251
▼ RocketChip (riscv_RocketChip_0)	224082	83768	3299	777	64577	218896	5186	288	251
▼ inst (riscv_RocketChip_0_Rocket64b1gem16jtag)	224082	83750	3299	777	64549	218896	5186	288	251
▼ rocket_system (riscv_RocketChip_0_RocketSy	224075	83716	3299	777	64547	218889	5186	288	251
▼ tile_prci_domain (riscv_RocketChip_0_TileP	206701	77251	1867	222	60131	204046	2655	152	251
▼ tile_reset_domain_tile (riscv_RocketChig	206173	77229	1867	222	59988	203913	2260	152	251
> v (riscv_RocketChip_0_Gemmini)	180142	64040	678	110	52531	178136	2006	128	236

Figure 8.8: Hierarchical Utilization of the Gemmini accelerator module.

Settings

Device

Part: xc7vx485tffg1761-2
Temp grade: commercial
Process: typical
Characterization: Production

Environment

Output Load: 0 pF
Ambient temperature: 25.0 °C
Airflow: 250 LFM
Heat sink: medium (Medium Profile)
 θ_{SA} : 2.2 °C/W
Board selection: medium (10"x10")
Number of board layers: 12to15 (12 to 15 Layers)
 θ_{JB} : 2.1 °C/W
Board temperature: 25.0 °C

Figure 8.9: Power analysis settings.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 3.856 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 29.4°C
 Thermal Margin: 55.6°C (47.0 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 1.1°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

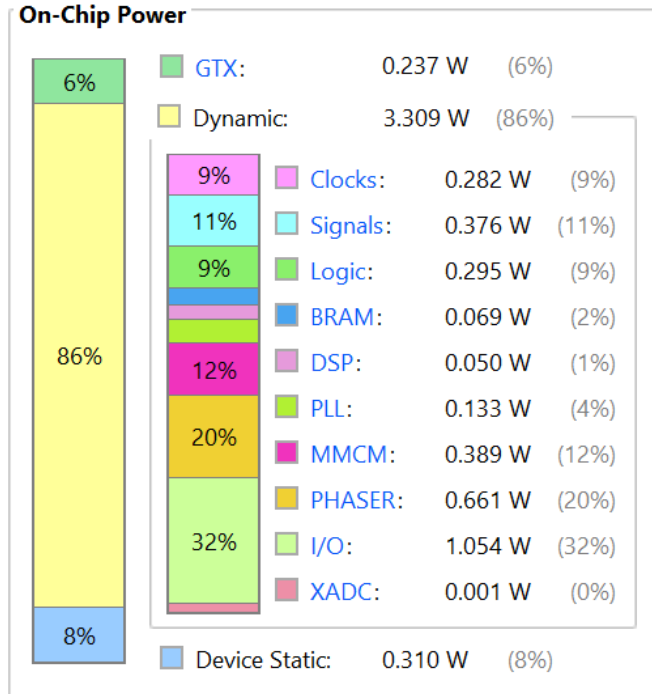


Figure 8.10: Post-implementation on-chip power summary.

Utilization	Name	Clocks (W)	Signals (W)	Data (W)	Clock Enable (W)	Set/Reset (W)	Logic (W)	BRAM (W)	DSP (W)
3.546 W (92% of total)	riscv_wrapper								
3.546 W (92% of total)	riscv_i (riscv)	0.282	0.375	0.363	0.01	0.002	0.295	0.069	0.05
2.228 W (58% of total)	DDR (DDR_imp_10I4PB3)	0.113	0.037	0.033	0.002	0.002	0.02	<0.001	<0.001
0.799 W (21% of total)	RocketChip (riscv_RocketChip_0)	0.106	0.325	0.318	0.007	<0.001	0.264	0.055	0.05
0.444 W (12% of total)	IO (IO_imp_44488Y)	0.061	0.013	0.012	0.002	<0.001	0.011	0.015	<0.001
0.069 W (2% of total)	clk_wiz_0 (riscv_clk_wiz_0_0)	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
0.004 W (<1% of total)	sys_diff_clock_buf (riscv_sys_diff_clock_buf_0)	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
0.001 W (<1% of total)	BSCAN (riscv_BSCAN_0)	0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
<0.001 W (<1% of total)	JTAG (riscv_JTAG_0)	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
<0.001 W (<1% of total)	Leaf Cells (17)								

Figure 8.11: Hierarchical power report for the entire system.

Q ≡ Hierarchical	
Utilization	Name
▼ 3.546 W (92% of total)	N riscv_wrapper
▼ 3.546 W (92% of total)	I riscv_i (riscv)
> 2.228 W (58% of total)	I DDR (DDR_imp_10J4PB3)
▼ 0.799 W (21% of total)	I RocketChip (riscv_RocketChip_0)
▼ 0.798 W (21% of total)	I inst (riscv_RocketChip_0_Rocket64b1gem16jtag
▼ 0.798 W (21% of total)	I rocket_system (riscv_RocketChip_0_RocketSyste
▼ 0.751 W (19% of total)	I tile_prci_domain (riscv_RocketChip_0_TilePRCIDc
▼ 0.749 W (19% of total)	I tile_reset_domain_tile (riscv_RocketChip_0_Rocke
> 0.67 W (17% of total)	I v (riscv_RocketChip_0_Gemmini)
> 0.036 W (1% of total)	I fpuOpt (riscv_RocketChip_0_FPU)
> 0.015 W (1% of total)	I frontend (riscv_RocketChip_0_Frontend)
> 0.014 W (1% of total)	I core (riscv_RocketChip_0_Rocket)
> 0.01 W (<1% of total)	I dcache (riscv_RocketChip_0_DCache)
> 0.003 W (<1% of total)	I ptw (riscv_RocketChip_0_PTW)
> <0.001 W (<1% of total)	I cmdRouter (riscv_RocketChip_0_RoccCommand
> <0.001 W (<1% of total)	I respArb_io_in_0_q (riscv_RocketChip_0_Queue_1
> <0.001 W (<1% of total)	I tlMasterXbar (riscv_RocketChip_0_TLXbar_8)
> <0.001 W (<1% of total)	I dcacheArb (riscv_RocketChip_0_HellaCacheArbit
> 0.002 W (<1% of total)	I buffer (riscv_RocketChip_0_TLBuffer_16)
> <0.001 W (<1% of total)	I intsink (riscv_RocketChip_0_IntSyncAsyncCrossin

Figure 8.12: Hierarchical power report for the Gemmini accelerator.

8.2 Software Stack Validation

A primary objective of this thesis was to implement a full-stack, Linux-capable system. This section provides evidence of this successful implementation by documenting the complete boot process, from the initial on-chip Boot ROM to a fully functional Debian Linux user environment.

8.2.1 System Boot Sequence

The system boot process involves a multi-stage software chain, starting with the first-stage bootloader in ROM, which then loads subsequent stages from the SD card. The console output in Listing 8.1 captures this entire sequence.

The process begins with the ‘RISC-V 64, Boot ROM’ identifying the hardware and proceeding to load the next stage, OpenSBI, which is the ‘boot.elf’ file on the SD card. OpenSBI (Open Source Supervisor Binary Interface) successfully initializes, reporting platform details such as the HART (Hardware Thread) count and timer frequency. It then loads and jumps to the U-Boot bootloader. U-Boot initializes the DRAM and other system hardware before loading the Linux kernel (‘image-v6.11.10’) and the Flattened Device Tree (FDT) blob into memory. Finally, the message ‘Starting kernel ...’ marks the successful handoff to the Linux operating system, which proceeds to boot, initialize its drivers, and present a login prompt.

```
RISC-V 64, Boot ROM V3.8
```

```
OpenSBI v1.6
```

```

  ----
 /  _  \
| | | | | _ _ _ _ _ | ( _ _ | | _ ) | | |
| | | | | ' _ \ / _ \ ' _ \ \ _ _ \ | _ < | |
| | _ | | | _ ) | _ _ / | | | _ _ _ ) | | _ | | _
 \ _ _ _ / | . _ _ / \ _ _ | _ | | _ _ _ _ / | _ _ _ / _ _ _ |
      | |
      | _ |
```

```
Platform Name           : freechips,rocketchip-vivado
```

```
...
```

```
(Abridged OpenSBI and U-Boot output for brevity)
```

```
...
```



```
## Flattened Device Tree blob at 00010080
   Booting using the fdt blob at 0x010080
   Loading Device Tree to 0000000084ffc000, end
   0000000084fffe3f ... OK

Starting kernel ...

[    0.000000] Linux version 6.11.10-dirty (btdat@btdat-hpZ4)
...
[    0.000000] Machine model: freechips,rocketchip-vivado
[    0.000000] SBI specification v2.0 detected
```

Listing 8.1: Console output capturing the full boot sequence from Boot ROM to the Linux kernel.

8.2.2 Debian Linux Environment

After booting, the system provides a standard Debian GNU/Linux environment. Listing 8.2 confirms this by showing a successful login and the output of standard Linux commands. The ‘uname -a’ command verifies the kernel version and the ‘riscv64’ architecture. Critically, the output of ‘cat /proc/cpuinfo’ confirms the processor’s identity as ‘sifive,rocket0’ and details its ISA as ‘rv64imafdc_...’, matching the design specification. This successful boot to a full-featured OS validates the entire hardware and software co-design process.

```
Debian GNU/Linux trixie/sid debian ttyAU0

debian login: debian
Password:
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
riscv64
...
debian@debian:~$ uname -a
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
riscv64 GNU/Linux
debian@debian:~$ cat /proc/cpuinfo
processor       : 0
hart           : 0
isa            : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
```

```
mmu          : sv39
uarch        : sifive,rocket0
mvendorid    : 0x0
marchid      : 0x1
mimpid       : 0x20181004
hart isa     : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
```

Listing 8.2: Demonstration of a functional Debian Linux environment on the implemented SoC.

8.3 Gemmini Accelerator Performance Evaluation

With the SoC’s correctness and stability confirmed, the final and most important evaluation is to measure the performance benefit of the Gemmini accelerator. The evaluation consists of two parts: verifying its functional correctness and quantifying its performance speedup on a representative workload.

8.3.1 Functional Correctness

To verify that the accelerator works correctly, a test was run that performs a matrix multiplication operation (‘In * Identity = Out’) and then compares the resulting ‘Out’ matrix with the original ‘In’ matrix. The test involves a series of memory operations managed by Gemmini: moving data from main memory to the on-chip scratchpad, executing the computation, and moving the result back to main memory.

The console output in Listing 8.3 shows the steps of the test. The final line, **‘Input and output matrices are identical, as expected’**, provides conclusive proof that the entire accelerator pipeline is functioning correctly. This includes the RoCC instruction dispatch from the CPU, the DMA transfers managed by Gemmini’s internal controllers, the systolic array computation, and the memory coherency with the CPU.

```
debian@debian:~/gemmini/build/bareMetalC$ ./template-linux
Flush Gemmini TLB of stale virtual addresses
Initialize our input and output matrices in main memory
Calculate the scratchpad addresses of all our matrices
    Note: The scratchpad is "row-addressed", where each address
          contains one matrix row
Move "In" matrix from main memory into Gemmini's scratchpad
```

```

Move "Identity" matrix from main memory into Gemmini's
    scratchpad
Multiply "In" matrix with "Identity" matrix with a bias of 0
Move "Out" matrix from Gemmini's scratchpad into main memory
Fence till Gemmini completes all memory operations
Check whether "In" and "Out" matrices are identical
Input and output matrices are identical, as expected

```

Listing 8.3: Verification of the Gemmini accelerator's functional correctness.

8.3.2 Performance Speedup

The primary motivation for this work is to accelerate DNN workloads. To quantify this, a convolution operation was performed twice: once using a C-language implementation running on the RISC-V CPU (software-only), and once using the Gemmini accelerator. The number of clock cycles for each execution was measured.

The results, shown in Listing 8.4, are definitive. The software-only implementation required **2,491,745 cycles** to complete the convolution. The same operation, when offloaded to the Gemmini accelerator, took only **2,600 cycles**.

$$\text{Speedup} = \text{CPU Cycles} / \text{Gemmini Cycles} = 2,491,745 / 2,600 \approx 958x$$

This result demonstrates a staggering **958-fold speedup**. This profound performance gain is the direct result of the architectural principles discussed in Chapter 4, namely the massive parallelism of the systolic array and the data reuse it enables, which effectively circumvents the memory wall bottleneck that cripples the general-purpose CPU. This quantitative result serves as the ultimate validation of this thesis's central hypothesis: that a tightly-coupled, specialized hardware accelerator, implemented through a full-stack, open-source methodology, provides a dramatic and essential performance uplift for modern computational workloads.

```

Input dimensions: 17 by 17
Output dimensions: 9 by 9

Randomize inputs...
Randomize weights...
Randomize bias...
CPU conv...
CPU conv took 2491745 cycles

```

```
Flatten weights...
Gemmini conv...
Gemmini conv took 2600 cycles
```

Listing 8.4: Cycle counts for a convolution operation on the CPU vs. the Gemmini accelerator.

Chapter 9: Conclusion and Future Work

This thesis successfully evaluated the Chipyard framework, demonstrating its superiority over traditional Quartus-based flows for our laboratory’s objectives due to its integrated VLSI toolchain, support for industry-standard AXI4 interconnects, and its open-source nature.

DANH MỤC CÔNG TRÌNH CỦA TÁC GIẢ

TÀI LIỆU THAM KHẢO

References

- [1] Alon Amid et al. “Chippyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs”. In: *IEEE Micro* 40.4 (2020), pp. 10–21. ISSN: 1937-4143. DOI: 10.1109/MM.2020.2996616.
- [2] Krste Asanović et al. *The Rocket Chip Generator*. Tech. rep. UCB/EECS-2016-17. Electrical Engineering and Computer Sciences, University of California, Berkeley, 2016. URL: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html>.
- [3] Cactus Technologies. *An Introduction To SD Card Interface*. White Paper CTWP009. Suite C, 15/F, Capital Trade Center, 62 Tsun Yip Street, Kwun Tong, Kowloon, Hong Kong: Cactus Technologies Limited. URL: <https://www.cactus-tech.com/wp-content/uploads/2019/03/An-Introduction-To-SD-Card-Interface.pdf>.
- [4] C. Celio, D. Patterson, and K. Asanovic. *The Berkeley Out-of-Order Machine (BOOM) Design Specification*. University of California, Berkeley. 2016. URL: <https://github.com/ccelio/riscv-boom-doc/raw/gh-pages/boom-spec.pdf>.
- [5] C. Celio, D. A. Patterson, and K. Asanovic. *The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor*. Tech. Rep. UCB/EECS-2015-167. EECS Department, University of California, Berkeley, 2015. URL: <https://www.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-167.html>.
- [6] Christopher Celio. *A Highly Productive Implementation of an Out-of-Order Processor Generator*. Tech. rep. UCB/EECS-2018-151. Electrical Engineering and Computer Sciences, University of California at Berkeley, 2018. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-151.html>.
- [7] Farzad Farshchi, Qijing Huang, and Heechul Yun. “Integrating NVIDIA Deep Learning Accelerator (NVDLA) with RISC-V SoC on FireSim”. In: *2019 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2)*. 2019, pp. 21–25. DOI: 10.1109/EMC249363.2019.00012.

- [8] Alex Forencich. *Verilog Ethernet Components*. GitHub Repository. Accessed: July 2025. 2023. URL: <https://github.com/alexforencich/verilog-ethernet>.
- [9] Hasan Genc et al. *Gemmini @ MLSys 2022*. Presentation Slides, MLSys 2022 Tutorial. Accessed: [Your Access Date]. 2022. URL: <https://mlsys.org/virtual/2022/tutorial/18433>.
- [10] Hasan Genc et al. “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration”. In: *Proceedings of the 58th Annual ACM/IEEE Design Automation Conference*. IEEE Press, 2022, pp. 769–774. ISBN: 9781665432740. URL: <https://doi.org/10.1109/DAC18074.2021.9586216>.
- [11] Dennis Agyemanh Nana Gookyi et al. “Deep Learning Accelerators’ Configuration Space Exploration Effect on Performance and Resource Utilization: A Gemmini Case Study”. In: *Sensors* 23.5 (2023). ISSN: 1424-8220. DOI: 10.3390/s23052380. URL: <https://www.mdpi.com/1424-8220/23/5/2380>.
- [12] Mark Horowitz. “1.1 Computing’s energy problem (and what we can do about it)”. In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323.
- [13] Norman P. Jouppi et al. “In-Datacenter Performance Analysis of a Tensor Processing Unit”. In: *SIGARCH Comput. Archit. News* 45.2 (June 2017), pp. 1–12. ISSN: 0163-5964. DOI: 10.1145/3140659.3080246. URL: <https://doi.org/10.1145/3140659.3080246>.
- [14] Binh Kieu-do-Nguyen et al. “Hardware Software Co-Design for Multi-Threaded Computation on RISC-V-Based Multicore System”. In: *IEEE Access* 12 (2024), pp. 177312–177326. DOI: 10.1109/ACCESS.2024.3505940.
- [15] Kung. “Why systolic architectures?” In: *Computer* 15.1 (1982), pp. 37–46. DOI: 10.1109/MC.1982.1653825.
- [16] Sparsh Mittal and Sumanth Umesh. “A survey On hardware accelerators and optimization techniques for RNNs”. In: *Journal of Systems Architecture* 112 (2021), p. 101839. ISSN: 1383-7621. DOI: <https://doi.org/10.1016/j.sysarc.2020.101839>. URL: <https://www.sciencedirect.com/science/article/pii/S1383762120301314>.

- [17] RISC-V International. *Members*. Accessed: Jun. 16, 2025. 2025. URL: <https://riscv.org/members/>.
- [18] SiFive. *A Winning Processor Portfolio*. Accessed: Dec. 30, 2024. 2024. URL: <https://www.sifive.com/risc-v-core-ip>.
- [19] SiFive Inc. *SiFive TileLink Specification*. Version 1.7.1. Accessed: Jun. 6, 2025. Dec. 2018. URL: https://sifive.cdn.prismic.io/sifive%2F57f93ecf-2c42-46f7-9818-bcdd7d39400a_tilelink-spec-1.7.1.pdf.
- [20] Vivienne Sze et al. *Efficient processing of deep neural networks*. Jan. 2020. DOI: 10.1007/978-3-031-01766-7. URL: <https://doi.org/10.1007/978-3-031-01766-7>.
- [21] A. Waterman et al. *The RISC-V instruction set manual volume II: Privileged architecture version 1.7*. Tech. Rep. UCB/EECS-2015-49. EECS Department, University of California, Berkeley, 2015.
- [22] Andrew Waterman and Krste Asanović, eds. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Document Version 20240411. RISC-V Foundation. Apr. 2024. URL: <https://github.com/riscv/riscv-isa-manual/releases/download/20240411/unpriv-isa-asciidoc.pdf>.
- [23] Wm. A. Wulf and Sally A. McKee. “Hitting the memory wall: implications of the obvious”. In: *SIGARCH Comput. Archit. News* 23.1 (Mar. 1995), pp. 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588. URL: <https://doi.org/10.1145/216585.216588>.
- [24] J. Zhao. *Chipyard Intro and Fundamentals*. Presentation Slides, ISCA 2021 Tutorial. Accessed: Jun. 6, 2025. 2021. URL: https://fires.im/isca21-slides-pdf/02_chipyard_basics.pdf.
- [25] Jerry Zhao et al. “SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine”. In: *2020 Workshop on Computer Architecture Research with RISC-V (CARRV)*. IEEE, 2020. DOI: 10.1109/CARRV51705.2020.9408173.